Resumen

Resumen

Palabras clave: interfaz cerebro-computador, actividad neuronal intracortical, decodificación neuronal, OWSM, aprendizaje por transferencia, CTC

Abstract

Abstract

Key words: brain–computer interface, intracortical neural activity, neural decoding, OWSM, transfer learning, CTC

Índice general

Índice general III
Índice de figuras VII
Índice de tablas VIII

1 Introducción 1
1.1 Introducción . . . 1
1.1.1 Contexto . . . 1
1.1.2 Del cuerpo al cerebro: evolución de los sistemas de comunicación . . . 1
1.2 Motivación . . . 3
1.2.1 Motivación . . . 3
2 Planteamiento del trabajo 5
2.1 Planteamiento del trabajo . . . 5
2.2 Objetivos . . . 6
2.2.1 Objetivos específicos . . . 6
2.3 Alcance y limitaciones . . . 6
2.4 Estructura de la memoria . . . 7
3 Estado del arte 9
3.1 Modalidades de registro en interfaces cerebro-computador . . . 9
3.2 Evolución de las neuroprótesis del habla . . . 10
3.3 Anatomía de un decodificador *brain-to-text* . . . 11
3.3.1 El cambio entre días . . . 11
3.3.2 El codificador . . . 11
3.3.3 La salida y el modelo de lenguaje . . . 11
3.4 El *benchmark* Brain-to-Text '25 . . . 12
3.5 Modelos de habla preentrenados y transferencia a la señal neuronal . . . 12
3.6 Comparativa de los sistemas revisados . . . 12
3.7 Posicionamiento de este trabajo . . . 13
4 Conjunto de datos 15
4.1 Selección del conjunto de datos . . . 15
4.2 El conjunto Brain-to-Text '25 . . . 16
4.2.1 Origen y participante . . . 16
4.2.2 Adquisición de la señal . . . 16
4.2.3 Organización de los datos . . . 17
4.2.4 Particiones y composición . . . 18
4.3 Análisis exploratorio . . . 18
5 Marco teórico 21
5.1 Planteamiento del problema . . . 21
5.2 Datos de entrada al modelo . . . 21
5.2.1 Lotes, relleno y máscaras . . . 21
5.2.2 Capa de entrada por sesión . . . 22
5.3 *Frontend* . . . 22
5.4 OWSM . . . 24

5.4.1 Origen y versiones . . . 24
5.4.2 Arquitectura . . . 24
5.4.3 Atención . . . 25
5.4.4 Codificador E-Branchformer . . . 25
5.4.5 Decodificador Transformer . . . 26
5.5 Clasificación temporal conexionista (CTC) . . . 27
5.5.1 Formulación y cabeza CTC . . . 27
5.5.2 Pérdida conjunta e InterCTC . . . 28
5.5.3 Decodificación . . . 29
5.6 Adaptación de un modelo preentrenado . . . 29
5.6.1 LoRA . . . 30
5.7 Entrenamiento . . . 30
5.8 Métricas de evaluación . . . 31
5.8.1 Exactitud de ESPnet . . . 31
5.8.2 Tasa de error por palabra . . . 31
6 Metodología **33**
6.1 Adaptación de OWSM a la señal neuronal . . . 33
6.2 Evaluación . . . 34
6.3 Fase 1: búsqueda del mejor modelo . . . 34
6.3.1 Puesta en marcha . . . 35
6.3.2 Codificador (OE2) . . . 36
6.3.3 *Frontend* (OE1) . . . 36
6.3.4 Decodificador (OE3) . . . 37
6.4 Fase 2: comportamiento en el tiempo y cantidad de datos . . . 38
6.4.1 Variabilidad entre sesiones (OE4) . . . 38
6.4.2 Entrenamiento cronológico (OE4) . . . 38
6.4.3 Análisis del modelo . . . 38
7 Resultados y discusión **41**
7.1 Fase 1: búsqueda del mejor modelo . . . 41
7.1.1 Codificador . . . 41
7.1.2 *Frontend* . . . 42
7.1.3 Capa de entrada por sesión . . . 44
7.1.4 Decodificador . . . 45
7.1.5 Resumen de la fase 1 . . . 46
7.2 Fase 2: comportamiento en el tiempo . . . 46
7.2.1 Variabilidad entre sesiones . . . 46
7.2.2 Entrenamiento cronológico . . . 47
7.2.3 Análisis del modelo . . . 49
7.3 Comparación con el estado del arte . . . 52
7.4 Discusión . . . 52
7.4.1 Resultados por objetivo . . . 52
7.4.2 Una lectura conjunta . . . 53
8 Limitaciones y trabajo futuro **55**
8.1 Limitaciones . . . 55
8.1.1 Un solo participante y un solo modelo . . . 55
8.1.2 Fiabilidad de las cifras . . . 55
8.1.3 Recursos . . . 55
8.1.4 Distancia al uso real . . . 56
8.2 Trabajo futuro . . . 56
8.2.1 Mejorar el modelo . . . 56
8.2.2 Usar fonemas . . . 56

8.2.3 Mejorar el decodificador . . . 57
8.2.4 Más datos . . . 57
8.2.5 Adaptarse a los días nuevos . . . 57
8.2.6 Entender qué se transfiere . . . 58
8.2.7 Hacia el uso real . . . 58

9 Conclusiones **59**
9.1 Respuesta a los objetivos . . . 59
9.2 Aportaciones . . . 60
9.3 Lecciones aprendidas . . . 60
9.4 Relación con los estudios cursados . . . 60
9.5 Legado . . . 61
9.6 Valoración final . . . 61

Bibliografía **63**

Apéndices

A Código del pipeline de entrenamiento **69**

B Análisis exploratorio adicional **71**

C Relación del trabajo con los Objetivos de Desarrollo Sostenible de la Agenda 2030 **73**

Índice de figuras

4.1 Esquema de una interfaz cerebro-texto con microelectrodos 17
4.2 Número de palabras por frase 19
4.3 Repetición de frases y solape entre particiones 19
4.4 Frecuencia de las palabras en el entrenamiento 20
4.5 Número de frases por sesión 20

5.1 Esquema del modelo 25
5.2 Alineamientos CTC válidos y no válidos 28

6.1 Prueba de sobreajuste: WER por época al entrenar con un subconjunto muy pequeño de frases. El modelo llega a aprendérselas de memoria, lo que muestra que la adaptación funciona y que el modelo puede aprender la tarea. 35

7.1 Entrenamiento de los seis *frontends* durante 40 épocas: acc de validación por época (a), CER de la rama CTC en validación (b) y acc frente a las horas de entrenamiento en una L4 (c). Las líneas discontinuas son los *frontends* $\times 4$. 43
7.2 WER de cada *frontend* frente al coste: horas de entrenamiento en una L4 durante 40 épocas (a) y operaciones por segundo de señal del *frontend* y el codificador (b). Los círculos son los *frontends* $\times 2$ y los cuadrados, los $\times 4$. Abajo a la izquierda es mejor. 43
7.3 Comparación de decodificadores, todos con el codificador aleatorio: acc de validación (izquierda), acc de entrenamiento (centro) y CER de la rama CTC en validación (derecha). El punto marca la mejor época de cada uno. «Aleatorio» es el decodificador entrenado desde cero, con la tasa del *frontend* ($\times 1$) o con la del codificador ($\times 0{,}05$). 45
7.4 WER de cada sesión con el modelo de la fase 1 entrenado con todas las sesiones (*frontend* profundo $\times 4$). La línea discontinua es el WER de toda la validación (0,206) y las cruces marcan las sesiones sin frases de validación (1, 29, 33 y 34). 46
7.5 WER de los modelos entrenados con las k primeras sesiones, en las sesiones vistas y en las futuras. Las bandas son el intervalo de confianza del 95 %. La línea gris discontinua es el modelo de la fase 1 que vio todas las sesiones, evaluado sobre las mismas frases futuras. 47
7.6 WER en las cinco últimas sesiones (de la 41 a la 45), que son futuras para todos los modelos, según el número de sesiones de entrenamiento k. La línea discontinua es el modelo de la fase 1 que vio todas las sesiones. . . . 48
7.7 Curvas de aprendizaje de los modelos entrenados con las k primeras sesiones: acc en la reserva por época. 48
7.8 Reparto de las frases según su error en las sesiones vistas (a) y en las futuras (b), para cada número de sesiones de entrenamiento k. Las cifras dentro de las barras son el porcentaje de frases perfectas. 49
7.9 Acierto del modelo entrenado con todas las sesiones por clase de palabra (a) y según el número de letras de la palabra (b). En azul, las palabras de contenido; en verde, las funcionales. 50

7.10 Acierto en palabras de contenido (a), en palabras funcionales (b) y en la palabra clave de cada frase (c), en las sesiones vistas y en las futuras, según el número de sesiones de entrenamiento *k*. El cuadrado gris de la derecha es el modelo de la fase 1 que vio todas las sesiones. 51
7.11 Parecido medio en la pronunciación entre la palabra correcta y la que escribe el modelo en las sustituciones (100 % = misma pronunciación), en las sesiones vistas y en las futuras, según el número de sesiones de entrenamiento *k*. La línea gris clara es el parecido entre parejas de palabras tomadas al azar, y el cuadrado, el modelo de la fase 1 que vio todas las sesiones. 51

Índice de tablas

3.1 Comparación de las modalidades de registro usadas en BCI del habla. . . 10
3.2 Resumen de los sistemas revisados en este capítulo. 13

4.1 Comparación de los conjuntos de datos considerados. 16
4.2 Composición de Brain-to-Text '25 por corpus de origen 18

5.1 Ejemplo de cálculo del WER. *S*: sustitución; *I*: inserción. 32

6.1 *Frontends* comparados en la fase 1. 37

7.1 Comparación de las cuatro formas de tratar el codificador. WER y CER con la búsqueda en haz, WER con la decodificación CTC voraz y WER con la señal falsa (200 frases). 41
7.2 Comparación de *frontends* tras 40 épocas, con el codificador aleatorio y el decodificador congelado: WER con la búsqueda en haz y con la decodificación CTC voraz, acc de la mejor época en validación, parámetros del *frontend*, operaciones por segundo de señal del *frontend* y el codificador, y horas de entrenamiento en una L4. En negrita, los mejores WER y acc. . . 42
7.3 WER con la búsqueda en haz de tres *frontends* sin y con capa de entrada por sesión, con la misma receta que la comparación de *frontends*. La capa añade unos 11,8 millones de parámetros. 44
7.4 Comparación de decodificadores, con el codificador aleatorio y el *frontend* profundo $\times 2$: acc de la mejor época en validación, WER y CER con la búsqueda en haz, WER con la decodificación CTC voraz y WER con la señal falsa (200 frases). En negrita, el mejor valor de cada columna de error y acierto; en rojo, estimaciones. 45
7.5 Entrenamiento cronológico. Para cada modelo, entrenado con las *k* primeras sesiones: número de frases de validación y WER en las sesiones vistas y en las futuras, WER en esas mismas frases futuras del modelo de la fase 1 que vio todas las sesiones, y WER en toda la validación. 47

C.1 Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible. . 73

CAPÍTULO 1
Introducción

1.1 Introducción

1.1.1. Contexto

La comunicación oral es una capacidad básica para vivir en sociedad. Sin embargo, hay enfermedades que pueden quitarla casi por completo, aunque la persona siga pensando y entendiendo con normalidad, esta puede ser la causa de muchos trastornos. Se suelen dividir en disartria y anartria. La disartria es un trastorno motor del habla en el que se pierde el control de los músculos del tracto vocal, cuando la incapacidad de producir un habla que se entienda [1] es total, se le llama anartria. Puede aparecer con la esclerosis lateral amiotrófica (ELA), el ictus, el Parkinson, la parálisis cerebral, los traumatismos craneoencefálicos o algunos tumores cerebrales, y su gravedad es variable entre las personas [2]. En la afasia falla el propio lenguaje. En la disartria, en cambio, las zonas del cerebro que planifican el habla siguen funcionando: la persona sabe lo que quiere decir e intenta decirlo, pero la orden no llega bien a los músculos [1].

La ELA y el ictus son dos ejemplos claros de pérdida completa del habla, y también las enfermedades de la mayoría de las personas que participan en los estudios de la siguiente sección. La ELA va destruyendo las neuronas motoras de la corteza, el tronco del encéfalo y la médula espinal, y los músculos se van paralizando, también los que usamos para hablar, tragar y respirar [3]. Su prevalencia en el mundo es de 4,42 casos por cada 100.000 habitantes [4]. En España la tienen entre 4.000 y 4.500 personas y se diagnostican unos tres casos nuevos al día [5]. Entre el 80 % y el 95 % de los pacientes llegan a un punto en el que ya no se pueden comunicar hablando en su día a día [3]. El ictus es mucho más frecuente: en 2021 hubo 11,9 millones de casos nuevos en el mundo [6], en España hay unos 90.000 al año [7], y entre el 20 % y el 42 % de los supervivientes tienen disartria [8]. Cuando daña la parte ventral de la protuberancia, en el tronco del encéfalo, puede provocar el síndrome de cautiverio (*locked-in syndrome*). La persona queda consciente pero con tetraplejia y anartria, y en la forma clásica solo puede comunicarse parpadeando o moviendo los ojos en vertical [9], [10].

Aunque aquí se usen estos dos casos como ejemplo, lo que se busca en este trabajo puede servir a cualquier persona que conserve la intención de hablar y no pueda articularla del todo, sea cual sea la causa.

1.1.2. Del cuerpo al cerebro: evolución de los sistemas de comunicación

Desde hace décadas se han ido creando sistemas para que estas personas puedan comunicarse de otra forma. Los primeros eran tableros con letras o dibujos que la persona

señalaba, y máquinas electromecánicas de escribir que se manejaban con un pulsador o con algún otro movimiento pequeño. En los años sesenta y setenta estas ideas se juntaron y dieron lugar a lo que hoy se llama comunicación aumentativa y alternativa (CAA) [11]. Después llegaron los ordenadores y los sintetizadores de voz, con los que se escribe un mensaje y el equipo lo lee en voz alta. Son sistemas sencillos, no necesitan cirugía y muchos pacientes los usan hasta el final de la enfermedad [3]. Su problema es que son lentos y cansados, y que dependen de que quede algún movimiento voluntario con el que manejarlos [1].

Uno de los sistemas más usados en la ELA es el seguimiento ocular. Una cámara detecta hacia dónde mira la persona y así puede elegir letras o palabras en una pantalla. Se usa mucho en fases avanzadas, cuando ya no se puede hablar ni mover las manos, aunque solo permite una comunicación muy lenta o limitada [12] y en algunos sitios, como al aire libre, estos sistemas no funcionan [13]. Por último, cuando la enfermedad llega a los músculos de los ojos, dejan de servir [12].

Una evolución que se investiga dependiendo de los usuarios son los electrodos de electromiografía (EMG). Se colocan en la cara y el cuello y se puede reconocer lo que una persona intenta decir aunque no salga ningún sonido, lo que se conoce como habla silenciosa [14]. Es un método no invasivo y está más cerca del habla real que ir eligiendo letras. Pero cuanta más severa es la parálisis, menos precisión tienen, hasta el punto de que puede no haber ninguna señal.

Si no queda ningún músculo útil, solo se puede recurrir al propio cerebro. Las interfaces cerebro-computador (*Brain-Computer Interfaces*, BCI) registran su actividad y la convierten en órdenes para un dispositivo. Las primeras usaban electroencefalografía (EEG), con electrodos sobre el cuero cabelludo. En 1988, Farwell y Donchin presentaron un deletreador basado en la onda P300. En una pantalla se iluminan filas y columnas de letras y el sistema detecta la respuesta del cerebro cuando se ilumina la letra que el usuario quiere [15]. Ya en 1999, Birbaumer y su equipo consiguieron que pacientes totalmente paralizados por ELA escribieran mensajes controlando los potenciales corticales lentos de su EEG [16]. El EEG no necesita cirugía, pero la señal llega muy atenuada a través del cráneo y tiene mucho ruido, así que estos sistemas eran muy lentos, de unas pocas letras por minuto.

Para tener mejor señal hay que acercarse a las neuronas, y eso ya obliga a operar. La electrocorticografía (ECoG) coloca electrodos sobre la superficie de la corteza, por debajo del cráneo. En 2016 se publicó el caso de una paciente con ELA en estado de cautiverio que usaba en su casa una BCI de este tipo totalmente implantada. Al intentar mover la mano generaba un «clic» con el que elegía letras y, aunque era lento, le permitía comunicarse cuando su sistema ocular fallaba [13]. Además, esto permitió dar el salto de elegir letras a decodificar palabras directamente a partir del intento de hablar, primero con un vocabulario pequeño [17] y después con frases completas, voz sintetizada y un avatar [18]. La estereoelectroencefalografía (sEEG) usa en cambio electrodos de profundidad, que se implantan a pacientes con epilepsia para localizar el origen de sus crisis. Con estos registros se ha llegado a sintetizar voz en tiempo real a partir del habla imaginada, aunque el resultado todavía no se entendía [19]. Su ventaja es que aprovecha una operación que el paciente ya necesita, pero los electrodos se colocan donde dicta el criterio médico y solo se quedan durante un tiempo corto.

El último paso son las matrices de microelectrodos intracorticales, que penetran en la corteza y registran la actividad de pequeños grupos de neuronas. En 2006 el ensayo BrainGate mostró que un hombre con tetraplejia podía mover un cursor, abrir el correo o abrir y cerrar una mano protésica solo con la actividad de su corteza motora [20]. A partir de ahí se pasó a escribir moviendo un cursor sobre un teclado en pantalla [21], luego a convertir en texto la escritura a mano imaginada [22] y, por último, a decodificar

directamente el habla intentada [23]. Hoy ya hay un participante con ELA que lleva meses usando un sistema así para conversar en su día a día [24]. Con el mismo tipo de implante también se ha conseguido sintetizar la voz al instante, con entonación e incluso cantando melodías sencillas [25]. Son, con diferencia, los sistemas con mejores resultados. A cambio, exigen una operación cerebral y de momento solo los han probado unas pocas personas dentro de ensayos clínicos.

Si se mira toda esta evolución, se ve que cada paso ha aparecido porque el anterior dependía de algo que la enfermedad podía quitar: las manos, los músculos de la boca y los ojos. La actividad del cerebro es lo último que queda, y cuanto más cerca de las neuronas se registra, mejor es la señal pero más invasivo es el método. Las tecnologías BCI han tenido un auge en las últimas décadas y han pasado de controlar cursores o movimientos a poder decodificar lenguaje. Este trabajo se sitúa en esta última etapa, la de los sistemas *brain-to-text*, que traducen a texto la actividad de la corteza motora mientras la persona intenta hablar.

1.2 Motivación

1.2.1. Motivación

La motivación principal de este trabajo es social. Según la encuesta de discapacidad del INE de 2020, en España había unas 947.500 personas de seis años o más con una discapacidad relacionada con la comunicación [26]. No todas tienen las enfermedades descritas en la sección 1.1.1, pero la cifra da una idea de cuánta gente tiene problemas para hacerse entender. Y es un problema que está creciendo, ya que la Sociedad Española de Neurología prevé que los casos de ELA en Europa aumenten un 40 % en los próximos 25 años [5]. Quien no puede hablar depende de otros hasta para pedir lo más básico y, con el tiempo, se va quedando aislado [18].

La comunicación pesa mucho en la calidad de vida. En personas con ELA, contar con un sistema de comunicación desde las primeras fases tiene un efecto positivo en su bienestar y en el de sus cuidadores [27]. En España este problema empieza a tener más reconocimiento, por ejemplo con la Ley 3/2024, conocida como «Ley ELA», que busca mejorar la calidad de vida de estas personas y agilizar el reconocimiento de su discapacidad y su dependencia [28]. Sin embargo, aún no existe una verdadera solución al problema, y seguir investigando puede ayudar a muchas personas que lo necesitan.

Asimismo, me motiva desde el punto de vista técnico, porque las BCI del habla son un campo que avanza muy rápido. En unos veinte años se ha pasado de mover un cursor a decodificar frases completas y sintetizar voz en tiempo real. Estos sistemas ya superan la velocidad de las herramientas de comunicación asistida que existen hoy [1]. Cada poco tiempo se aplican a problemas nuevos [25], y se están consolidando como una de las mejores soluciones para la comunicación. Además, retos públicos como Brain-to-Text han puesto estos datos al alcance de cualquier grupo de investigación, aunque no tenga acceso a pacientes [29], [30]. El principal inconveniente sigue siendo que hay que operar el cerebro. Aun así, ya se están probando opciones menos invasivas, como electrodos que se introducen por un vaso sanguíneo sin abrir el cráneo [31], y en el futuro se espera que estas soluciones se apliquen cada vez más.

Por último, también hay una motivación personal. Siempre me ha interesado la biomedicina, y en especial cómo la tecnología puede ayudar a las personas con discapacidad. He participado en hackathons y, sobre todo, he formado parte durante tres años de Daidalonic, un grupo de estudiantes de la UPV que diseña prótesis y otros proyectos biomédicos. Allí trabajé con la EMG y pude hablar con personas con discapacidad. Pero además pude

ver de primera mano en el Cybathlon 2024 a muchos grupos de investigación utilizando las BCI para ayudar a personas completamente paralizadas, ya sea con la movilidad, el entretenimiento o la comunicación, y supe que quería investigar algo relacionado.

CAPÍTULO 2
Planteamiento del trabajo

2.1 Planteamiento del trabajo

Este trabajo estudia la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala como codificador de señal neuronal en una tarea *brain-to-text*, es decir, para pasar a texto la actividad del cerebro mientras una persona intenta hablar. En concreto, se adapta OWSM (*Open Whisper-style Speech Model*) [32], [33], de la librería ESPnet [34], [35]. OWSM sigue la idea de Whisper [36], pero se entrena solo con datos públicos y sus autores publican el código, los modelos y los registros de entrenamiento [33]. Por eso es una de las mejores opciones abiertas y reproducibles. La idea de fondo es aprovechar lo que estos modelos han aprendido de grandes cantidades de audio en un problema en el que los datos son muy escasos, porque cada grabación necesita a un paciente con un implante y muchas horas de entrenamiento.

La pregunta principal es qué hay que cambiar del modelo para que funcione con señal neuronal y si esos cambios hacen que la idea sea viable. La entrada de OWSM está pensada para audio, así que hay que sustituirla por un módulo nuevo, el *frontend*, que adapte la señal neuronal al resto del modelo. A partir de ahí se analiza qué partes se pueden aprovechar tal cual, cuáles hay que reentrenar y con qué nivel de ajuste. Este proyecto no se centra en la limpieza y el análisis de las señales neuronales, para las que se usa el preprocesado estándar del conjunto de datos, sino en la capacidad del modelo y sus resultados.

Además de la viabilidad, se quiere ver cómo se comporta el modelo en condiciones parecidas a las de un uso real. Por un lado, con palabras y frases que no ha visto durante el entrenamiento. Por otro, con el paso del tiempo. La actividad que registran los electrodos cambia de un día para otro, y para mantener el rendimiento estos sistemas suelen necesitar recalibraciones frecuentes, que obligan al usuario a dejar de usarlos y grabar datos nuevos [37]. Por eso se estudia cuántos datos hacen falta para tener un modelo que no necesite más, y qué se puede concluir sobre la necesidad de datos de este tipo de sistemas.

Tampoco se pretende superar a los mejores sistemas ni competir en los *benchmarks*. Los primeros puestos de estos retos combinan las salidas de varios decodificadores independientes mediante un modelo de lenguaje de gran tamaño [29]. Aquí el interés está en ver hasta dónde llega un modelo de habla abierto por sí solo y entender qué estrategias son las más viables con recursos limitados.

Así pues, el trabajo se divide en dos bloques de experimentos. En el primero se entrena con todas las sesiones y se busca la mejor forma de adaptar OWSM, probando distintas opciones para el *frontend*, el *encoder* y el *decoder*. En el segundo se toma el mejor modelo del primer bloque y se simula su uso real: se entrena solo con las primeras sesiones, en

orden de fecha, y se evalúa tanto en esas sesiones como en las que vienen después. Así se analiza su capacidad predictiva en cada sesión para poder concluir el grado de eficiencia.

2.2 Objetivos

El objetivo general de este trabajo es evaluar la viabilidad de reutilizar un modelo de reconocimiento del habla preentrenado a gran escala, OWSM, para decodificar a texto la actividad neuronal intracortical durante el habla intentada. Para ello se determina qué partes del modelo pueden aprovecharse y cuáles deben adaptarse o reentrenarse, y con qué coste. También se analiza cómo se comporta el sistema ante frases no vistas y con el paso del tiempo, con el fin de estimar cuántos datos necesita.

2.2.1. Objetivos específicos

El objetivo general se concreta en cuatro objetivos específicos. Los tres primeros corresponden al primer bloque de experimentos y el cuarto, al segundo. Las partes del sistema que se nombran en ellos se explican en el capítulo 5.

O1. ***Frontend.*** Diseñar y comparar varios módulos de entrada que adapten la señal neuronal al *encoder* de OWSM, desde una capa lineal sencilla hasta una red convolucional profunda, y con distinto submuestreo temporal. Se mide tanto el resultado como el coste de entrenamiento, para elegir el mejor compromiso entre ambos.

O2. ***Encoder.*** Comprobar si los pesos preentrenados del *encoder* de OWSM aportan algo frente al mismo *encoder* entrenado desde cero cuando la entrada es actividad neuronal en lugar de audio.

O3. ***Decoder.*** Evaluar si el *decoder* preentrenado de OWSM, que aporta el conocimiento del lenguaje, puede usarse sin modificar, si mejora con una adaptación ligera como LoRA [38] o si es mejor entrenarlo desde cero.

O4. **Ventana temporal.** Medir cómo varía el error del mejor modelo de una sesión a otra y simular después un uso real del sistema, entrenando con un número creciente de sesiones en orden cronológico y evaluando en sesiones ya vistas y en sesiones futuras. Con ello se quiere saber cuánto empeora el modelo con el paso del tiempo y cuántos datos hacen falta para que deje de necesitar más. También se analiza qué palabras reconoce mejor y peor, según lo frecuentes que sean en el entrenamiento y si aparecen o no en él, y cómo se comporta con frases que no ha visto.

2.3 Alcance y limitaciones

Este trabajo es un estudio intrapersonal. Se centra en una sola persona, que además habla en inglés, y busca el mejor sistema para ella, no un modelo que funcione con cualquiera. Por eso sus conclusiones no se pueden trasladar sin más a otros participantes. Es lo habitual en este campo, donde la mayoría de los estudios trabajan con un único participante [23], [24]. Además, todo se evalúa en diferido, sobre grabaciones ya hechas y sin estudiar el funcionamiento en tiempo real, y solo se trabaja con habla intentada, en la que la persona intenta hablar aunque no pueda. El habla imaginada queda fuera.

La limitación más importante es el coste. Los sistemas *brain-to-text* actuales son grandes y caros de entrenar y de usar. Además de combinar varios modelos, el decodificador de

referencia de estos retos necesita unos 320 GB de memoria RAM, algo que limita el acceso tanto a los pacientes como a los investigadores [39]. Este trabajo, en cambio, se ha hecho con una sola GPU en Google Colab y con un tiempo y un presupuesto limitados. Cada entrenamiento completo tarda varias horas, así que cada configuración se ha entrenado una sola vez y el número de pruebas ha sido limitado. Las limitaciones que salen de los propios resultados se comentan en el capítulo 8.

2.4 Estructura de la memoria

El resto de la memoria se organiza de la siguiente forma. El capítulo 3 repasa el estado del arte de la decodificación del habla a partir de actividad neuronal y de los modelos de habla preentrenados. El capítulo 4 presenta el conjunto de datos utilizado. El capítulo 5 explica los fundamentos de los modelos y técnicas empleados. El **??** describe el sistema construido a partir de OWSM y las adaptaciones que se le hacen. El capítulo 6 detalla cómo se entrenan y evalúan los modelos y el diseño de los dos bloques de experimentos. El capítulo 7 recoge y discute los resultados. El capítulo 8 reúne las limitaciones del estudio y las líneas de trabajo futuro y, por último, el capítulo 9 resume las conclusiones. Al final se incluyen los anexos, entre ellos el de la relación del trabajo con los Objetivos de Desarrollo Sostenible.

CAPÍTULO 3
Estado del arte

Este capítulo repasa lo que se ha hecho hasta ahora en la decodificación del habla a partir de la actividad cerebral y en la reutilización de modelos de habla. Primero se comparan las técnicas de registro y se recogen las cifras de las neuroprótesis del habla más recientes (secciones 3.1 y 3.2). Después se explica cómo están construidos los decodificadores actuales y por qué (sección 3.3), y qué resultados se obtienen en el *benchmark* Brain-to-Text '25 (sección 3.4). Por último se resumen los modelos de habla preentrenados y los trabajos que los han llevado a la señal neuronal (sección 3.5), se comparan todos los sistemas vistos (sección 3.6) y se sitúa este trabajo respecto a ellos (sección 3.7). En el capítulo aparecen algunos conceptos, como las métricas de error o las partes de un modelo, que se explican más adelante, en el marco teórico (capítulo 5).

3.1 Modalidades de registro en interfaces cerebro-computador

La técnica de registro condiciona qué información se puede extraer y qué tipo de decodificador tiene sentido construir. Las técnicas utilizadas en BCI del habla van de menos a más invasivas. En general, cuanto más invasiva es la técnica, mejor es la señal:

Electroencefalografía (EEG). Es una técnica no invasiva, barata y fácil de conseguir. Registra, a través del cráneo, la actividad sumada de grandes grupos de neuronas, por lo que la señal tiene mucho ruido. Funciona bien para elegir entre pocas opciones, pero la decodificación de texto a partir de EEG está en discusión: se ha visto que algunos modelos publicados rinden casi igual utilizando solo ruido que utilizando EEG [40]. Además, suele presentar problemas de portabilidad y comodidad, y es la técnica más afectada por el paso del tiempo.

Estereoelectroencefalografía (sEEG). Utiliza electrodos de profundidad que se implantan con fines diagnósticos en clínicas, a través de pequeños orificios y sin necesidad de abrir el cráneo [41]. Tiene buena resolución temporal y permite llegar a estructuras profundas, pero la colocación de los electrodos responde a criterios clínicos y no a las necesidades del decodificador. Por ello, no suele utilizarse para este tipo de problemas.

Electrocorticografía (ECoG). Utiliza rejillas de electrodos colocadas sobre la superficie de la corteza, lo que obliga a abrir el cráneo [42]. Estas rejillas cubren una zona amplia, son estables durante mucho tiempo y han permitido construir los primeros sistemas de habla en tiempo real [17], [18].

Matrices intracorticales de microelectrodos. Son matrices de electrodos que penetran en la corteza, normalmente de tipo Utah, y registran la actividad de pequeños grupos

de neuronas cercanos a cada electrodo. Es la técnica con mayor riesgo quirúrgico, pero con ella se han conseguido prestaciones propias de una conversación [23], [24]. Es la técnica utilizada para obtener los datos de este trabajo, cuyas características se describen en el capítulo 4.

La tabla 3.1 resume las cuatro.

Tabla 3.1: Comparación de las modalidades de registro usadas en BCI del habla.

Modalidad	Cirugía	Resolución espacial	Cobertura	Duración de los registros
EEG	No	Baja	Toda la cabeza, solo superficie	Sesiones de laboratorio
sEEG	Mínima, sin abrir el cráneo [41]	Alta, en puntos dispersos	Dispersa, incluye estructuras profundas	Lo que dura el ingreso clínico
ECoG	Apertura del cráneo [42]	Media	Una zona amplia de la superficie	Larga: unas 81 semanas en [17]
Intracortical	Apertura del cráneo e implante penetrante	Muy alta: grupos pequeños de neuronas	Pequeña: $3{,}2 \times 3{,}2$ mm por matriz [24]	Larga: 20 meses en Brain-to-Text '25 [30]

3.2 Evolución de las neuroprótesis del habla

En esta sección se recogen los trabajos que marcaron cada salto en el campo de las neuroprótesis. Es importante entenderlos para poder justificar decisiones más tarde.

Uno de los primeros sistemas que produjo lenguaje de forma continua decodificaba la escritura: el participante intentaba escribir en vez de hablar. Llegó a unos 90 caracteres por minuto, con más de un 99 % de acierto usando un autocorrector [22].

Con el habla intentada, Moses *et al.* decodificaron frases en tiempo real con ECoG en una persona con anartria, con un vocabulario de 50 palabras y una tasa de error por palabra (WER) del 25,6 % [17]. En 2023, dos trabajos pasaron a vocabularios mucho más grandes. Metzger *et al.*, con ECoG de alta densidad y también síntesis de voz y un avatar, llegaron a un 25 % de WER sobre 1.024 palabras [18]. Willett *et al.*, con matrices intracorticales, llegaron a un 23,8 % sobre 125.000 palabras [23].

El sistema de Card *et al.*, del que proceden los datos de este trabajo, es el referente actual. Con 30 minutos de datos de calibración alcanzó un 99,6 % de acierto con 50 palabras el primer día, y un 90,2 % con 125.000 palabras el segundo día, tras 1,4 horas más de datos. Mantuvo un 97,5 % de acierto durante 8,4 meses, y el participante lo usó para conversar más de 248 horas [24]. Su decodificador combina una red recurrente que predice fonemas con un modelo de n-gramas y un modelo de lenguaje de gran tamaño que reordena las frases candidatas.

Desde entonces el campo se ha abierto mucho. Los datos de Willett *et al.* y de Card *et al.* se publicaron en los retos Brain-to-Text '24 y '25 [29], [30], y ahora muchos grupos sin acceso a pacientes trabajan sobre los mismos registros e intentan mejorar sus cifras. En el reto de 2025, el sistema de referencia de los organizadores tenía un 6,7 % de WER, y los primeros puestos bajaron hasta cerca del 1,5 % juntando varios decodificadores y modelos de lenguaje de gran tamaño [30]. Debemos tener en cuenta que estos resultados se obtienen en diferido, sobre datos ya grabados, y no se han probado en tiempo real con el participante, como sí se hizo con el sistema de Card.

3.3 Anatomía de un decodificador *brain-to-text*

Los sistemas anteriores siguen casi todos el mismo esquema, heredado del reconocimiento de voz clásico, y lo van mejorando: un codificador neuronal convierte la señal en probabilidades de fonemas o palabras y un modelo de lenguaje forma las frases [43]. Esta sección explica por qué se ha llegado a ese esquema y hacia dónde está cambiando.

3.3.1. El cambio entre días

Lo que más condiciona el diseño es que la señal cambia de un día a otro, así que un decodificador entrenado con los datos de un día va perdiendo precisión [44]. La respuesta del campo ha ido cambiando. Willett *et al.* añadieron delante de un modelo compartido una capa de entrada propia de cada día [22], una solución (sección 5.2.2) que se ha mantenido en los sistemas posteriores [23], [24]. Como cada día nuevo necesita datos etiquetados para su capa, el siguiente paso fue reducir esa calibración: Card *et al.* calibran el sistema con 30 minutos de datos [24], y Fan *et al.* lo recalibran con las propias frases que decodifica, sin pedir datos nuevos al usuario [37]. Los sistemas más recientes comparten parámetros entre días cercanos [45].

3.3.2. El codificador

El codificador de referencia sigue siendo una red recurrente, la de los sistemas de Willett *et al.* y Card *et al.* y la del *benchmark* [23], [24], [30]. Se mantiene porque ha costado mucho mejorarla: en el Brain-to-Text '24, otras arquitecturas más recientes no consiguieron superarla [29]. Los avances recientes llegan cuando se añade otra fuente de información: preentrenar el codificador con datos neuronales de muchas tareas, personas e incluso especies [43], partir de un modelo de voz preentrenado [45] o usar arquitecturas de reconocimiento de voz como Conformer [46].

3.3.3. La salida y el modelo de lenguaje

La unidad de salida más usada es el fonema. La corteza motora controla la articulación y no la ortografía, y con unos 40 fonemas el problema es más sencillo que con miles de palabras [23].

Pero los fonemas no son texto. Para pasar a palabras hace falta un diccionario de pronunciación, que indica qué fonemas forman cada palabra, y un modelo de lenguaje. En este paso está buena parte del rendimiento final. Los sistemas de referencia usan un modelo de n-gramas, que estima la probabilidad de cada palabra a partir de las anteriores [47], y reordenan sus resultados con modelos de lenguaje grandes.

Este esquema es caro. Por ejemplo, un modelo de 5-gramas necesita unos 300 GB de memoria y cada decodificación tarda unos 750 ms, frente a menos de 2 GB y unos 50 ms de un sistema de extremo a extremo [45]. Además, el codificador y el modelo de lenguaje se entrenan por separado y no se pueden optimizar juntos [43]. Por eso están apareciendo sistemas de extremo a extremo, que pasan de la señal al texto con un solo modelo, prediciendo directamente letras o palabras [45], [46]. Todavía quedan por detrás de los sistemas con modelo de lenguaje (sección 3.4), pero son más baratos y sencillos de usar y están aumentando rápidamente su precisión.

3.4 El *benchmark* Brain-to-Text '25

Los *benchmarks* públicos, con particiones fijas y una métrica común, han acelerado mucho el campo. El Brain-to-Text '25, el que se usa en este trabajo, se describe en el capítulo 4. Su sistema de referencia, la red recurrente con fonemas y un modelo de 5-gramas, consigue un 6,7 % de WER [45]. Las primeras posiciones se consiguen con modelos de lenguaje y, a menudo, combinando varios modelos, como ya pasó en la edición anterior [29]. BrainWhisperer, por ejemplo, quedó cuarto de 466 equipos con una versión ampliada de su sistema con modelo de 5-gramas [45].

Estas cifras se consiguen con sistemas grandes y lentos. Combinan varios decodificadores con modelos de lenguaje de gran tamaño, y solo el modelo de lenguaje de referencia de la competición necesita cientos de gigabytes de memoria [30]. Hoy sería difícil llevar algo así a un dispositivo que funcione en tiempo real y en casa del usuario. Aun así, marcan hasta dónde se puede llegar con estos datos y por eso es importante tenerlos en cuenta.

3.5 Modelos de habla preentrenados y transferencia a la señal neuronal

Por último, el reconocimiento de voz ha pasado a preentrenar modelos con enormes cantidades de audio y a adaptarlos después a cada tarea. Algunos, como wav2vec 2.0 [48], aprenden de audio sin transcribir y necesitan después un ajuste supervisado. Otros, como Whisper [36], se entrenan directamente con cientos de miles de horas de audio transcrito. Whisper es un modelo codificador-decodificador, así que, además de leer el audio, trae un decodificador que ha aprendido cómo son las frases. En *brain-to-text* eso interesa, porque el conocimiento del lenguaje es justo lo que le falta a un codificador entrenado con pocos datos. Su inconveniente para la investigación es que no se conocen sus datos ni todo su entrenamiento.

Llevar estos modelos a la actividad cerebral se ha probado de varias formas. Algunos trabajos toman del modelo de voz su arquitectura o su objetivo de entrenamiento, pero no los pesos aprendidos con audio. BrainTalker proyecta la señal de ECoG al espacio de wav2vec 2.0 para sintetizar voz con muy pocos datos [49], y Yuan y Makin entrenan wav2vec con ECoG sin etiquetar y mejoran así un decodificador supervisado, incluso pasando de un paciente a otro [50].

Otros transfieren los pesos aprendidos con audio. Fiedler *et al.* sustituyen el extractor de audio de wav2vec 2.0 por otro pensado para señal neuronal y comparan, con los datos del participante, el ajuste completo de los pesos preentrenados, el entrenamiento desde cero y el modelo preentrenado congelado. El ajuste completo da el mejor resultado, con un 18,54 % de error por carácter y un 30,97 % de WER sin modelo de lenguaje, unos 20 puntos de error por carácter mejor que desde cero [51]. BrainWhisperer lleva la idea a Whisper y a los datos del Brain-to-Text '25. Adapta la entrada y ajusta el decodificador de Whisper, entre otros cambios, y llega a un 10,2 % de WER.

3.6 Comparativa de los sistemas revisados

La tabla 3.2 reúne los sistemas que han aparecido en este capítulo. No todas las cifras se pueden comparar entre sí. Los sistemas clínicos se evaluaron en tiempo real, cada uno con su participante y su vocabulario, mientras que el resto se evalúa en diferido. Pero podemos usar estos datos para discutir los resultados de este trabajo.

Tabla 3.2: Resumen de los sistemas revisados en este capítulo.

Trabajo	Registro	Participante	Vocabulario	WER	Velocidad
Sistemas clínicos, en tiempo real					
Moses *et al.* [17]	ECoG	Anartria	50 palabras	25,6 %	15 ppm
Willett *et al.* [22]	Intracortical	Tetraplejia	Escritura	<1 %*	90 cpm
Metzger *et al.* [18]	ECoG	Anartria	1.024 palabras	25 %	78 ppm
Willett *et al.* [23]	Intracortical	ELA (T12)	125.000 palabras	23,8 %	62 ppm
Card *et al.* [24]	Intracortical	ELA (T15)	125.000 palabras	$\sim$2,5 %	32 ppm
Brain-to-Text '25 con modelo de lenguaje externo, en diferido					
Referencia de la competición [30]	Intracortical	ELA (T15)	125.000 palabras	6,7 %†	–
BrainWhisperer [45]	Intracortical	ELA (T15)	125.000 palabras	5,9 % (5,2 %)†	–
Primeros puestos [30]	Intracortical	ELA (T15)	–	$\approx$1,5 %†	–
Brain-to-Text '25 de extremo a extremo, sin modelo de lenguaje externo, en diferido					
Red recurrente de referencia [45]	Intracortical	ELA (T15)	Abierto	16,0 %†	–
BrainWhisperer [45]	Intracortical	ELA (T15)	Abierto	10,2 % (8,7 %)†	–
Khanday *et al.* [46]	Intracortical	ELA (T15)	Abierto	23,8 %‡	–

ppm: palabras por minuto; cpm: caracteres por minuto. Entre paréntesis, entrenando también con datos de otros participantes (BrainWhisperer) o combinando varios modelos (BIT). *Error por carácter, en diferido y con autocorrector. †Partición de prueba de la competición. ‡Error por carácter en la partición de validación.

3.7 Posicionamiento de este trabajo

De esta revisión salen tres huecos, que son los que aborda este trabajo. Primero, la transferencia desde modelos de habla funciona en algunos casos, pero no se sabe bien qué parte la explica. Fiedler *et al.* usan un modelo que solo tiene codificador y otro participante, y BrainWhisperer cambia muchas cosas a la vez para conseguir la mejor cifra. Falta separar, con un mismo presupuesto, qué aporta cada parte de un modelo codificador-decodificador (el *frontend*, el codificador y el decodificador), y hacerlo con un modelo abierto como OWSM.

Segundo, en el *benchmark* las frases de validación y de prueba son de días que también tienen frases de entrenamiento (sección 4.2), así que las cifras publicadas no dicen cómo se comporta un modelo en un día nuevo. Los trabajos que estudian el paso del tiempo lo hacen con sistemas recurrentes y modelo de lenguaje [37]. Khanday *et al.* señala que el cambio entre sesiones es lo que más hace variar su error. Falta ver cómo se comporta en días futuros un sistema de extremo a extremo basado en un modelo de voz, y cuántos datos necesita.

Por último, las primeras posiciones necesitan muchos recursos. Este trabajo se limita a una GPU, un solo modelo y ningún modelo de lenguaje externo, así que su referencia son los sistemas de extremo a extremo de la tabla 3.2 y no la clasificación. Aun así, parte con desventajas conocidas frente a ellos: no usa datos de otros participantes y su presupuesto de cómputo es pequeño. Son técnicas que se pueden añadir después sobre el mismo sistema, por tanto la distancia mide lo que queda por ganar.

El trabajo más cercano es el de Khanday *et al.*, que usa los mismos datos y un Conformer entrenado desde cero que predice letras directamente, sin modelo de lenguaje, y obtiene un 23,80 % de error por carácter en validación [46]. Este trabajo sigue la misma idea de extremo a extremo, pero parte de un modelo de voz preentrenado aprovechando su decodificador.

CAPÍTULO 4
Conjunto de datos

En este capítulo se explica con qué datos se trabaja. Primero se cuenta por qué se eligió el conjunto Brain-to-Text '25. Después se describe cómo se registraron sus datos, cómo están organizados y, por último, qué tipo de frases contiene.

4.1 Selección del conjunto de datos

OWSM es un modelo de reconocimiento de voz: recibe una secuencia de varios segundos y la convierte en texto. Para adaptarlo, lo ideal es un conjunto con frases completas en lugar de palabras sueltas, y con su transcripción. También hacen falta bastantes ejemplos, porque el modelo tiene unos 100 millones de parámetros. Por último, ayuda mucho que existan particiones fijas y resultados publicados con los que comparar.

Otros aspectos son más discutibles. El primero es el tipo de señal. El EEG de superficie no necesita cirugía, así que es fácil encontrar conjuntos con muchos participantes, incluso en español. Es el caso de UGR-MINDVOICE [52], grabado en la Universidad de Granada con 15 hablantes nativos que pronuncian sílabas y palabras en voz alta y también de forma imaginada. El problema es la señal EEG, que llega con bastante ruido, y los estímulos son sílabas y palabras aisladas. Además, si el modelo fallara con EEG, no se podría saber si el problema está en OWSM o en una señal que no lleva suficiente información sobre el habla.

Con electrodos implantados (sEEG) la señal es mucho más limpia, pero hay pocos participantes porque suelen ser pacientes a los que se colocan electrodos durante su ingreso para localizar el origen de algun problema. De este tipo son sEEG2Speech [42] y SingleWordProductionDutch [41], los dos en neerlandés. El primero es el único de con frases: tres pacientes leen 100 frases de cinco a siete palabras. El segundo tiene más participantes, diez, y más de mil electrodos en total, lo que lo hace interesante para comparar entre personas, pero solo contiene palabras sueltas. En los dos casos hay entre 5 y 20 minutos de grabación por paciente, poco para los propósitos propuestos. Además, están pensados para reconstruir el audio de la voz a partir de la señal, mientras que aquí lo que se busca es texto.

Brain-to-Text '25 [24], [30] tiene la limitación contraria: un solo participante, que además habla inglés. A cambio, la señal se registra con 256 microelectrodos implantados en la corteza, que dan la señal de más calidad de las cuatro opciones, y hay cerca de 11.000 frases transcritas, grabadas en 45 sesiones a lo largo de 20 meses. Tiene particiones oficiales y una competición pública con resultados publicados, y los datos vienen ya preparados y con código para cargarlos. Es el único que cumple lo que se buscaba y por eso es el que se usa en este trabajo. Que abarque tantos meses es otra ventaja, porque permite estudiar cómo se comporta el modelo con el paso del tiempo, que es uno de los objetivos del trabajo

(OE4 en la sección 2.2). Tener un solo participante limita las conclusiones a esa persona, pero encaja con el planteamiento intrapersonal del trabajo (sección 2.3).

La tabla 4.1 resume las cuatro opciones.

Tabla 4.1: Comparación de los conjuntos de datos considerados.

	UGR-MINDVOICE [52]	**sEEG2Speech** [42]	**SingleWord-ProductionDutch** [41]	**Brain-to-Text '25** [24], [30]
Señal	EEG de superficie	sEEG (111–127 contactos por paciente)	sEEG (54–127 electrodos por paciente, 1.103 en total)	Microelectrodos intracorticales (4 matrices, 256 electrodos)
Participantes	15 hablantes nativos	3 pacientes con epilepsia	10 pacientes con epilepsia	1 paciente con ELA
Idioma	Español	Neerlandés	Neerlandés	Inglés
Material	Sílabas y palabras aisladas, en voz alta e imaginadas	100 frases de 5 a 7 palabras; 10–20 min por paciente	100 palabras aisladas; 5 min por paciente	10.948 frases de habla intentada en 45 sesiones (20 meses)
Orientación	Neurociencia e interfaces de habla silenciosa	Síntesis de voz	Producción del habla y síntesis de voz	Decodificación a texto
Acceso	OSF	OSF	OSF (iBIDS/NWB)	Dryad (HDF5 por sesión)

4.2 El conjunto Brain-to-Text '25

4.2.1. Origen y participante

Los datos proceden del ensayo clínico BrainGate2 descrito por Card *et al.* [24] y se publicaron más tarde como *benchmark* abierto y como competición [30]. El participante, identificado como T15, es un hombre de 45 años con ELA, tetraparesia y una disartria grave. Los electrodos se le implantaron cinco años después del inicio de la enfermedad, y la recogida de datos empezó en agosto de 2023, 25 días después de la operación [24].

La tarea es de copia. En la pantalla aparece una frase y, tras una señal de inicio, T15 intenta pronunciarla a su ritmo mientras se registra su actividad neuronal. No hay audio con el que alinear la señal, así que no se sabe en qué momento dice cada palabra.

4.2.2. Adquisición de la señal

La señal se registra con cuatro matrices de microelectrodos de tipo Utah implantadas en el giro precentral izquierdo. Cada matriz mide 3,2 $\times$ 3,2 mm, tiene 64 electrodos en una rejilla de 8 $\times$ 8 y se inserta 1,5 mm en la corteza, así que en total hay 256 electrodos. Se colocaron en cuatro zonas: el área 55b, relacionada con el lenguaje, y tres áreas que controlan los articuladores del habla, que son las partes dorsal y ventral del área premotora 6v y la corteza motora primaria (área 4) [24].

De cada electrodo se obtienen dos características, de modo que cada instante se describe con un vector de 512 valores:

Cruces de umbral (*threshold crossings*). Número de veces que la señal filtrada cruza un umbral fijado en $-4{,}5$ veces el valor eficaz del ruido del canal. Da una idea de la actividad de las neuronas cercanas al electrodo sin tener que separar las espigas de cada una.

Potencia en la banda de espigas (*spike band power*). Energía de la señal en la banda de frecuencias de las espigas. Complementa a la anterior y depende menos del umbral elegido.

Las dos se agregan en ventanas de 20 ms, lo que da una secuencia a 50 Hz, y se normalizan con una puntuación z dentro de cada bloque de grabación [39]. En este trabajo se usan tal y como se publican.

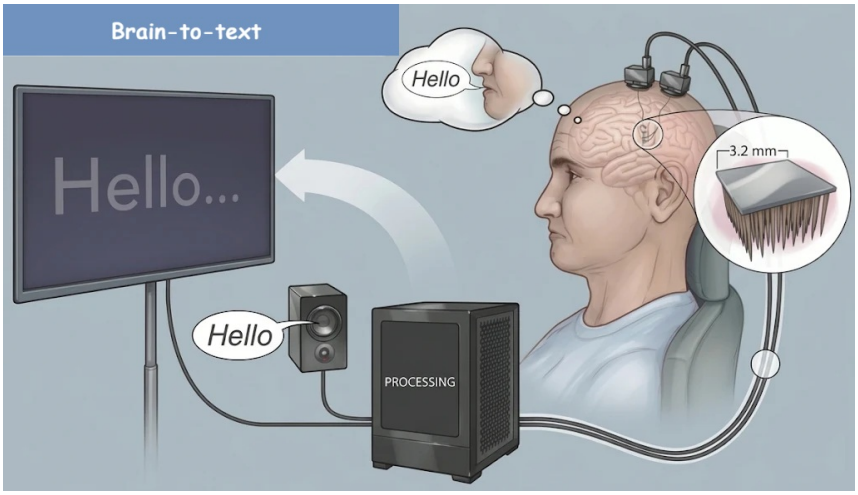


Figura 4.1: Esquema de una interfaz cerebro-texto. Las matrices de microelectrodos de tipo Utah (3,2 × 3,2 mm) se implantan en la corteza motora del habla; la señal que registran mientras el participante intenta hablar se procesa para mostrar la frase como texto en una pantalla.

4.2.3. Organización de los datos

Los datos se reparten en ficheros HDF5 por sesión. Cada sesión se identifica por su fecha (`t15.AAAA.MM.DD`) y tiene hasta tres ficheros, uno por partición: entrenamiento, validación y prueba. No todas las sesiones tienen las tres. Dentro de cada fichero, cada ensayo (una frase) se guarda por separado con estos elementos [30]:

`input_features`**.** Matriz de $T \times 512$ con las características neuronales, donde T es el número de ventanas del ensayo.

`seq_class_ids`**.** Secuencia de fonemas de la frase.

`transcription`**.** Transcripción de la frase.

Atributos. Número de ventanas, longitud de la secuencia de fonemas, texto de la frase e identificadores de sesión, bloque y ensayo.

Las 45 sesiones van del 11 de agosto de 2023 al 13 de abril de 2025, y en ese tiempo la señal no se mantiene igual. Lo que capta cada electrodo cambia de un día a otro, e incluso dentro del mismo día por movimiento de los electodos. Es lo que se conoce como no estacionariedad de la señal. La normalización por bloque corrige una parte de estos cambios, pero no todos. A esto se suma que en unos bloques T15 intenta hablar en voz

alta y en otros sin voz, y los organizadores no indican, a propósito, cuál es cuál [30]. Con todo esto, no es lo mismo evaluar el modelo en un día del que ya ha visto frases que en un día nuevo, y es lo que se estudia en el segundo bloque de experimentos (capítulo 6).

4.2.4. Particiones y composición

El conjunto tiene 8.072 ensayos de entrenamiento, 1.426 de validación y 1.450 de prueba [39]. Las frases se graban en bloques de entre 5 y 50, y los bloques no se mezclan entre entrenamiento y evaluación: 198 bloques (en 45 sesiones), forman el entrenamiento, y los 67 restantes (en 41 sesiones), forman la validación y la prueba.

Son aproximadamente 8.000 frases de entrenamiento, todas de la misma persona. A pesar de ser un dataset grande para BCI, los modelos se suelen entrenar con muchas más horas de audio. Esto condiciona qué partes del modelo se reutilizan y cómo se entrena (capítulo 6).

Las frases proceden de seis corpus distintos (tabla 4.2). La mayoría son de Switchboard [53], un corpus de conversaciones telefónicas en inglés, seguidas de un corpus de palabras frecuentes preparado por los autores y de frases con un vocabulario cerrado de 50 palabras [30]. Tres corpus aparecen solo en validación y prueba: OpenWebText, con texto de páginas web; las frases Harvard, pensadas para cubrir los sonidos del inglés de forma equilibrada [54], y secuencias de palabras al azar. Suman 349 frases de un tipo que el modelo no ve al entrenar. La validación tiene, por tanto, frases de otro estilo y palabras que no salen en el entrenamiento, y las secuencias al azar no tienen ninguna estructura gramatical que el modelo pueda aprovechar.

Tabla 4.2: Composición de Brain-to-Text '25 por corpus de origen, según [39]. Estos recuentos suman algunas frases más (11.162) que las tres particiones juntas (10.948).

Corpus	Bloques	Frases	Partición
Switchboard	182	7.672	Entrenamiento y validación/prueba
Palabras frecuentes	48	2.050	Entrenamiento y validación/prueba
Vocabulario de 50 palabras	25	1.091	Solo entrenamiento
OpenWebText	5	172	Solo validación/prueba
Frases Harvard	3	98	Solo validación/prueba
Palabras al azar	2	79	Solo validación/prueba

Las transcripciones de la partición de prueba no son públicas. La clasificación de la competición se calcula con ellas y, como este trabajo no participó, no es posible evaluar con esa partición. Por eso aquí la validación oficial hace de conjunto de prueba. Hay indicios de que esto apenas cambia las cifras: LightBeam, por ejemplo, obtiene un WER del 6,36 % en validación y del 5,77 % y el 6,47 % en las partes pública y privada de la prueba [39]. Cómo se usa cada partición en los experimentos se explica en el capítulo 6. No obstante, se mencionará cuando los resultado se comparen al conjunto de validación o al de prueba.

4.3 Análisis exploratorio

Este análisis es breve y se centra en las frases. Las características neuronales no se estudian aquí, porque vienen ya calculadas y normalizadas con el procedimiento habitual en la literatura [23], [24] y este trabajo se centra en el modelo.

Casi siete de cada diez frases son de Switchboard (tabla 4.2), así que el estilo que más se repite es el de una conversación espontánea. Las frases tienen de media 6,2 palabras, y

la mayoría tiene entre 4 y 9 (figura 4.2). La distribución es muy parecida en entrenamiento y validación, aunque en validación hay algo más de frases largas.

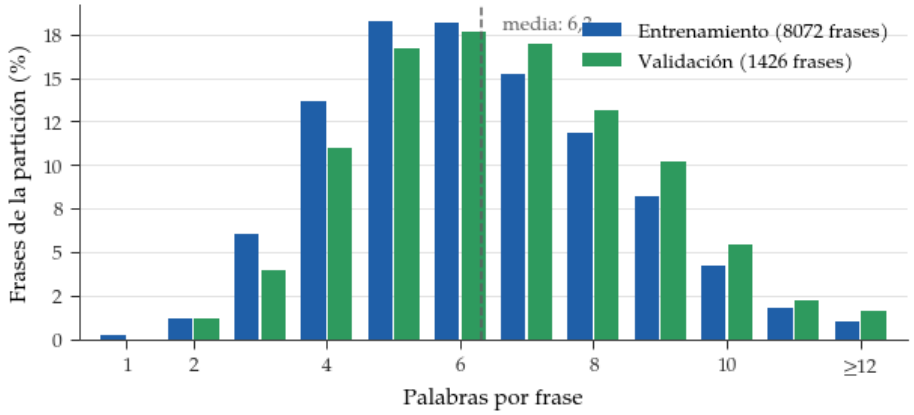


Figura 4.2: Número de palabras por frase en las particiones de entrenamiento y validación. La línea discontinua marca la media.

Las frases casi no se repiten. La mayoría aparece una sola vez en todo el conjunto (figura 4.3a), y solo el 11,4 % de las frases de validación está también, idéntica, en el entrenamiento. La figura 4.3b muestra además cuánto se parece cada frase de validación a la más parecida del entrenamiento: lo habitual es que compartan alrededor de la mitad. El modelo, por tanto, se evalúa casi siempre con frases nuevas.

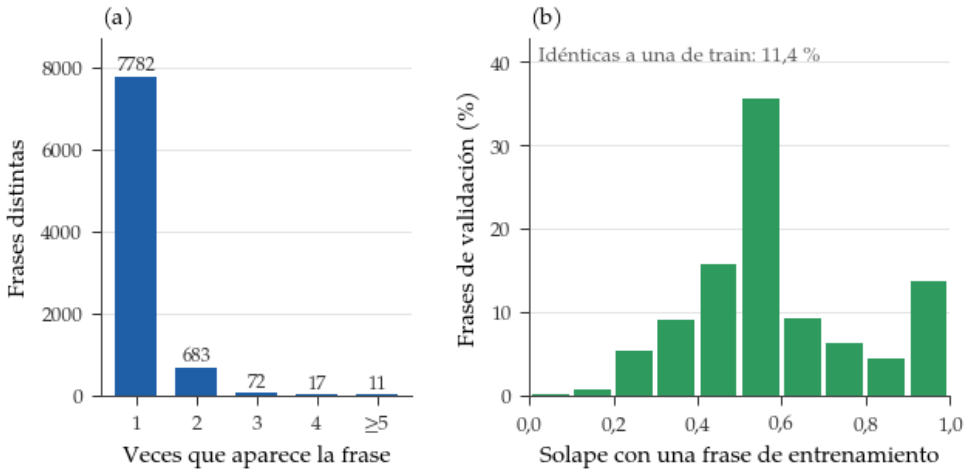


Figura 4.3: Repetición de frases y solape entre particiones. (a) Número de frases distintas según las veces que aparecen en el conjunto. (b) Solape de cada frase de validación con la frase de entrenamiento más parecida; un solape de 1 indica que la frase está tal cual en el entrenamiento.

Con las palabras ocurre lo normal en cualquier texto: unas pocas aparecen muchísimo y la mayoría muy poco. Las más frecuentes son pronombres, artículos y preposiciones, como *I*, *the*, *to* o *you* (figura 4.4a), que son las más usadas en una conversación normal. El vocabulario de entrenamiento tiene 3.745 palabras, pero las 100 más frecuentes ya cubren el 61 % del texto (figura 4.4b). Aun así, es importante ver que un 4,2 % de las palabras de validación no aparece nunca en el entrenamiento. El modelo solo puede escribir estas palabras nuevas relacionándolas con el contexto y usando su conocimiento de los audios.

Por último, las particiones comparten días. Todas las sesiones tienen frases de entrenamiento, así que cada frase de validación tiene otras grabadas el mismo día en el entrenamiento, aunque en bloques distintos. Es una situación optimista si se compara con el uso real, en el que el sistema tiene que funcionar en días que todavía no ha visto.

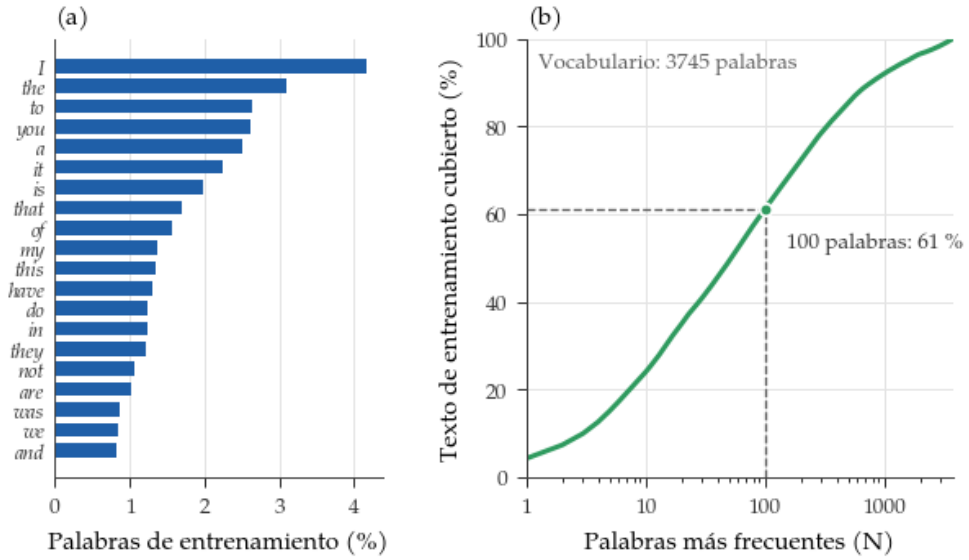


Figura 4.4: Frecuencia de las palabras en el entrenamiento. (a) Las 20 palabras más frecuentes, como porcentaje del total de palabras. (b) Porcentaje del texto de entrenamiento que cubren las N palabras más frecuentes (eje horizontal en escala logarítmica).

Por eso, además de la evaluación con todas las sesiones, se hace una evaluación en orden cronológico (capítulo 6). La figura 4.5 muestra cuántas frases hay en cada sesión a lo largo de los 20 meses; las cuatro sesiones marcadas no tienen frases de validación.

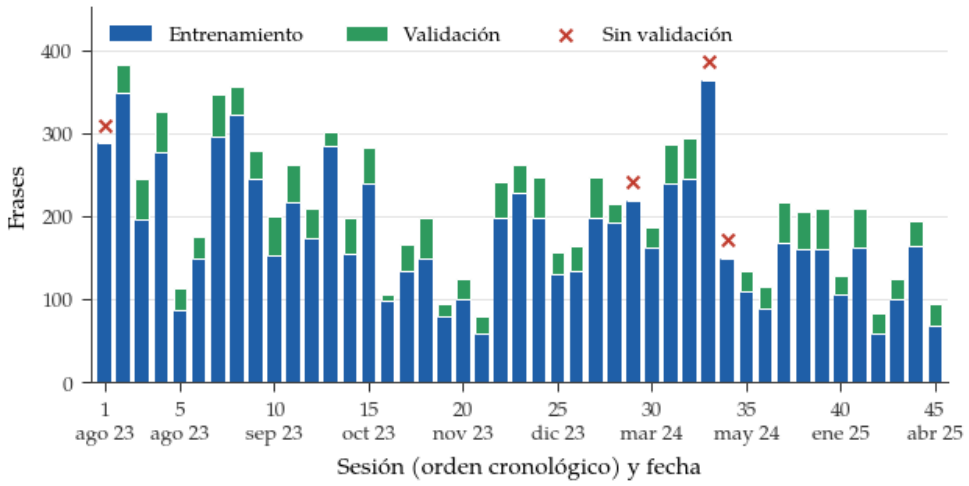


Figura 4.5: Número de frases de entrenamiento y validación en cada una de las 45 sesiones, en orden cronológico. Las cruces marcan las sesiones sin frases de validación.

CAPÍTULO 5
Marco teórico

En este capítulo se explican los modelos y técnicas que forman el sistema: qué hace cada uno y para qué sirve. Cómo se han combinado, con qué valores y qué se ha probado se cuenta en el capítulo 6.

5.1 Planteamiento del problema

Sea $\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_T)$ una secuencia de T vectores de características neuronales, con $\mathbf{x}_t \in \mathbb{R}^{512}$, obtenida durante un intento de habla, y sea $\mathbf{y} = (y_1, \ldots, y_L)$ la secuencia de unidades de texto de la frase que el participante intentaba pronunciar, con $y_l \in \mathcal{V}$. En este trabajo, $\mathcal{V}$ es el vocabulario de subpalabras de OWSM

El problema consiste en aprender una función $f_\theta : \mathbb{R}^{T\times 512} \to \mathcal{V}^*$ que estime $\mathbf{y}$ a partir de $\mathbf{X}$. Con tres condiciones:

(a) Es una tarea de transducción de secuencias de longitud variable, con $T > L$: a 50 Hz, una frase corta produce muchas de ventanas temporales frente a unas decenas de unidades de salida.

(b) No se dispone de alineamiento entre las posiciones de $\mathbf{X}$ y las de $\mathbf{y}$.

(c) El orden se preserva: la unidad y_l nunca precede temporalmente a y_{l-1}.

5.2 Datos de entrada al modelo

5.2.1. Lotes, relleno y máscaras

Durante el entrenamiento, las frases se procesan en batches de B frases a la vez, porque así se aprovecha mejor la GPU. Cada frase tiene su propia longitud T_b, pero un batch se debe guardar como un único tensor de tamaño igual. En este caso, $B \times T_{\text{máx}} \times 512$, donde $T_{\text{máx}}$ es la longitud de la frase más larga del lote y las demás frases se completan con ceros al final. Este relleno lo llamamos (*padding*) y no es una señal. El modelo no debe tenerlo en cuenta. Para ello, junto al lote se guardan las longitudes reales y, a partir de ellas, una máscara:

$$m_{b,t} = \begin{cases} 1 & \text{si } t \leq T_b \text{ (dato real)}, \\ 0 & \text{si } t > T_b \text{ (relleno)}. \end{cases} \tag{5.1}$$

La máscara se usa en varios sitios. En la atención (sección 5.4.3), las posiciones de relleno reciben una puntuación de $-\infty$ antes de la *softmax*, así que su peso es cero y

ninguna posición real las tiene en cuenta. En la pérdida, las posiciones y los tokens de relleno no se cuentan. Hay que tener en cuenta que cualquier capa con término de sesgo convierte los ceros del relleno en otros valores, de modo que antes de una convolución hay que volver a ponerlos a cero multiplicandolos por la máscara. Por último, cuando una capa reduce el número de ventanas, las longitudes y la máscara se recalculan con la misma regla que esa capa, ecuación (5.6).

5.2.2. Capa de entrada por sesión

Como se vio en el capítulo 4, la señal que registra cada electrodo cambia de un día a otro. Un mismo intento de habla puede producir una actividad algo distinta en dos sesiones, y un modelo único tendría que aprender a la vez varias correspondencias entre señal y texto. La solución habitual en el campo es poner delante del modelo una capa de entrada propia de cada sesión [22], [23]. Es una transformación afín seguida de una no linealidad acotada:

$$\mathbf{z}_t = \text{softsign}\left(\mathbf{W}_s\,\mathbf{x}_t + \mathbf{b}_s\right), \qquad \text{softsign}(u) = \frac{u}{1+|u|}, \tag{5.2}$$

donde s es la sesión a la que pertenece la frase, $\mathbf{W}_s \in \mathbb{R}^{512\times512}$ y $\mathbf{b}_s \in \mathbb{R}^{512}$. La idea es que cada día tenga su propia corrección y que, después de ella, la señal de todos los días se parezca más.

Cada sesión añade $512^2 + 512 = 262{,}656$ parámetros, unos 11,8 millones con las 45 sesiones. Además, la capa solo existe para los días que tienen datos de entrenamiento: para un día nuevo no hay matriz aprendida y habría que recalibrar con datos de ese día. Incluimos esto en el marco teórico porque es una capa muy común en la literatura, y podemos comprobar su utilidad en este trabajo.

5.3 *Frontend*

En reconocimiento de voz se llama *frontend* a la primera etapa del modelo, la que convierte la entrada que tenemos en algo que el codificador pueda procesar. En OWSM, esa etapa toma la forma de onda, calcula un espectrograma en escala mel y reduce su resolución temporal con convoluciones (sección 5.4.2). Pero para esta tarea no tiene sentido proque nosotros no tenemos una señal de unidimensional de audio sino las características de 256 señales. Así que esa parte se sustituye por un módulo nuevo. En este trabajo se llama *frontend* a ese módulo.

El *frontend* tiene dos tareas. La primera es pasar los 512 valores de cada ventana a la dimensión d que espera el codificador (384 en el OWSM escogido). La segunda es reducir el número de ventanas por un factor s, de manera que cada ventana que recibe el codificador resume $20s$ ms de señal (80 ms con $s = 4$). Menos ventanas hacen el codificador más barato, porque el coste de la atención crece con el cuadrado de la longitud, pero CTC necesita al menos una ventana por cada unidad de salida (sección 5.5), así que hay que encontrar hasta dónde merece la pena reducir.

Los *frontends* de la literatura se construyen combinando unas pocas piezas, que se describen a continuación:

Capa lineal. Multiplica cada vector por una matriz y le suma un sesgo:

$$\mathbf{y}_t = \mathbf{W}\mathbf{x}_t + \mathbf{b}, \qquad \mathbf{W} \in \mathbb{R}^{d_{\text{sal}}\times d_{\text{ent}}}. \tag{5.3}$$

Se aplica igual en todas las ventanas, mezcla todos los electrodos entre sí y sirve para cambiar de dimensión.

Apilado de ventanas. Concatena k ventanas seguidas en un solo vector, que después pasa por una capa lineal:

$$\mathbf{u}_t = [\mathbf{x}_{st};\, \mathbf{x}_{st+1};\, \ldots;\, \mathbf{x}_{st+k-1}] \in \mathbb{R}^{512k}. \tag{5.4}$$

El paso s indica cuántas ventanas se avanza de un vector al siguiente. Con $k = s$ los grupos no se solapan y la secuencia se reduce por s. Si $k > s$ se solapan y ocurre el *patching* donde cada vector ve más contexto.

Convolución temporal. Recorre la secuencia con un filtro que mira k ventanas seguidas, el núcleo:

$$\mathbf{y}_t = \mathbf{b} + \sum_{j=0}^{k-1} \mathbf{W}_j\, \mathbf{x}_{st+j}, \qquad \mathbf{W}_j \in \mathbb{R}^{d_{\text{sal}} \times d_{\text{ent}}}. \tag{5.5}$$

Se parece al apilado, pero se pueden encadenar varias capas, y cada salida acaba viendo cada vez más contexto. Con paso $s > 1$ la convolución reduce la secuencia. Si se añaden p ventanas de relleno en cada extremo, la longitud de salida es

$$T' = \left\lfloor \frac{T + 2p - k}{s} \right\rfloor + 1. \tag{5.6}$$

Función de activación. Se aplica a cada valor después de una capa lineal o convolucional, para que el modelo pueda aprender relaciones no lineales. Las dos usadas en el sistema son ReLU y GELU[55]:

$$\text{ReLU}(u) = \text{máx}(0, u) \tag{5.7}$$

$$\text{GELU}(u) = u\, \Phi(u), \tag{5.8}$$

donde Φ es la función de distribución de una normal atenuando los negativos poco a poco.

Normalización por capas (*LayerNorm*). Normaliza cada vector con su propia media y su propia varianza añadiendo unos parámetros que se aprenden [56]:

$$\text{LN}(\mathbf{h}) = \boldsymbol{\gamma} \odot \frac{\mathbf{h} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \boldsymbol{\beta}, \tag{5.9}$$

***Dropout*.** Durante el entrenamiento pone a cero cada valor con probabilidad p y multiplica los demás por $1/(1-p)$ [57]. Se hace para evitar sobreajuste y asi no cae todo el peso en unas pocas neuronas.

Conexión residual. Suma la entrada de un bloque a su salida [58]:

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}). \tag{5.10}$$

El bloque solo tiene que aprender la corrección $F(\mathbf{x})$ y el gradiente llega mejor a las primeras capas permitiendo apilar muchas.

Codificación posicional. La atención no sabe por sí misma en qué orden están las ventanas (sección 5.4.3). Para que lo sepa, se suma a cada vector un patrón que depende de su posición t [59]:

$$\mathrm{PE}_{t,2i} = \sin\left(\frac{t}{10000^{2i/d}}\right), \qquad \mathrm{PE}_{t,2i+1} = \cos\left(\frac{t}{10000^{2i/d}}\right). \tag{5.11}$$

Cada dimensión oscila con una frecuencia distinta, así que cada posición tiene un patrón único y las posiciones cercanas se parecen.

Qué combinación de estas capas forma cada *frontend* probado se explica en el capítulo 6.

5.4 OWSM

5.4.1. Origen y versiones

Whisper [36] es un modelo de reconocimiento de voz de OpenAI que, en su versión original, se entrenó con unas 680.000 horas de audio de internet. Es multilingüe y hace varias tareas, como transcribir, traducir o identificar el idioma. Sus pesos son públicos, pero no lo son los datos ni el proceso completo de entrenamiento. OWSM (*Open Whisper-style Speech Model*) reproduce ese tipo de entrenamiento con datos públicos y con la herramienta de código abierto ESPnet [34], y publica las recetas, el código, los registros de entrenamiento y los pesos [32]. Las primeras versiones usan un codificador y un decodificador Transformer, y OWSM v3 se entrenó con unas 180.000 horas.

OWSM v3.1 [33] cambia el codificador por un E-Branchformer (sección 5.4.4) con los mismos datos mejorando precisión y velocidad. Cubre 151 idiomas y se publica en tres tamaños: *base* (101 millones de parámetros), *small* (367 millones) y *medium* (1.020 millones). .En este trabajo se usa la version base de v3.1. Aunque hay versiones posteriores como v4, se ha escogido por v3.1 por tener mas información al comienzo del proyecto, pero cabe destacar que hay margen de mejora

5.4.2. Arquitectura

OWSM es un modelo codificador-decodificador con atención, al que se añade una cabeza CTC sobre la salida del codificador. Con audio, el proceso completo sería el siguiente:

1. ***Frontend*** La forma de onda se convierte en un espectrograma en escala mel y se normaliza.
2. **Submuestreo convolucional.** Dos convoluciones bidimensionales, que tratan el espectrograma como una imagen de tiempo por frecuencia, reducen la secuencia (por cuatro en OWSM v3.1). Una capa lineal la lleva a la dimensión d y se suma la codificación posicional.
3. **Codificador.** Una pila de bloques E-Branchformer transforma la secuencia en otra de la misma longitud, $\mathbf{H} = (\mathbf{h}_1, \ldots, \mathbf{h}_{T'})$, en la que cada vector describe lo que pasa en ese instante teniendo en cuenta toda la frase.
4. **Decodificador.** Un Transformer genera el texto token a token. En cada paso mira los tokens que ya ha escrito y, mediante la atención cruzada, la salida del codificador.

5. **Cabeza CTC.** Una capa lineal sobre cada $\mathbf{h}_t$ da una distribución sobre los tokens. Se usa en el entrenamiento y en la decodificación (sección 5.5).

En la versión base, la que usamos, el codificador y el decodificador tienen 6 capas cada uno y la dimensión de los vectores es 384

El vocabulario de OWSM son 50000 subpalabras obtenido con SentencePiece [60]. Las palabras frecuentes son un solo token y las raras se parten en varios trozos, así que el modelo puede escribir palabras que no ha visto nunca juntando trozos conocidos. Además hay tokens especiales para etiquetar como el idioma o las marcas de tiempo.

La figura 5.1 resume cómo se adapta esta arquitectura en este trabajo: la señal neuronal sustituye al audio y un *frontend* nuevo ocupa el lugar del espectrograma y el submuestreo, mientras que el codificador, el decodificador y la cabeza CTC son los de OWSM.

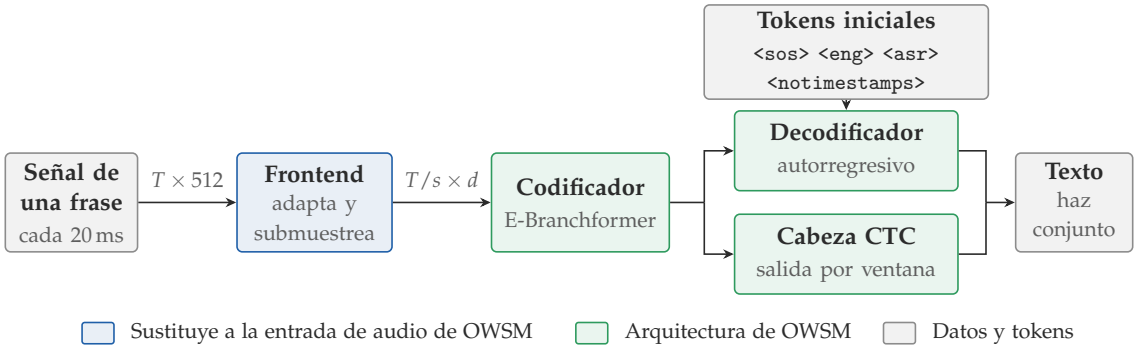


Figura 5.1: Esquema del modelo.

5.4.3. Atención

La atención es la operación que permite a cada posición de una secuencia mirar a las demás y decidir cuáles le importan, de modo que su representación tenga en cuenta el contexto. Cada vector de la secuencia se proyecta, con matrices aprendidas, en tres vectores: una consulta, una clave y un valor. La forma matricial de la operación es:

$$\text{Atención}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}, \tag{5.12}$$

donde las filas de $\mathbf{Q}$, $\mathbf{K}$ y $\mathbf{V}$ son las consultas, las claves y los valores, d_k es su dimensión y $\mathbf{M}$ es la máscara, con 0 en las posiciones permitidas y $-\infty$ en las que no se pueden mirar (sección 5.2.1). Cada salida es una media ponderada de los valores, con más peso en las posiciones que más se parecen a la consulta.

En la autoatención, las consultas, las claves y los valores salen de la misma secuencia; la usan el codificador y el decodificador. En la atención cruzada, que solo usa el decodificador, las consultas salen del texto y las claves y los valores, de la salida del codificador [59]. En la práctica, la operación se repite varias veces en paralelo con proyecciones distintas (atención multicabeza), para que cada cabeza pueda fijarse en un tipo de relación distinto.

Así, cualquier posición puede relacionarse con cualquier otra en una sola capa, pero el coste crece con el cuadrado de la longitud de la secuencia.

5.4.4. Codificador E-Branchformer

En el habla se distinguen dos escalas de patrones. Los locales duran unas pocas decenas de milisegundos, como el paso de un sonido al siguiente, y los globales abarcan la frase

entera. La atención capta bien lo global; las convoluciones captan bien lo local, pero solo ven lo que cabe en su núcleo. Por eso los codificadores de voz actuales combinan las dos: Conformer [61] las pone una detrás de otra, Branchformer [62] las pone en paralelo y E-Branchformer [63] mejora la forma de unirlas.

Cada bloque E-Branchformer recibe una secuencia $\mathbf{E} \in \mathbb{R}^{T' \times d}$ y calcula:

$$\mathbf{E}_1 = \mathbf{E} + \tfrac{1}{2}\,\mathrm{FFN}(\mathrm{LN}(\mathbf{E})), \tag{5.13}$$

$$\mathbf{E}_G = \mathrm{MHSA}(\mathrm{LN}(\mathbf{E}_1)), \qquad \mathbf{E}_L = \mathrm{cgMLP}(\mathrm{LN}(\mathbf{E}_1)), \tag{5.14}$$

$$\mathbf{E}_2 = \mathbf{E}_1 + \mathrm{Fusión}(\mathbf{E}_G, \mathbf{E}_L), \tag{5.15}$$

$$\mathbf{E}_3 = \mathbf{E}_2 + \tfrac{1}{2}\,\mathrm{FFN}(\mathrm{LN}(\mathbf{E}_2)), \tag{5.16}$$

$$\mathbf{E}_{\mathrm{sal}} = \mathrm{LN}(\mathbf{E}_3), \tag{5.17}$$

donde LN es la normalización por capas y MHSA, la autoatención multicabeza. Las piezas son estas:

Redes *feed-forward* (FFN). Dos capas lineales con una activación entre ellas, aplicadas a cada vector por separado. Hay una al principio y otra al final del bloque, y cada una suma solo la mitad de su salida.

Rama global. La autoatención multicabeza, que relaciona cada instante con toda la frase.

Rama local (cgMLP). Un perceptrón con una convolución dentro. La convolución filtra cada canal por separado a lo largo del tiempo, así que es barata y solo mira a los instantes vecinos. Su salida actúa como una puerta que decide cuánto deja pasar de cada valor.

Fusión. Junta las salidas de las dos ramas y las devuelve a la dimensión d. Antes de juntarlas les aplica otra convolución corta, de modo que la mezcla tiene en cuenta los instantes vecinos. Esta convolución es lo que añade E-Branchformer respecto a Branchformer.

Los bloques se apilan uno detrás de otro. Cada bloque mantiene la longitud y la dimensión, y la salida del último es la secuencia $\mathbf{H}$ que usan el decodificador y la cabeza CTC.

5.4.5. Decodificador Transformer

El decodificador genera la frase token a token. Para escribir el token y_l recibe los anteriores, $y_1, \ldots, y_{l-1}$, que se convierten en vectores con una tabla de *embeddings*, después se suma la codificación posicional. Cada una de sus capas tiene tres partes, todas con conexión residual y normalización:

1. **Autoatención con máscara causal.** Cada token mira solo a los anteriores y a sí mismo. La máscara causal pone $-\infty$ en las posiciones futuras, de modo que el modelo no puede ver lo que todavía no ha escrito.

2. **Atención cruzada.** Las consultas salen del texto y las claves y los valores, de la salida del codificador $\mathbf{H}$.

3. **Red *feed-forward***, como las del codificador.

Al final, se añade una capa lineal y una *softmax* sobre el último vector $\mathbf{g}_l$ para dar la probabilidad de cada token del vocabulario:

$$p(y_l \mid y_{<l}, \mathbf{X}) = \text{softmax}\left(\mathbf{W}_o\, \mathbf{g}_l + \mathbf{b}_o\right)_{y_l}. \tag{5.18}$$

Se minimiza la entropía cruzada:

$$\mathcal{L}_{\text{att}} = -\sum_{l=1}^{L} \log p(y_l \mid y_{<l}, \mathbf{X}). \tag{5.19}$$

En la inferencia no hay frase real: el modelo usa sus propias predicciones y sigue hasta que genera `<eos>`.

Como cada token depende de los anteriores, el decodificador aprende qué secuencias de palabras son probables, es decir, funciona como un modelo de lenguaje interno. El de OWSM lo ha aprendido con las transcripciones de cientos de miles de horas de habla, y por eso es la parte del modelo que aporta el conocimiento del lenguaje.

5.5 Clasificación temporal conexionista (CTC)

5.5.1. Formulación y cabeza CTC

Al entrenar un modelo de habla se conoce la frase, pero no en qué momento de la señal se dice cada token. CTC [64] permite entrenar sin esa información. El modelo emite un símbolo en cada ventana de salida del codificador, y se da por buena cualquier secuencia de símbolos que, una vez limpia, dé la frase correcta.

Para poder emitir algo en todas las ventanas, CTC añade al vocabulario un símbolo en blanco, $\varnothing$, que quiere decir «aquí no sale nada nuevo». Una secuencia con un símbolo de $\mathcal{V}' = \mathcal{V} \cup \{\varnothing\}$ por ventana se llama alineamiento, $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_{T'})$.

La cabeza CTC es la capa que da, en cada ventana, la probabilidad de cada símbolo. Es una capa lineal seguida de una *softmax* sobre $\mathcal{V}'$:

$$p(\pi_t \mid \mathbf{X}) = \text{softmax}\left(\mathbf{W}_c\, \mathbf{h}_t + \mathbf{b}_c\right)_{\pi_t}, \qquad t = 1, \ldots, T'. \tag{5.20}$$

Un alineamiento se convierte en texto con la función de colapso $\mathcal{B}$: primero se juntan los símbolos repetidos seguidos y después se quitan los blancos. Con letras se ve enseguida. En ocho ventanas, tanto `h h ∅ o l l ∅ a` como `∅ h o o l ∅ a a` se quedan en `hola`, y hay muchos más alineamientos que dan lo mismo. Para escribir dos símbolos iguales seguidos, como las dos `l` de `llama`, hace falta un blanco entre ellos (`l ∅ l`); sin él, el colapso los juntaría en uno. De aquí sale un límite: el codificador tiene que dar al menos una ventana por cada token de la frase, más una por cada pareja de tokens iguales seguidos.

Para calcular la probabilidad de la frase, CTC supone que, conocida la señal, lo que se emite en una ventana no depende de lo que se emite en las otras. Así, la probabilidad de un alineamiento es el producto de las de sus ventanas, y la de la frase es la suma de las de todos los alineamientos que colapsan a ella:

$$p(\boldsymbol{\pi} \mid \mathbf{X}) = \prod_{t=1}^{T'} p(\pi_t \mid \mathbf{X}), \tag{5.21}$$

$$p(\mathbf{y} \mid \mathbf{X}) = \sum_{\boldsymbol{\pi} \in \mathcal{B}^{-1}(\mathbf{y})} p(\boldsymbol{\pi} \mid \mathbf{X}). \tag{5.22}$$

El entrenamiento minimiza $\mathcal{L}_{\text{CTC}} = -\log p(\mathbf{y} \mid \mathbf{X})$, lo que sube a la vez la probabilidad de todos los alineamientos que llevan a la frase correcta. Son tantos que no se pueden sumar uno a uno: su número crece de forma exponencial con T'. Pero muchos comparten el principio, así que se puede avanzar ventana a ventana guardando sumas parciales. Este algoritmo de programación dinámica, llamado hacia delante y hacia atrás, da el resultado exacto con un coste proporcional a $T' \cdot L$, donde L es el número de tokens de la frase.

La independencia entre ventanas es lo que permite hacer esta cuenta, pero tiene un precio. Cada ventana decide sin saber qué han emitido las demás, de modo que CTC, por sí solo, no aprende qué palabras suelen ir juntas.

La figura 5.2 muestra algunos ejemplos con la palabra `cat` en seis ventanas. En la figura, el blanco se representa con ϵ.

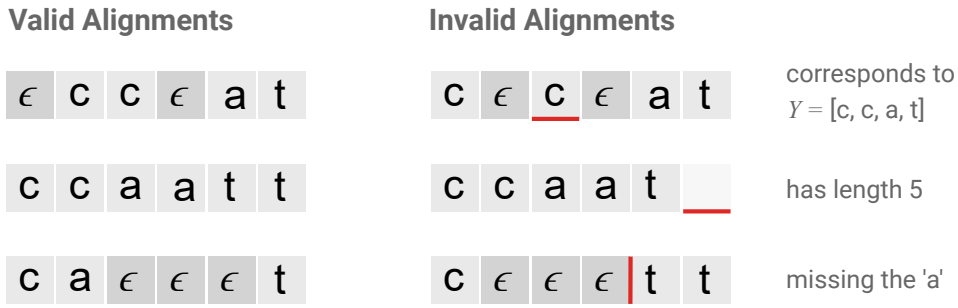


Figura 5.2: Alineamientos CTC válidos y no válidos para la salida `cat` en seis ventanas; ϵ es el símbolo en blanco. A la izquierda, tres alineamientos que colapsan a `cat`. A la derecha, tres que no: el primero da `ccat`, porque el blanco separa las dos `c`; el segundo solo tiene cinco símbolos para seis ventanas, y el tercero no emite la `a`. Adaptada de [65].

5.5.2. Pérdida conjunta e InterCTC

OWSM se entrena con CTC y con el decodificador a la vez, sumando las dos pérdidas con un peso λ [66], [67]:

$$\mathcal{L} = \lambda\, \mathcal{L}_{\text{CTC}} + (1 - \lambda)\, \mathcal{L}_{\text{att}}, \qquad \lambda \in [0, 1]. \tag{5.23}$$

Cada parte cubre un hueco de la otra. CTC solo admite alineamientos que avanzan en orden, del principio de la señal al final, y eso ayuda al decodificador a encontrar antes en qué parte de la señal se tiene que fijar. El decodificador, por su lado, tiene en cuenta los tokens anteriores, que es lo que le falta a CTC.

InterCTC [68] añade una segunda pérdida CTC a mitad del codificador. La misma cabeza se aplica a la salida de una capa intermedia k, además de a la de la última capa N, y las dos pérdidas se mezclan con un peso w:

$$\mathcal{L}_{\text{CTC}} = (1 - w)\, \mathcal{L}_{\text{CTC}}^{(N)} + w\, \mathcal{L}_{\text{CTC}}^{(k)}, \qquad k < N. \tag{5.24}$$

Con esto, las primeras capas reciben el gradiente de forma más directa y tienen que dar ya una salida que se pueda convertir en texto. Sirve de regularización y solo se usa durante el entrenamiento.

5.5.3. Decodificación

Con el modelo entrenado hay dos maneras de pasar de la señal al texto. La más sencilla solo usa la cabeza CTC: en cada ventana se queda con el símbolo más probable y luego aplica el colapso,

$$\hat{\mathbf{y}}_{\text{voraz}} = \mathcal{B}\left(\arg\max_{\pi_1} p(\pi_1 \mid \mathbf{X}), \ldots, \arg\max_{\pi_{T'}} p(\pi_{T'} \mid \mathbf{X})\right). \tag{5.25}$$

Es muy rápida, porque no pasa por el decodificador, y enseña lo que ha aprendido el codificador sin ayuda. Aun así, es una aproximación: la probabilidad de una frase está repartida entre muchos alineamientos, y la frase más probable no tiene por qué salir del alineamiento más probable.

La otra manera es la búsqueda en haz [67], que sí usa el decodificador. El texto se escribe token a token, pero en cada paso se guardan las K frases parciales con mejor puntuación, no solo la mejor. La puntuación de cada una, $\hat{y}_{1:l}$, junta las dos partes del modelo:

$$s(\hat{y}_{1:l}) = \lambda \log p_{\text{CTC}}(\hat{y}_{1:l} \mid \mathbf{X}) + (1-\lambda) \log p_{\text{att}}(\hat{y}_{1:l} \mid \mathbf{X}), \tag{5.26}$$

donde p_{att} es el producto de las probabilidades de la ecuación (5.18) y p_{CTC} es la probabilidad, según CTC, de que la frase empiece por $\hat{y}_{1:l}$. CTC castiga las frases que no cuadran con la señal, como las que repiten palabras o se alargan sin motivo, y el decodificador puntúa mejor las que tienen sentido. La búsqueda termina cuando las mejores hipótesis acaban en `<eos>`. En este trabajo no se usa ningún modelo de lenguaje externo, así que todo el conocimiento del lenguaje viene del decodificador.

5.6 Adaptación de un modelo preentrenado

Un modelo preentrenado es un modelo que ya se ha entrenado con muchos datos para una tarea. OWSM ha sido entrenado con mas de cien mil horas de audio. Reutilizarlo en una tarea nueva se llama aprendizaje por transferencia: en lugar de empezar de cero, se parte de unos pesos que ya han aprendido algo útil y se ajustan con los datos nuevos (*fine-tuning*) [69].

Entrenar un componente desde cero significa partir de pesos aleatorios que sortean valores pequeños, con una escala que depende del tamaño de la capa para que las señales no crezcan ni se apaguen al atravesar la red [70]. Hay casos concretos en los que sí tiene sentido empezar con un valor fijo, como la identidad de la capa por sesión (sección 5.2.2) o la matriz $\mathbf{B}$ de LoRA.

Cuando mencionamos congelar una capa, nos refierimos a pesos no se actualizan durante el entrenamiento. Sigue funcionando igual en la pasada hacia delante y el gradiente lo sigue atravesando hacia las capas anteriores, de modo que un decodificador congelado puede seguir indicando al codificador qué salida necesita. Descongelar es lo contrario, y se puede hacer con todo el componente o solo con algunas capas. También se puede dar a cada parte del modelo una tasa de aprendizaje distinta (sección 5.7), por ejemplo más baja para lo preentrenado que para lo nuevo.

5.6.1. LoRA

LoRA (*Low-Rank Adaptation*) es un método de aprendizaje por transferencia muy eficiente que ajuste cambiando muy pocos parámetros[38]. Parte de la idea de que el cambio que necesitan los pesos para la tarea nueva tiene rango bajo. Para una matriz preentrenada $\mathbf{W}_0 \in \mathbb{R}^{d\times k}$, en lugar de aprender un cambio completo $\Delta\mathbf{W}$, se aprende como el producto de dos matrices estrechas:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}\,\mathbf{B}\mathbf{A}, \qquad \mathbf{B} \in \mathbb{R}^{d\times r},\ \mathbf{A} \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{5.27}$$

$\mathbf{W}_0$ se queda congelada y solo se entrenan $\mathbf{A}$ y $\mathbf{B}$, así que los parámetros entrenables pasan de $d \cdot k$ a $r(d+k)$. $\mathbf{A}$ empieza con valores aleatorios y $\mathbf{B}$ a cero, de modo que al principio $\Delta\mathbf{W} = \mathbf{0}$ y el modelo se comporta exactamente como el preentrenado. El rango r controla cuánto puede cambiar la matriz y el factor α/r fija la escala del cambio. Suele aplicarse a las proyecciones de la atención, como las de las consultas y los valores. Como el cambio queda limitado a un subespacio pequeño, LoRA también actúa como regularización: el modelo no puede alejarse mucho de lo que ya sabía.

5.7 Entrenamiento

Esta sección resume los conceptos de entrenamiento que aparecen en el resto de la memoria. Los valores de cada experimento están en el capítulo 6.

Parámetros e hiperparámetros. Los parámetros son los pesos que aprende el modelo. Los hiperparámetros se fijan antes de entrenar, como la tasa de aprendizaje, el tamaño del lote o el número de épocas. Los parámetros entrenables son los que se actualizan durante el entrenamiento.

Conjuntos de datos. Divisiónes que realizamos sobre los datos para poder obtener métricas reales. El conjunto de entrenamiento sirve para ajustar los pesos. El de validación sirve para tomar decisiones durante el entrenamiento y el de prueba solo se usa al final, para medir el resultado con datos que no han influido en ninguna decisión.

Lote, paso y época. Un lote es el grupo de frases que se procesa a la vez. En cada paso se calcula la pérdida de un lote y se actualizan los pesos una vez. Una época es una pasada completa por el conjunto de entrenamiento.

Optimización. Entrenar es buscar los parámetros que minimizan la pérdida. En cada paso se calcula el gradiente de la pérdida con retropropagación y los parámetros se mueven un poco en la dirección contraria. Cuánto se mueven lo decide la tasa de aprendizaje η. Aquí se usa AdamW [71], [72], una variante que adapta el tamaño del paso a cada parámetro según cómo han ido sus gradientes anteriores. Además, aplica un decaimiento de pesos que los empuja poco a poco hacia cero y sirve de regularización.

Tasa de aprendizaje y calentamiento. Al principio del entrenamiento, los pasos grandes pueden desestabilizar el modelo, sobre todo en los Transformers. Por eso la tasa de aprendizaje empieza siendo pequeña y sube poco a poco durante unos pasos de calentamiento (*warmup*). Después va bajando despacio. ESPnet usa la regla del Transformer original [59]: la tasa sube en línea recta hasta su valor máximo y luego baja de forma proporcional a la inversa de la raíz cuadrada del número de pasos.

Recorte del gradiente. Si el gradiente es demasiado grande, se reduce para que su norma no pase de un umbral. Así un lote raro no puede dar un paso enorme y desestabilizar el entrenamiento [73].

Sobreajuste. Ocurre cuando el modelo aprende de memoria el entrenamiento y deja de generalizar: la pérdida de entrenamiento sigue bajando, pero la de validación se estanca o sube.

Parada temprana. Se evalúa en validación al final de cada época, se guarda la mejor y se para si no mejora durante un número de épocas seguidas, la paciencia [74].

Semilla. La inicialización, el orden de los lotes y el *dropout* dependen de números aleatorios. La semilla fija esa aleatoriedad para poder repetir un entrenamiento.

5.8 Métricas de evaluación

Durante el entrenamiento, ESPnet da muchas métricas en cada época. La más directa es la propia pérdida, la combinación de CTC y del decodificador de la ecuación (5.23). Es lo que el modelo optimiza, pero su valor depende del peso λ y no dice cuántos tokens o palabras acierta, así que no sirve para comparar. En este trabajo se usan dos métricas, cada una con un papel distinto: la exactitud de ESPnet, para comparar las épocas de un mismo modelo y quedarse con la mejor, y la tasa de error por palabra, para comparar modelos distintos entre sí.

5.8.1. Exactitud de ESPnet

La exactitud (*accuracy*, acc) es la proporción de tokens que acierta el decodificador cuando recibe como contexto los tokens reales de la frase (*teacher forcing*):

$$\text{acc} = \frac{1}{\sum_i L_i} \sum_i \sum_{l=1}^{L_i} \mathbb{I}\left[\arg\max_{v \in \mathcal{V}} p(v \mid y_{i,<l}, \mathbf{X}_i) = y_{i,l}\right], \tag{5.28}$$

donde L_i es el número de tokens de la frase i e $\mathbb{I}[\cdot]$ vale 1 si se cumple la condición y 0 si no.

Es el criterio que se usa durante el entrenamiento para elegir la mejor época de cada modelo. Su gran ventaja es que sale casi gratis: se obtiene en la misma pasada que la pérdida de validación, sin generar texto, así que se tiene en cada época. Además, mide si el decodificador acierta los tokens, que son los que al final forman el texto.

Su inconveniente es que resulta optimista. Con el contexto real, un error no afecta a los tokens siguientes; al generar texto, en cambio, un error al principio puede desviar el resto de la frase. También cuenta tokens y no palabras, y deja fuera la cabeza CTC y la búsqueda en haz. Por eso sirve para ordenar las épocas de un mismo modelo, pero no para comparar modelos distintos, que pueden fallar de maneras distintas al generar.

5.8.2. Tasa de error por palabra

Para comparar modelos hay que mirar el texto que escriben. Las métricas de texto se basan en la distancia de Levenshtein [75], que cuenta el número mínimo de errores entre la hipótesis y la referencia: sustituciones (S, palabras cambiadas), eliminaciones (D, palabras de la referencia que faltan en la hipótesis) e inserciones (I, palabras de más en la

hipótesis). La tasa de error por palabra (WER, *Word Error Rate*) divide ese número entre las N palabras de la referencia:

$$\mathrm{WER} = \frac{S + D + I}{N}. \tag{5.29}$$

Es la métrica estándar en reconocimiento de voz y la que dan los trabajos de decodificación del habla del estado del arte. Sobre un conjunto de frases se puede calcular de dos formas: como la media de los WER de cada frase, o sumando primero los errores y las palabras de todas. Lo habitual es lo segundo, el WER de corpus, que da a cada frase un peso proporcional a su longitud:

$$\mathrm{WER}_{\mathrm{corpus}} = \frac{\sum_i (S_i + D_i + I_i)}{\sum_i N_i}. \tag{5.30}$$

Cuanto más bajo es el WER, mejor. En el ejemplo de la tabla 5.1, la referencia tiene cuatro palabras y la hipótesis comete una sustitución y una inserción, así que el WER es $2/4 = 50\,\%$. Como el número de inserciones no tiene límite, el WER puede pasar del 100 %: si la referencia es *yes* y el modelo escribe *yes i do*, hay dos inserciones sobre una palabra y el WER de esa frase es del 200 %.

Tabla 5.1: Ejemplo de cálculo del WER. *S*: sustitución; *I*: inserción.

Referencia	*me*	*encantan*	*los*	*datos*	—
Hipótesis	*me*	*encantan*	*los*	*datos*	*cerebrales*
Error			*S*		*I*

El WER es la medida más fiel, pero no se puede calcular de forma eficiente en cada época. Para obtenerlo hay que generar el texto de cada frase con la búsqueda en haz (sección 5.5.3), token a token y guardando varias hipótesis a la vez, mientras que la exactitud sale de una sola pasada que procesa todos los tokens en paralelo. Por eso se calcula al final, con el modelo ya elegido.

La tasa de error por carácter (CER) y la tasa de error por fonema (PER) se calculan igual, pero con caracteres o fonemas en lugar de palabras. Son las métricas naturales cuando el modelo escribe esas unidades. El CER es además menos severo, porque una palabra con una sola letra mal cuenta como un error de una letra y no de una palabra entera; ESPnet lo calcula durante el entrenamiento sobre la salida de la cabeza CTC. Aquí el objetivo es escribir palabras, así que la medida final es el WER. Por último, algunos trabajos dan el acierto en lugar del error, que es su complementario: un acierto por palabra del 95 % corresponde a un WER del 5 %.

CAPÍTULO 6
Metodología

La metodología tiene dos fases, que corresponden a los dos bloques de experimentos de la sección 2.1. En la primera se busca el mejor modelo posible con un presupuesto de cómputo limitado, comparando lo que mejora cada cambio con lo que cuesta. En la segunda se toma ese modelo y se estudia cómo se comporta con el paso del tiempo y cuántos datos necesita. Antes se describen el sistema de partida y la forma de evaluarlo. Todos los entrenamientos se han hecho en Google Colab, con una sola GPU NVIDIA L4, usando ESPnet a través de su interfaz ESPnet-EZ [34], [35].

6.1 Adaptación de OWSM a la señal neuronal

El punto de partida es OWSM v3.1 en su versión base (sección 5.4). Se eligió OWSM porque es abierto y está entrenado solo con datos públicos, lo que permite reproducir el trabajo, y la versión base porque, con unos 100 millones de parámetros, se puede entrenar en una sola GPU y es un tamaño más razonable para las cerca de 8.000 frases de entrenamiento del conjunto. Para que trabaje con señal neuronal en lugar de audio se hacen estos cambios:

- El *frontend* acústico y el submuestreo convolucional se eliminan, porque están pensados para el espectrograma del audio y aquí la entrada ya son características. En su lugar se pone un *frontend* nuevo (sección 5.3) que recibe las 512 características de cada ventana, tal y como se publican, y entrega vectores de dimensión 384. La codificación posicional se reutiliza de OWSM, porque no depende del tipo de entrada.
- El codificador mantiene la arquitectura E-Branchformer de OWSM v3.1 base, con 6 bloques y $d = 384$. Según el experimento, sus pesos son los preentrenados o aleatorios.
- El decodificador es el de OWSM v3.1 base, con su vocabulario de subpalabras y sus tokens especiales. Cada frase se le presenta como `<eng><asr><notimestamps>` seguido del texto, igual que una transcripción en inglés. Así el decodificador hace la misma tarea para la que se entrenó y puede aprovechar lo que sabe del idioma.
- La cabeza CTC trabaja sobre el mismo vocabulario, de modo que CTC y el decodificador escriben con las mismas unidades. Se carga de OWSM y se sigue entrenando junto con el *frontend*.

El modelo se entrena con la pérdida conjunta de la ecuación (5.23), con $\lambda = 0{,}3$, el mismo valor con el que se entrenó OWSM. Se añade InterCTC en la segunda capa del codificador, con $w = 0{,}3$ (ecuación (5.24)), para que las primeras capas reciban un

gradiente más directo, lo que ayuda sobre todo cuando el codificador empieza desde cero. El *frontend* y la cabeza CTC usan una tasa de aprendizaje de 10^{-3}, y el codificador, una 20 veces menor: con la misma tasa para todo el modelo, el entrenamiento divergía. El resto de la receta está en el **??**.

6.2 Evaluación

Como el test oficial no tiene transcripciones públicas, la validación oficial hace de conjunto de prueba (sección 4.2). Es lo habitual con este conjunto: Khanday et al. solo dan resultados en validación [46], y LightBeam da los dos y obtiene cifras parecidas en validación y en las dos partes del test [39]. Las cuatro sesiones sin validación solo aportan datos de entrenamiento. Antes de calcular el error, la referencia y la hipótesis se normalizan para que las mayúsculas y la puntuación no cuenten como errores: se pasan a minúsculas y se quitan los signos de puntuación, salvo los apóstrofos y los guiones.

Cada métrica tiene un papel distinto:

Para comparar modelos. El WER de corpus (ecuación (5.30)) en toda la validación, con la búsqueda en haz de la sección 5.5.3: haz de 5, peso de CTC de 0,3, detección automática del final de la frase y sin modelo de lenguaje externo. Es la cifra con la que se comparan los modelos, y la configuración es la misma para todos. Se da también el WER de la decodificación CTC voraz, que muestra lo que consigue el codificador sin la ayuda del decodificador.

Para elegir la época. La acc de ESPnet (ecuación (5.28)), porque se tiene en cada época sin generar texto. En la fase 1 se calcula sobre la validación y en la fase 2, sobre una reserva sacada del entrenamiento (sección 6.4).

Para medir la eficiencia. El número de parámetros, totales y entrenables, y el tiempo de entrenamiento en horas de GPU. En los *frontends* se miden además, sin entrenar, las operaciones por segundo de señal (GFLOPs) y la memoria máxima que ocupan en la GPU.

Para comprobar que un modelo lee de verdad la señal, se evalúa también con una señal falsa: a cada frase se le da la señal de otra. Si el error apenas cambia, el modelo no está usando la señal y se limita a escribir frases probables con el decodificador.

Hay dos limitaciones que afectan a la lectura de los resultados. En la fase 1, la época se elige con la misma validación con la que se evalúa, lo que da cifras algo optimistas, aunque igual de optimistas para todos los modelos; la fase 2 lo evita con la reserva. Además, por coste, cada configuración se entrena una sola vez y con una sola semilla, así que diferencias de uno o dos puntos de WER pueden deberse al azar. Para tener una idea de cuánto pesa el azar de las frases, en el análisis de la fase 2 se calculan intervalos de confianza (sección 6.4.3), aunque no recogen la variación entre semillas.

6.3 Fase 1: búsqueda del mejor modelo

El objetivo de esta fase es encontrar el mejor modelo dentro de un coste razonable, es decir, ver qué cambios compensan lo que cuestan. Se parte de modelos sencillos y con pocos datos, y se van añadiendo, de uno en uno, elementos que usa la literatura. Después de la puesta en marcha se estudia primero el codificador, que es donde más se nota el cambio de modalidad, luego el *frontend* y por último el decodificador.

6.3.1. Puesta en marcha

Adaptar OWSM a la señal neuronal fue un proceso largo, que se hizo por pasos y de menos a más: las pruebas empezaron con casi todo el modelo congelado y lo fueron descongelando poco a poco.

Audio. Antes de tocar la señal neuronal, se ajustó OWSM con audio siguiendo el tutorial de ESPnet-EZ. Así se comprobó que la instalación, la carga del modelo preentrenado y el entrenamiento funcionaban, y se tuvo una referencia de cómo se comporta un entrenamiento que va bien.

Adaptación básica. El primer paso fue preparar los datos. Hubo que escribir la lectura de los ficheros HDF5, limpiar las transcripciones para que encajaran con el vocabulario de OWSM y con la normalización usada al evaluar, y montar un sistema de guardado en Google Drive: cada entrenamiento guarda sus pesos, su receta y sus resultados, y puede seguir donde se quedó si Colab se desconecta, algo habitual en entrenamientos de varias horas. Después se adaptó el modelo. En lugar de modificar el código de ESPnet, se construye a partir de una copia de OWSM en la que se cambian las dimensiones de entrada, se sustituye el *frontend* y se cargan los pesos de las partes que se reutilizan, comprobando que no falta ni sobra ninguno. También hubo que ajustar algunos detalles a lo que espera ESPnet, como la forma de pasar la máscara al nuevo *frontend*.

Prueba de sobreajuste. Una de las primeras pruebas fue comprobar que el modelo era capaz de aprender. Se entrenó con muy pocas frases, buscando a propósito el sobreajuste: si un modelo no puede aprenderse de memoria unas pocas frases, el problema está en la cadena de datos o en el propio modelo; si puede, la cadena funciona y lo que queda es un problema de generalización. El modelo llegó a aprenderse las frases (figura 6.1), así que la adaptación era correcta y el modelo podía aprender la tarea.

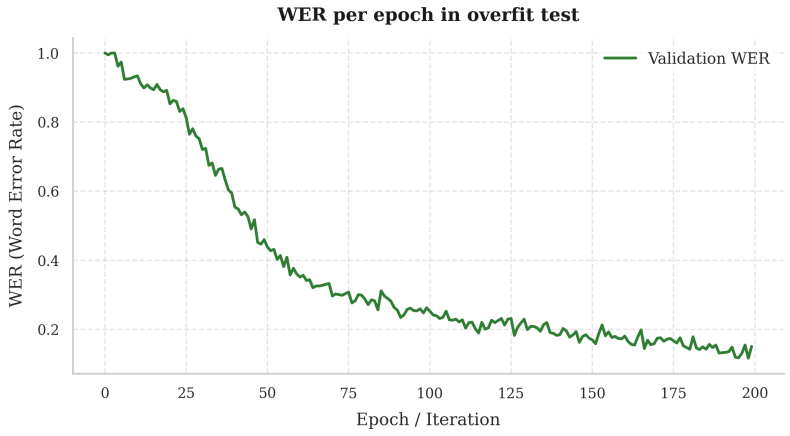


Figura 6.1: Prueba de sobreajuste: WER por época al entrenar con un subconjunto muy pequeño de frases. El modelo llega a aprendérselas de memoria, lo que muestra que la adaptación funciona y que el modelo puede aprender la tarea.

Primeras pruebas. A partir de ahí empezaron las pruebas con más datos, que fueron creciendo poco a poco. Las primeras se hicieron con unas diez sesiones, porque cada época tardaba unos 20 minutos en el equipo disponible, y con modelos sencillos: un *frontend* lineal y casi todo OWSM congelado, con solo unas pocas capas del codificador

entrenables. Se probaron muchas combinaciones y los resultados eran bajos. En la mayoría, el decodificador acababa escribiendo casi la misma frase para cualquier entrada, con un error por carácter cercano al 99 %. El cambio llegó al ampliar la escala en Colab: se pasó de unas diez sesiones a las 45, se descongelaron más capas del codificador y el *frontend* lineal se cambió por uno convolucional profundo. Como estos cambios se hicieron a la vez, no se puede saber cuánto aportó cada uno, aunque lo que más parece haber influido es usar todas las sesiones.

Con lo aprendido se fijó una receta general (**??**): como mucho 80 épocas, con parada temprana, quedándose con la época de mejor acc en validación. Desde aquí, cada experimento de la fase cambia una sola cosa respecto a esa receta, para poder atribuirle la diferencia.

6.3.2. Codificador (OE2)

El codificador es la parte del modelo que convierte la señal en una representación que el decodificador pueda leer, así que es la que se enfrenta al cambio de modalidad: OWSM aprendió a codificar audio y ahora recibe actividad neuronal. Por eso fue lo primero que se estudió. Responde a la pregunta principal del trabajo, si lo que OWSM aprendió del audio sirve para la señal neuronal, y es además el paso más grande, porque de él depende todo lo demás. Si el codificador no aprende a leer la señal, da igual qué *frontend* o qué decodificador se usen. Y la decisión cambia lo que se pide al resto del modelo: con un codificador preentrenado, el *frontend* tiene que darle algo parecido a lo que esperaba del audio; con uno aleatorio, basta con que le dé una representación útil.

Siguiendo la progresión de la puesta en marcha, de congelado a descongelado, se comparan cuatro formas de tratar el codificador (sección 5.6), de más a menos uso de lo que aprendió OWSM:

- preentrenado y congelado, de modo que solo aprenden el *frontend* y la cabeza CTC, que tienen que traducir la señal a lo que el codificador entiende;
- preentrenado con LoRA, que permite pequeños ajustes con pocos parámetros;
- preentrenado y ajustado entero, con la tasa de aprendizaje reducida;
- con pesos aleatorios, es decir, la misma arquitectura entrenada desde cero.

El resto del sistema y la receta no cambian. Si el conocimiento del audio se transfiere, las variantes preentrenadas deberían aprender antes o llegar más lejos que la aleatoria. Aquí es donde más útil resulta la señal falsa, porque distingue un codificador que lee la señal de uno que deja todo el trabajo al decodificador. Una vez elegida la forma de tratar el codificador, el resto de la fase 1 se construye sobre ella.

6.3.3. *Frontend* (OE1)

Con el codificador elegido y el decodificador congelado, se comparan los *frontends* de la tabla 6.1. Se estudian por dos motivos. El *frontend* decide cuántas ventanas procesa el codificador, que es la parte cara del modelo, así que es donde más se puede ahorrar. Además, cada sistema de la literatura usa uno distinto y ninguno publica una comparación entre ellos. Van de menos a más complejos, cada uno añade algo al anterior y casi todos están tomados de la literatura. Los cuatro primeros reducen la secuencia a la mitad; los dos últimos, a la cuarta parte, que es lo habitual en los sistemas revisados (un vector cada 80 ms).

Tabla 6.1: *Frontends* comparados en la fase 1.

Frontend	Qué hace	Submuestreo
Lineal ×2	Apila 2 ventanas y aplica una capa lineal.	×2
Patching ×2	Agrupa 14 ventanas (280 ms) con paso 2 y aplica una capa lineal, como el sistema de referencia de Brain-to-Text '25 [30].	×2
Conv1d ×2	Dos convoluciones temporales de núcleo 3 con GELU, como la entrada de Whisper [36].	×2
Profundo ×2	Diseño propio (ver el texto), con una etapa.	×2
Profundo ×4	Diseño propio, con dos etapas.	×4
Patching ×4	El *frontend* del sistema de referencia de Brain-to-Text '25 tal cual.	×4

El *frontend* profundo empieza con una capa lineal de 512 a 512 con LayerNorm y GELU, que mezcla todos los electrodos entre sí. Después vienen una o dos etapas, cada una con una convolución de núcleo 5 y paso 2, que reduce la secuencia a la mitad, y dos bloques convolucionales residuales de núcleo 5 con LayerNorm, GELU y *dropout* de 0,1. Termina con una capa lineal a 384 y la codificación posicional. Con una etapa, cada vector de salida ve 420 ms de señal. El submuestreo convolucional original de OWSM y una variante con una capa recurrente solo se midieron en coste, sin entrenarlos, porque eran más lentos.

Todos se entrenan con la misma receta y la misma semilla, durante 40 épocas y sin parada temprana, para que tengan el mismo presupuesto. Así la comparación es justa, aunque las cifras no son el máximo que podría dar cada uno. El profundo ×2 es el mismo sistema de los apartados anteriores, de modo que sirve también de control: sus primeras 40 épocas deben reproducir las de aquel entrenamiento.

Junto con los *frontends* se probó la capa de entrada por sesión (sección 5.2.2), que usan casi todos los sistemas del campo para compensar el cambio de la señal entre días. Se entrenó el mismo sistema con ella y sin ella. No mejoró el resultado y, además, tiene un problema de fondo para este trabajo: solo existe para los días con datos de entrenamiento, y en la fase 2 lo que interesa son los días nuevos. Por eso no forma parte de la receta.

6.3.4. Decodificador (OE3)

Por último se estudia el decodificador, que es la parte que aporta el conocimiento del lenguaje y la última que se descongeló. Se comparan tres variantes:

- el decodificador preentrenado y congelado, que es el más barato y el que se ha usado en los apartados anteriores;
- el mismo con LoRA (sección 5.6.1), de rango 8, en las proyecciones de consultas y valores de la autoatención y de la atención cruzada de cada capa, que lo adapta a esta tarea entrenando muy pocos parámetros;
- un decodificador con la misma arquitectura entrenado desde cero, que dice si lo que el decodificador aprendió en OWSM sirve de algo o si bastan las frases de entrenamiento.

Estas variantes se entrenaron con el *frontend* profundo ×2. Como no cuestan lo mismo, si dos dan un WER parecido se elige la más barata. La variante que se elige aquí es la que se usa en el resto del trabajo.

La combinación de codificador, *frontend* y decodificador que sale de esta fase es la receta final, la que se usa en la fase 2.

6.4 Fase 2: comportamiento en el tiempo y cantidad de datos

6.4.1. Variabilidad entre sesiones (OE4)

Antes de simular el uso real, se evalúa sesión a sesión, sin reentrenarlo, el modelo de la fase 1 con el decodificador congelado, que se entrenó con todas las sesiones. Así se ve cuánto cambia el error de un día a otro incluso cuando el modelo ha visto datos de todos los días. Es una referencia necesaria para la fase 2: si el error ya cambia mucho entre días conocidos, hay que tenerlo en cuenta al leer el error en días nuevos.

6.4.2. Entrenamiento cronológico (OE4)

Evaluar con frases de los mismos días que se han usado para entrenar es optimista y está lejos del uso real de una BCI, en la que el sistema se usa en días que todavía no se han grabado. Por eso se simula ese uso: se graban sesiones, se entrena con ellas y se usa en los días siguientes. Para ello se entrena desde cero un modelo con las k primeras sesiones, en orden de fecha, para $k = 5, 10, \ldots, 40$; con las 45 ya no quedarían sesiones futuras. Entrenar un modelo para cada valor de k habría costado demasiado, así que se avanza de cinco en cinco. Todos usan la receta final de la fase 1.

Como aquí la validación oficial se usa solo para evaluar, cada modelo aparta como reserva 1 de cada 10 frases de entrenamiento de cada sesión. Con ella se elige la mejor época y se hace la parada temprana, con una paciencia de 6 épocas. Cada modelo da como mucho 40 pasadas por sus datos y como poco 4.000 pasos, para que los modelos con pocas sesiones también entrenen un mínimo. Las épocas de ESPnet se fijan en 250 pasos, unas 8.000 frases, para todos los modelos. Así la paciencia equivale al mismo número de pasos en todos, aunque con pocas sesiones una época suponga varias pasadas por los datos: con $k = 5$, unas siete.

Cada modelo se evalúa en toda la validación. De cada frase se guarda su relación con el entrenamiento: si su sesión estaba entre las de entrenamiento (sesión vista) o es posterior (sesión futura), cuántas sesiones y cuántos días han pasado desde la última sesión de entrenamiento, si la frase aparece tal cual en el entrenamiento, cuánto se parece a la frase de entrenamiento más parecida y cuántas de sus palabras aparecen en él. Esto permite separar el efecto del paso del tiempo del de que la frase o sus palabras ya estuvieran en el entrenamiento, que es otra fuente de optimismo. Con esto se comparan:

- el error en sesiones vistas y en sesiones futuras para cada k;
- el error en las sesiones futuras con el de un modelo que sí las vio: el profundo $\times 4$ de la fase 1, entrenado con todas las sesiones y evaluado sobre las mismas frases;
- el error en las cinco últimas sesiones (de la 41 a la 45, de enero a abril de 2025), que son futuras para todos los modelos y permiten compararlos sobre las mismas frases.

También se guardan las curvas de aprendizaje, para comprobar que cada modelo llega a su meseta y que un mal resultado no se debe a haber entrenado poco.

6.4.3. Análisis del modelo

El WER resume en una cifra cuánto se equivoca un modelo, pero no dice en qué. Por eso, con las hipótesis guardadas de los modelos cronológicos y del modelo de la fase 1 entrenado con todas las sesiones, se analizan los errores con más detalle. Todo se calcula por separado en sesiones vistas, en sesiones futuras y en las cinco últimas sesiones.

Cómo salen las frases. Además del WER y del CER, se cuenta qué parte de los errores son sustituciones, eliminaciones e inserciones, y cómo se reparten las frases según su error: perfectas, con un error de cada cuatro palabras o menos, con hasta la mitad mal y con más de la mitad mal. Un mismo WER puede venir de muchas frases con algún error o de unas pocas totalmente equivocadas, y no es lo mismo para quien usa el sistema.

Intervalos de confianza. Como cada modelo se entrena una sola vez, se calcula un intervalo de confianza del WER por *bootstrap* sobre las frases [76]: se sacan frases al azar con reemplazo, se recalcula el WER y se repite 2.000 veces. Para comparar dos modelos se remuestrean las mismas frases en los dos. Si el intervalo de la diferencia no incluye el cero, la diferencia no se explica por qué frases han tocado en la validación.

Palabra a palabra. Cada referencia se alinea con su hipótesis con la misma distancia de Levenshtein del WER, de modo que cada palabra de la referencia queda como acertada, sustituida (y por cuál) o eliminada, y cada palabra de más, como inserción. Con esto se miran tres cosas:

- **El tipo de palabra.** Con la clase gramatical de cada palabra, que se obtiene con spaCy [77], se separan las palabras de contenido (sustantivos, verbos, adjetivos, adverbios, nombres propios y números) de las funcionales (pronombres, determinantes, preposiciones, auxiliares y conjunciones). Las funcionales son muy frecuentes y fáciles de adivinar, así que un modelo puede bajar el WER acertándolas aunque falle las que llevan el significado. Por eso se mide también, en cada frase, si acierta su palabra clave, la palabra de contenido menos frecuente, y si acierta todas sus palabras de contenido, que es una aproximación a si la frase se entiende.
- **Cuántas veces la ha visto.** El acierto de cada palabra según cuántas veces salía en el entrenamiento de ese modelo, incluidas las que no salían nunca. Esto dice cuántos ejemplos necesita una palabra para que el modelo la aprenda y si el decodificador es capaz de escribir palabras que el modelo no ha visto nunca con la señal.
- **Cómo evolucionan con k.** El acierto de las 20 palabras de contenido más frecuentes, una a una, para cada modelo cronológico.

Qué aporta el decodificador. Se compara la búsqueda en haz con la decodificación CTC voraz, que no usa el decodificador, en WER y en la proporción de palabras inventadas, que no existen en un diccionario de inglés ni en las frases del conjunto. CTC escribe subpalabra a subpalabra y puede formar palabras que no existen; el decodificador debería evitarlo. También se mira si las sustituciones se parecen en la pronunciación más de lo que se parecerían dos palabras al azar. Si el codificador lee la articulación, al fallar debería escribir palabras que suenan parecido; si es el decodificador el que escribe a ciegas, no.

Qué escribe cuando falla. Por último, se cuenta cuántas hipótesis distintas da cada modelo, cuántas son una frase del entrenamiento copiada tal cual y cuánto miden respecto a la referencia. En los días nuevos, un modelo que no entiende la señal tiende a repetir las mismas frases o a escribir frases que ya conoce, y esto permite verlo.

CAPÍTULO 7
Resultados y discusión

Este capítulo sigue el orden de la metodología. Primero se presentan los experimentos de la fase 1, con los que se elige el sistema, y después los de la fase 2, que estudian cómo se comporta con el paso del tiempo. Al final se compara el resultado con el estado del arte y se discute todo en conjunto.

Salvo que se diga otra cosa, las cifras son el WER de corpus con la búsqueda en haz sobre toda la validación (sección 6.2). Cada configuración se entrenó una sola vez, así que las diferencias pequeñas hay que leerlas con cuidado. Como referencia, para un WER de 0,209 el intervalo de confianza del 95 % calculado sobre las frases es de unos $\pm$0,012, de modo que diferencias de una o dos centésimas pueden deberse a qué frases hay en la validación.

7.1 Fase 1: búsqueda del mejor modelo

7.1.1. Codificador

La tabla 7.1 compara las cuatro formas de tratar el codificador, de más a menos uso de los pesos de OWSM. El preentrenado ajustado y el aleatorio usan el *frontend* profundo $\times 2$ y el decodificador congelado, y se entrenan con la misma receta.

Tabla 7.1: Comparación de las cuatro formas de tratar el codificador. WER y CER con la búsqueda en haz, WER con la decodificación CTC voraz y WER con la señal falsa (200 frases).

Codificador	WER haz	CER haz	WER CTC voraz	WER señal falsa
Preentrenado y congelado	0,971	0,852	0,987	0,992
Preentrenado con LoRA	0,971	0,814	0,989	1,006
Preentrenado y ajustado	0,967	0,780	0,972	0,992
Aleatorio	**0,209**	**0,161**	**0,273**	1,160

Ninguna de las tres variantes que usan los pesos de OWSM llega a aprender: congelado o con LoRA, el WER se queda en 0,971, y ajustado entero, en 0,967. Con pesos aleatorios, en cambio, el sistema llega a un WER de 0,209. El ajustado entero se paró en la época 20, cuando seguía estancado, así que no dura lo mismo que el aleatorio, pero eso no cambia la conclusión, porque en la época 20 el codificador aleatorio ya tenía una acc de validación de unos 0,83.

La señal falsa explica qué está pasando. Con el codificador preentrenado, el WER casi no cambia al darle a cada frase la señal de otra (0,992 frente a 0,971 con el congelado, 1,006 frente a 0,971 con LoRA y 0,992 frente a 0,967 con el ajustado), así que el modelo no está

leyendo la señal: el decodificador escribe frases en inglés que no tienen que ver con lo que se intentaba decir. Por eso el CER, de entre 0,780 y 0,852, es bastante menor que el WER, como cabe esperar de frases bien escritas pero sin relación con la referencia, que comparten letras y casi ninguna palabra. La rama CTC, que depende solo del codificador, se queda en un WER de entre 0,972 y 0,989: prácticamente no produce texto útil. Con el codificador aleatorio pasa lo contrario. La señal falsa dispara el WER de 0,209 a 1,160, lo que demuestra que el modelo se apoya en la señal, y el CTC voraz llega a 0,273 sin la ayuda del decodificador.

La explicación más probable, que se comentó con los tutores, tiene dos partes. La primera es que la señal neuronal se parece poco al audio: son 512 características de los electrodos cada 20 ms, no un espectrograma, y lo que el codificador aprendió de los sonidos no encaja con lo que le llega ahora. La segunda es que el conjunto es de una sola persona y tiene bastantes datos, suficientes para que un codificador aprenda desde cero.

Otros trabajos han encontrado lo contrario. Fiedler et al. ajustaron wav2vec 2.0 con la señal de otro participante, y con los pesos preentrenados obtuvieron un WER del 31,0 %, frente al 66,2 % entrenando desde cero [51]. Entre los dos trabajos cambian muchas cosas: el modelo y la forma en que se preentrenó (wav2vec 2.0 aprende directamente de la forma de onda y OWSM, de espectrogramas), el participante y los datos. Con un solo experimento no se puede saber cuál pesa más. Lo que sí muestra este resultado es que, en este sistema, el codificador de OWSM no aporta nada, y el resto del trabajo usa el codificador aleatorio.

7.1.2. *Frontend*

Con el codificador aleatorio y el decodificador congelado, la tabla 7.2 compara los seis *frontends* tras 40 épocas. La figura 7.1 muestra cómo aprende cada uno y la figura 7.2, el error frente al coste.

Tabla 7.2: Comparación de *frontends* tras 40 épocas, con el codificador aleatorio y el decodificador congelado: WER con la búsqueda en haz y con la decodificación CTC voraz, acc de la mejor época en validación, parámetros del *frontend*, operaciones por segundo de señal del *frontend* y el codificador, y horas de entrenamiento en una L4. En negrita, los mejores WER y acc.

Frontend	WER haz	WER CTC	acc	Parámetros (M)	GFLOPs/s	Horas
Lineal $\times 2$	0,243	0,314	0,853	0,39	1,37	5,1
Patching $\times 2$	0,305	0,462	0,824	2,75	1,49	5,2
Conv1d $\times 2$	0,254	0,389	0,849	1,03	1,43	5,2
Profundo $\times 2$	0,224	0,293	0,864	4,40	1,58	5,2
Profundo $\times 4$	**0,206**	**0,287**	**0,877**	8,33	0,98	3,0
Patching $\times 4$	0,329	0,453	0,810	2,75	0,72	2,9

Los dos *frontends* profundos son los mejores y los únicos que bajan de 0,24. El $\times 4$ consigue el WER más bajo, 0,206, y la mejor acc, 0,877. Su rama CTC es también la que mejor funciona por sí sola: con la decodificación voraz llega a 0,287, y el CER de la rama en validación acaba en 0,235, frente a 0,281 del $\times 2$ (figura 7.1b).

Además, el $\times 4$ es el más barato de los que funcionan bien. Entrena en 3,0 horas frente a las 5,2 del $\times 2$, un 42 % menos, porque el codificador procesa la mitad de vectores. Por lo mismo, hace un 38 % menos de operaciones por segundo de señal (0,98 GFLOPs frente a 1,58), necesita menos de la mitad de memoria al entrenar (7,9 GB de pico frente a 17,0) y transcribe cada frase con la búsqueda en haz en casi la mitad de tiempo (0,77 s frente a 1,37). Y todo ello aunque su *frontend*, con una etapa de convolución más, tiene casi el doble de parámetros (8,3 millones frente a 4,4): lo que añade el *frontend* se recupera con creces

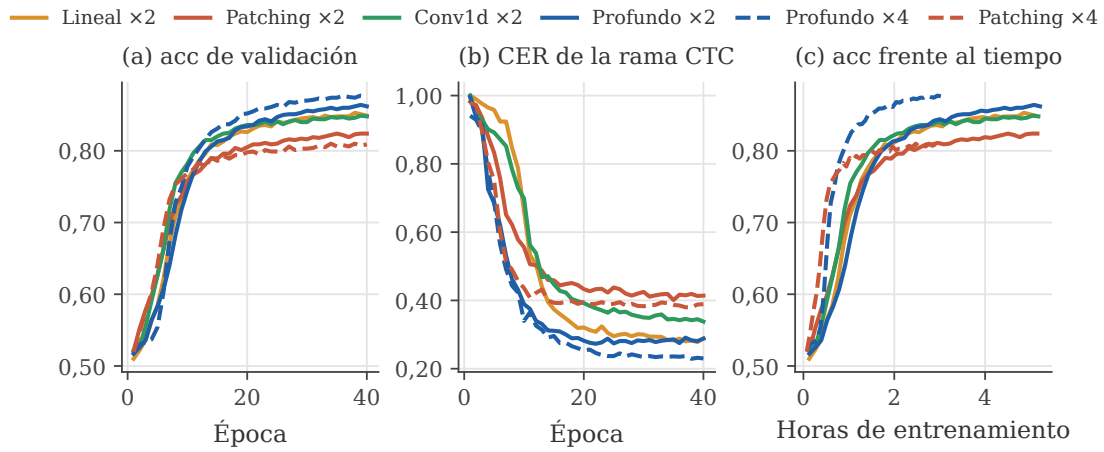


Figura 7.1: Entrenamiento de los seis *frontends* durante 40 épocas: acc de validación por época (a), CER de la rama CTC en validación (b) y acc frente a las horas de entrenamiento en una L4 (c). Las líneas discontinuas son los *frontends* ×4.

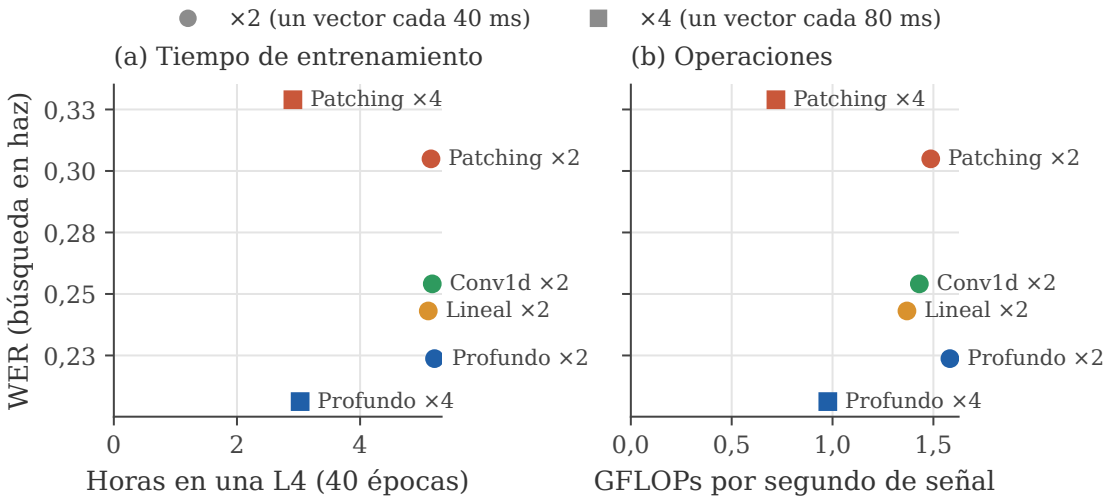


Figura 7.2: WER de cada *frontend* frente al coste: horas de entrenamiento en una L4 durante 40 épocas (a) y operaciones por segundo de señal del *frontend* y el codificador (b). Los círculos son los *frontends* ×2 y los cuadrados, los ×4. Abajo a la izquierda es mejor.

en el codificador. En este caso, reducir la secuencia a la cuarta parte abarata el modelo y además baja el error.

Con el *patching* ocurre lo contrario: el $\times 4$ (0,329) es peor que el $\times 2$ (0,305), y los dos son los peores de la comparación, también en la rama CTC (0,453 y 0,462), aunque el $\times 4$ es el *frontend* del sistema de referencia del Brain-to-Text '25. Al principio están entre los que más rápido aprenden, pero hacia la época 10 se estancan por debajo de los demás (figura 7.1a). Una posible explicación es que juntan 14 ventanas de golpe con una sola capa lineal, de 2,75 millones de parámetros, que tiene muchos pesos que aprender. En el sistema de referencia, además, va acompañado de una capa de entrada por día y de una red recurrente, que aquí no están [30].

El lineal, el más sencillo de todos y con solo 0,39 millones de parámetros, queda tercero (0,243) y mejora al conv1d (0,254), que imita la entrada de Whisper. Como el codificador ya modela el tiempo, un *frontend* sencillo funciona razonablemente bien, y lo que añade el profundo $\times 2$ frente al lineal son 0,019 de WER con casi el mismo tiempo de entrenamiento.

El profundo $\times 2$ es el mismo sistema de los apartados anteriores, que allí se entrenó 80 épocas y llegó a 0,209. Con 40 se queda en 0,224: entrenar el doble mejora algo, pero poco.

Los otros dos *frontends*, el submuestreo convolucional original de OWSM y la variante con una capa recurrente, solo se midieron en coste. El de OWSM necesita 5,35 GFLOPs por segundo de señal, más de cinco veces los del profundo $\times 4$, y tiene 20,1 millones de parámetros solo en el *frontend*. La variante recurrente hace menos operaciones (1,54 GFLOPs), pero su capa recurrente recorre los vectores uno detrás de otro y aprovecha peor la GPU. En la prueba de velocidad, cada lote de ocho frases tardaba 0,28 s con el primero y 0,33 s con el segundo, frente a 0,19 s del profundo $\times 4$, es decir, entre un 45 y un 73 % más, así que no se entrenaron.

En conjunto, el profundo $\times 4$ es el mejor compromiso entre resultado y coste: es el que menos se equivoca y, de los que funcionan bien, el más barato. Es el que pasa a la receta final.

7.1.3. Capa de entrada por sesión

Con tres de los *frontends* se entrenó también el mismo sistema con una capa de entrada por sesión (tabla 7.3). La capa cambia el WER en menos de 0,015 y no siempre a mejor: con el conv1d sube de 0,254 a 0,259, con el profundo $\times 2$ baja de 0,224 a 0,211 y con el profundo $\times 4$, el mejor, apenas se mueve, de 0,206 a 0,203. Son diferencias del mismo orden que el margen de error de cada cifra (unos $\pm 0{,}012$), así que en este sistema la capa no aporta nada apreciable, aunque añade unos 11,8 millones de parámetros. Esto no coincide con lo que se hace en el campo, donde casi todos los sistemas la usan. Puede que el resto del modelo ya compense las diferencias entre los días que ha visto, pero con un solo entrenamiento por configuración no se puede comprobar.

Tabla 7.3: WER con la búsqueda en haz de tres *frontends* sin y con capa de entrada por sesión, con la misma receta que la comparación de *frontends*. La capa añade unos 11,8 millones de parámetros.

Frontend	Sin capa	Con capa	Diferencia
Conv1d $\times 2$	0,254	0,259	$+0{,}005$
Profundo $\times 2$	0,224	0,211	$-0{,}013$
Profundo $\times 4$	0,206	0,203	$-0{,}003$

En cualquier caso, la capa tiene un problema de fondo para este trabajo: solo existe para los días con datos de entrenamiento. En un día nuevo, que es lo que estudia la fase 2,

no habría ninguna matriz aprendida y habría que recalibrar con datos de ese día. Por eso la receta final no la incluye.

7.1.4. Decodificador

Por último, la tabla 7.4 compara las tres formas de tratar el decodificador, todas con el codificador aleatorio y el *frontend* profundo $\times 2$, y la figura 7.3 muestra sus curvas. El decodificador entrenado desde cero se probó con dos tasas de aprendizaje: la del *frontend* y la del codificador, veinte veces menor.

Tabla 7.4: Comparación de decodificadores, con el codificador aleatorio y el *frontend* profundo $\times 2$: acc de la mejor época en validación, WER y CER con la búsqueda en haz, WER con la decodificación CTC voraz y WER con la señal falsa (200 frases). En negrita, el mejor valor de cada columna de error y acierto; en rojo, estimaciones.

Decodificador	acc	WER haz	CER haz	WER CTC voraz	WER señal falsa
Preentrenado y congelado	0,873	0,209	0,161	**0,273**	1,160
Preentrenado con LoRA	**0,885**	**0,202**	**0,153**	0,306	1,156
Desde cero, tasa del *frontend*	0,770	≈0,44	≈0,39	≈0,29	pendiente
Desde cero, tasa del codificador	0,726	0,542	0,491	0,292	1,066

Nota Estoy reentrenando esta parte por los errores que tuve

[Figura pendiente: falta `images/07_1_4_curvas_decoders.pdf`]

Figura 7.3: Comparación de decodificadores, todos con el codificador aleatorio: acc de validación (izquierda), acc de entrenamiento (centro) y CER de la rama CTC en validación (derecha). El punto marca la mejor época de cada uno. «Aleatorio» es el decodificador entrenado desde cero, con la tasa del *frontend* ($\times 1$) o con la del codificador ($\times 0{,}05$).

El decodificador entrenado desde cero es claramente peor. Con la tasa del *frontend*, su acc de validación se queda en 0,770, frente a 0,873 del congelado, y en entrenamiento llega casi al 100 % de acierto antes de la época 25, mucho antes que los demás. Aprende de memoria las frases de entrenamiento, pero 8.000 frases no bastan para aprender el idioma, mientras que el decodificador de OWSM trae lo que aprendió de 180.000 horas de transcripciones. Con la tasa del codificador aprende mucho más despacio: tras 62 épocas tiene una acc de 0,726 y sigue subiendo, y su WER, 0,542, es más del doble que el del congelado.

La rama CTC, por su parte, llega a valores parecidos en las cuatro variantes, con un CER en validación de entre 0,20 y 0,24, así que el codificador aprende más o menos lo mismo en todos los casos y la diferencia está en el decodificador. Con un decodificador entrenado desde cero, de hecho, el CTC voraz (0,292) es mucho mejor que la búsqueda en haz (0,542): el decodificador empeora lo que el codificador ya sabe leer. La señal falsa confirma que todos estos modelos leen la señal, porque con ella el WER pasa de 1 en los tres en los que se ha medido.

Con LoRA, el WER baja de 0,209 a 0,202. Es una diferencia de 0,007, más pequeña que el margen de error de cada cifra (unos $\pm 0{,}012$), así que puede deberse al azar, y su rama CTC

sale algo peor (0,306 frente a 0,273). Además, LoRA añade 0,15 millones de parámetros que entrenar. Por eso se mantiene el decodificador congelado, que da prácticamente lo mismo con menos coste.

7.1.5. Resumen de la fase 1

La fase 1 deja como receta final un codificador entrenado desde cero, el *frontend* profundo $\times 4$ y el decodificador de OWSM congelado. Con 40 épocas, unas 3 horas en una L4, llega a un WER de 0,206 en la validación. La decisión que más pesa es la del codificador: con los pesos de OWSM el sistema no funciona. El decodificador preentrenado también es necesario, porque entrenado desde cero generaliza mucho peor, aunque entre congelarlo o ajustarlo con LoRA apenas hay diferencia. El *frontend* cambia el WER en hasta 0,12 y decide además buena parte del coste, mientras que la capa de entrada por sesión no cambia el resultado de forma apreciable. Esta es la receta que se usa en la fase 2.

7.2 Fase 2: comportamiento en el tiempo

7.2.1. Variabilidad entre sesiones

La figura 7.4 muestra el WER de cada sesión con el modelo de la fase 1 entrenado con todas las sesiones, el profundo $\times 4$, que tiene un WER global de 0,206. Aunque el modelo ha visto frases de todos los días, el error cambia mucho de una sesión a otra: va de menos de 0,05 en las sesiones 19, 20 y 21 a más de 0,40 en las sesiones 13, 14, 16, 40 y 44. No sigue ninguna tendencia con el tiempo: hay sesiones difíciles al principio y al final, y las mejores están en medio. Algunas tienen pocas frases de validación (la 16 tiene 9, y las 19 a 21, entre 15 y 25, frente a unas 35 de media), así que su WER es menos preciso, pero las diferencias son demasiado grandes para explicarse solo por eso.

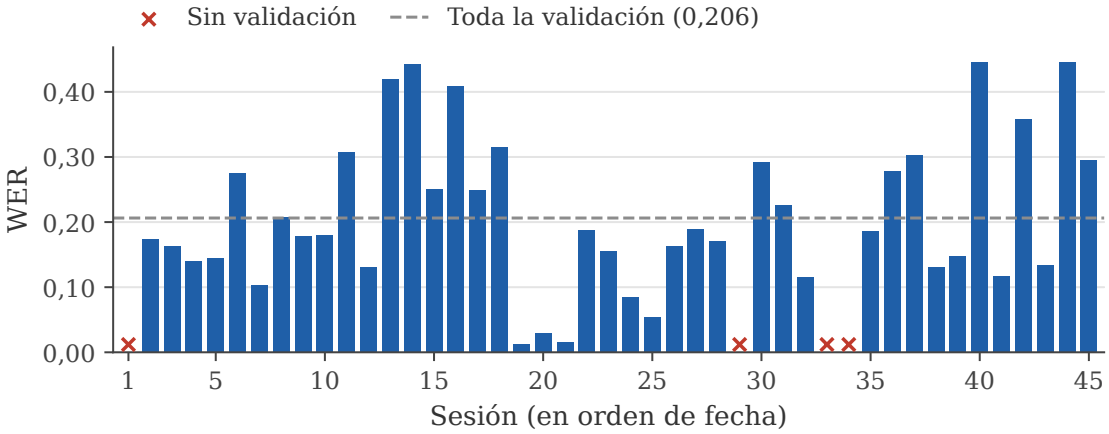


Figura 7.4: WER de cada sesión con el modelo de la fase 1 entrenado con todas las sesiones (*frontend* profundo $\times 4$). La línea discontinua es el WER de toda la validación (0,206) y las cruces marcan las sesiones sin frases de validación (1, 29, 33 y 34).

Una parte del error, por tanto, depende de cómo es cada día, aunque el modelo lo conozca. Hay que tenerlo en cuenta al leer la fase 2: una sesión futura puede salir mejor o peor en parte por el propio día, y no solo por no haberla visto.

7.2.2. Entrenamiento cronológico

La tabla 7.5 y la figura 7.5 resumen los ocho modelos entrenados con las k primeras sesiones. Hay tres resultados claros.

Tabla 7.5: Entrenamiento cronológico. Para cada modelo, entrenado con las k primeras sesiones: número de frases de validación y WER en las sesiones vistas y en las futuras, WER en esas mismas frases futuras del modelo de la fase 1 que vio todas las sesiones, y WER en toda la validación.

	Sesiones vistas		Sesiones futuras		Vio todas	Toda la
k	Frases	WER	Frases	WER	(futuras)	validación
5	157	0,449	1.269	0,955	0,212	0,904
10	348	0,266	1.078	0,783	0,218	0,667
15	533	0,254	893	0,803	0,201	0,606
20	665	0,261	761	0,813	0,197	0,564
25	838	0,226	588	0,811	0,220	0,478
30	965	0,222	461	0,777	0,227	0,408
35	1.086	0,203	340	0,790	0,245	0,347
40	1.276	0,214	150	0,657	0,255	0,261

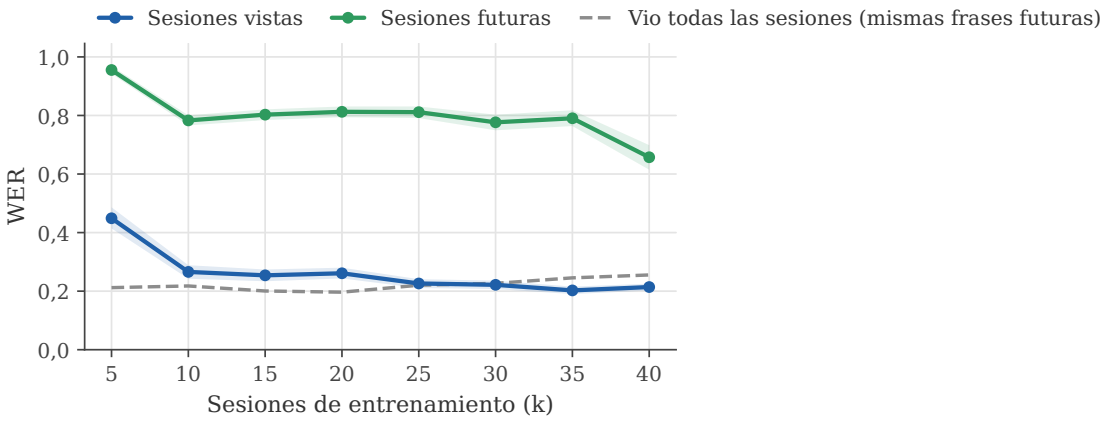


Figura 7.5: WER de los modelos entrenados con las k primeras sesiones, en las sesiones vistas y en las futuras. Las bandas son el intervalo de confianza del 95 %. La línea gris discontinua es el modelo de la fase 1 que vio todas las sesiones, evaluado sobre las mismas frases futuras.

El primero es que en los días que ha visto el modelo funciona bien. Desde $k = 10$, el WER en las sesiones vistas está entre 0,20 y 0,27, y con 35 o 40 sesiones baja a 0,20-0,21, lo mismo que el modelo de la fase 1 entrenado con todas (0,206). Con cinco sesiones se queda en 0,449, porque son muy pocos datos.

El segundo es que en los días nuevos falla. Con k entre 10 y 35, el WER en las sesiones futuras se queda entre 0,78 y 0,81, y añadir sesiones antiguas casi no lo cambia: con 35 sesiones, más del triple que con 10, el error es prácticamente el mismo, y los intervalos de confianza de esos seis modelos se solapan. Las frases de esos días, además, no son más difíciles: el modelo de la fase 1, que sí vio esas sesiones, tiene en las mismas frases un WER de entre 0,20 y 0,26. El hueco, de más de 0,5, se debe por tanto a no haber visto el día.

El tercero es la bajada con $k = 40$, hasta 0,657. Para este modelo, las sesiones futuras son justo las cinco últimas (de la 41 a la 45, de enero a abril de 2025). Entre $k = 35$ y $k = 40$ entran en el entrenamiento las sesiones 36 a 40, y la 40 ya es de enero de 2025: llega tras casi seis meses sin grabaciones y está a solo dos días de la 41. Lo más probable es que las sesiones 40 a 45 se parezcan entre sí por estar cerca en el tiempo. Con estos datos no se puede decir que 40 sesiones sean un umbral.

Para comparar todos los modelos sobre las mismas frases, la figura 7.6 muestra el WER en esas cinco últimas sesiones, que son futuras para todos. Con k de 5 a 35 está entre 0,91 y 1,03, y solo baja, a 0,66, cuando el modelo entrena con sesiones cercanas. El modelo que vio todas las sesiones, incluidas esas cinco, tiene en las mismas frases un WER de 0,255. Todo apunta a que el error en un día nuevo depende más de lo cerca que estén los días de entrenamiento que de cuántos haya.

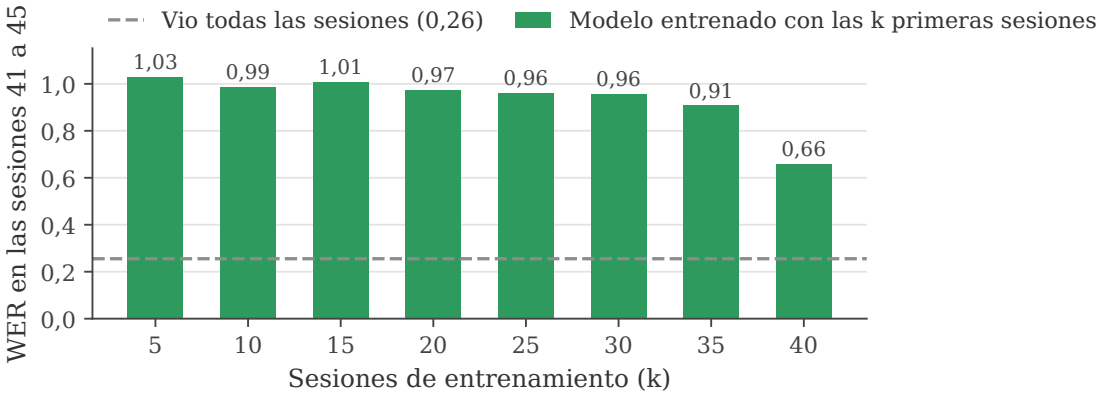


Figura 7.6: WER en las cinco últimas sesiones (de la 41 a la 45), que son futuras para todos los modelos, según el número de sesiones de entrenamiento k. La línea discontinua es el modelo de la fase 1 que vio todas las sesiones.

Las curvas de aprendizaje (figura 7.7) muestran que todos los modelos llegaron a su meseta, así que el mal resultado en días nuevos no se debe a haber entrenado poco. Los intervalos de confianza tampoco cambian la lectura: son de entre $\pm 0{,}01$ y $\pm 0{,}03$ en casi todos los grupos, y de $\pm 0{,}04$ en los dos más pequeños, las 157 frases vistas de $k = 5$ y las 150 futuras de $k = 40$, muy lejos de la diferencia entre días vistos y futuros.

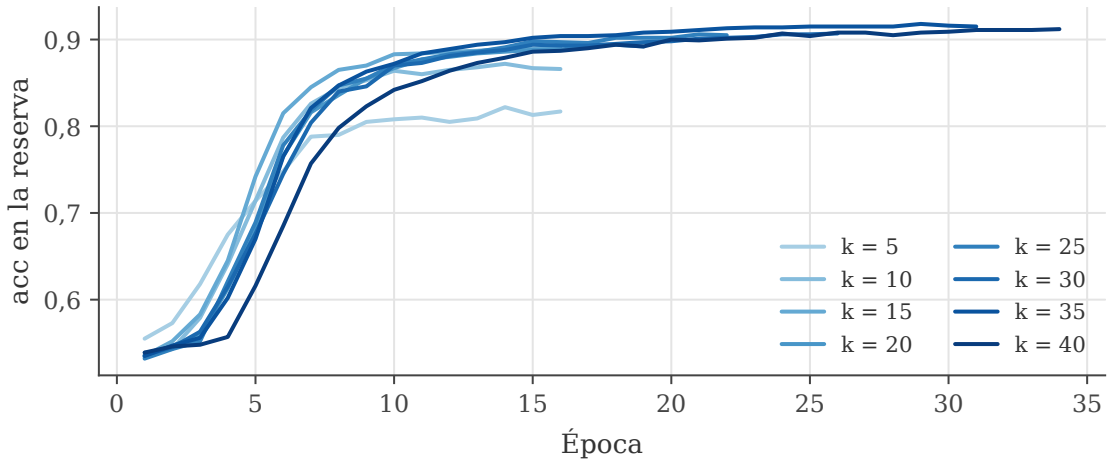


Figura 7.7: Curvas de aprendizaje de los modelos entrenados con las k primeras sesiones: acc en la reserva por época.

Este resultado encaja con lo que se sabe de las señales intracorticales: lo que registra cada electrodo cambia de un día para otro (sección 4.2), y por eso los sistemas clínicos recalibran a menudo o usan una capa de entrada por día [37], [44]. En este trabajo, el modelo aprende bien los días que ve, pero no generaliza a días nuevos, y entrenar con más sesiones antiguas no lo arregla. Para usarlo en un día nuevo haría falta recalibrarlo con algunas frases de ese día o adaptarlo mientras se usa, algo que queda como trabajo futuro (capítulo 8).

7.2.3. Análisis del modelo

Este apartado mira dentro de los errores, tanto de los modelos cronológicos como del modelo de la fase 1 que vio todas las sesiones, el profundo $\times 4$.

Cómo salen las frases. El modelo que vio todas las sesiones transcribe perfecta algo más de una de cada tres frases (el 35 %), y otro 33 % tiene como mucho un error de cada cuatro palabras. Un 23 % tiene hasta la mitad mal y solo un 9 %, más de la mitad. Con los modelos cronológicos (figura 7.8), el reparto en las sesiones vistas mejora a medida que crece k, y en las futuras casi ninguna frase sale perfecta: menos del 6 % tiene bien siquiera todas sus palabras de contenido.

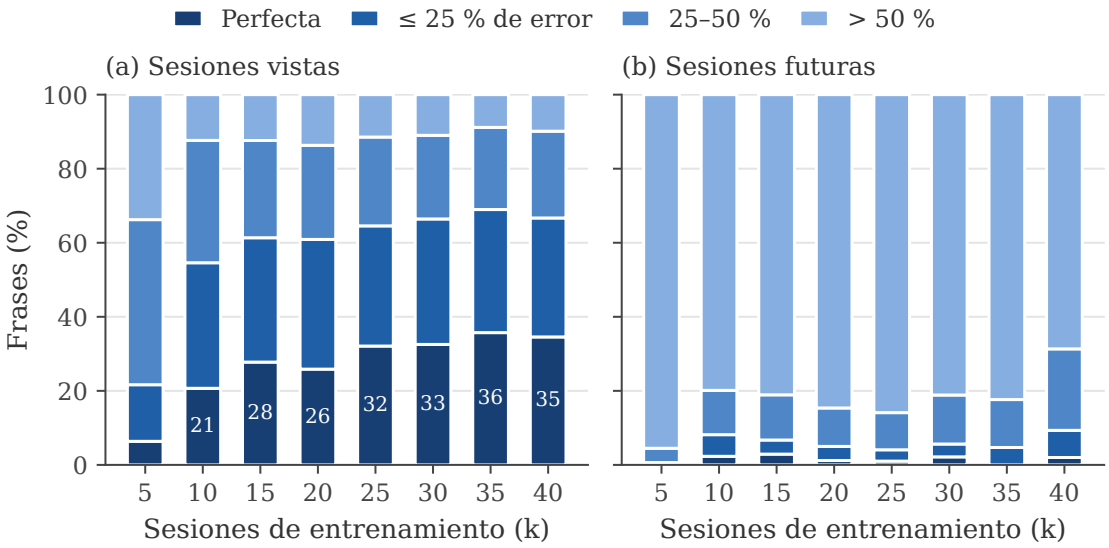


Figura 7.8: Reparto de las frases según su error en las sesiones vistas (a) y en las futuras (b), para cada número de sesiones de entrenamiento k. Las cifras dentro de las barras son el porcentaje de frases perfectas.

Qué palabras acierta. El modelo entrenado con todas las sesiones acierta casi siempre las palabras funcionales (figura 7.9a): el 96 % de los determinantes, el 93 % de los pronombres y el 92 % de las preposiciones y de los auxiliares. Con las de contenido le cuesta más: acierta el 68 % de los adjetivos y de los verbos, el 59 % de los sustantivos y solo el 28 % de los nombres propios. En total, acierta el 93 % de las palabras funcionales y el 66 % de las de contenido. De las 322 palabras que aparecen al menos cinco veces en la validación, solo hay dos que no acierta nunca: *york* e *its*, que confunde con *it's*. Es un error típico del sistema: cuando falla, muchas veces escribe una palabra que suena igual o casi igual. Entre las confusiones más repetidas están *to* en lugar de *two*, *too* o *do*, *close* en lugar de *clothes*, *think* en lugar de *thank* y *of* en lugar de *have*.

Por eso el WER dice menos de lo que parece. Buena parte de los aciertos son palabras funcionales, frecuentes y fáciles de adivinar, y en las palabras de contenido, que son las que llevan el significado, el error es bastante mayor.

Palabras vistas y no vistas. El acierto depende sobre todo de si la palabra estaba en el entrenamiento del modelo. En las sesiones vistas, los modelos cronológicos aciertan el 87 % de las palabras que habían visto al entrenar y apenas el 1 % de las que no; en las futuras, el 22 % y prácticamente ninguna (el 0,1 %). Y no son pocas: el 6,3 % de las palabras de la

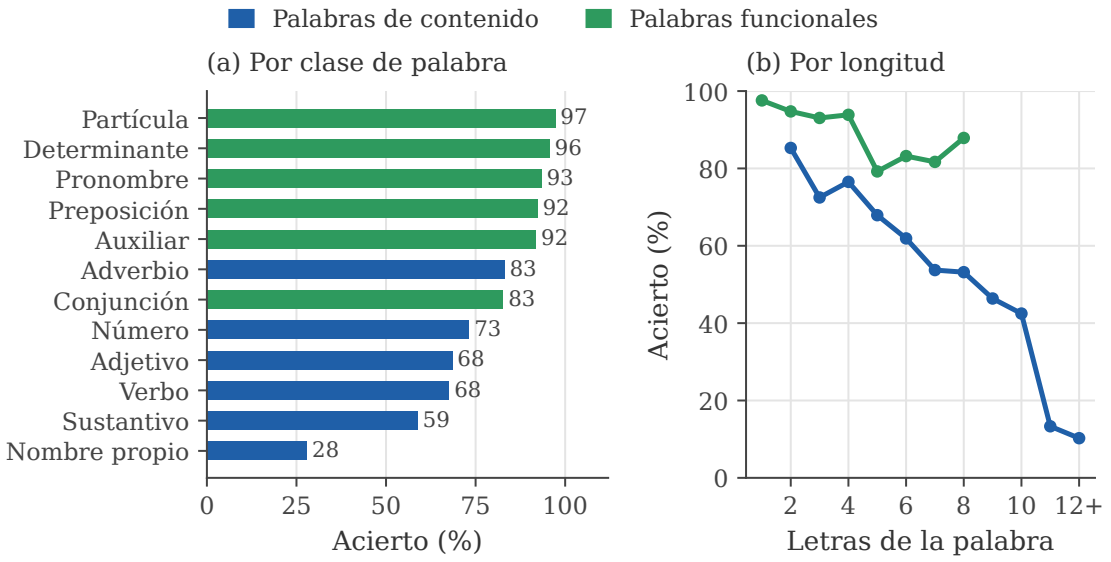


Figura 7.9: Acierto del modelo entrenado con todas las sesiones por clase de palabra (a) y según el número de letras de la palabra (b). En azul, las palabras de contenido; en verde, las funcionales.

validación no sale nunca en el entrenamiento, y más de una de cada cuatro frases tiene alguna (sección 4.3). El decodificador de OWSM conoce esas palabras y el vocabulario de subpalabras permitiría escribirlas, pero en la práctica el sistema se comporta casi como si tuviera un vocabulario cerrado: solo escribe bien las palabras que ha visto junto a su señal.

Cómo evolucionan con k. La figura 7.10 separa el acierto en palabras de contenido, en funcionales y en la palabra clave de cada frase, que es la de contenido menos frecuente. En las sesiones vistas, las funcionales se aciertan entre el 92 y el 95 % desde $k = 10$, mientras que las de contenido suben del 29 % con cinco sesiones al 53 % con diez y a cerca del 65 % con 35 o 40, y la palabra clave, del 12 % al 51 %. El modelo que vio todas las sesiones llega al 66 % en las de contenido y al 52 % en la palabra clave, así que con 35 o 40 sesiones ya se está a su nivel. En las sesiones futuras todo cae, incluso las funcionales, que se quedan entre el 27 y el 33 % de $k = 10$ a $k = 35$. Las de contenido no pasan del 14 % en esos modelos y solo suben al 21 % con $k = 40$, y la palabra clave se queda en el 10 % o menos. En un día nuevo, por tanto, el modelo casi nunca acierta la palabra que más información lleva.

Parecido en la pronunciación. Cuando el modelo se equivoca en un día que ha visto, la palabra que escribe suele sonar parecido a la correcta (figura 7.11). Desde $k = 10$, el parecido medio en la pronunciación de las sustituciones está entre el 33 y el 37 %, en una escala en la que el 100 % es la misma pronunciación, frente a un 8 % entre palabras tomadas al azar, y entre el 5 y el 8 % de las sustituciones son casi homófonas (con un parecido del 75 % o más). El modelo que vio todas las sesiones da lo mismo: un 34 %, con un 6 % de casi homófonas. Es lo que cabe esperar si el codificador lee la articulación y falla en los detalles. En las sesiones futuras, el parecido baja a entre el 8 y el 18 %, cerca del azar (un 6 %), y las casi homófonas se quedan por debajo del 1 %: en un día nuevo, los errores se parecen a la palabra correcta poco más que dos palabras cualquiera, así que el modelo escribe sin apoyarse en la señal. El valor más alto, el 18 %, es el del modelo con 40 sesiones, cuyas sesiones futuras son las más cercanas a las de entrenamiento. Hay que tener en cuenta que el parecido en la escritura sale casi igual (entre el 34 y el 37 % en los

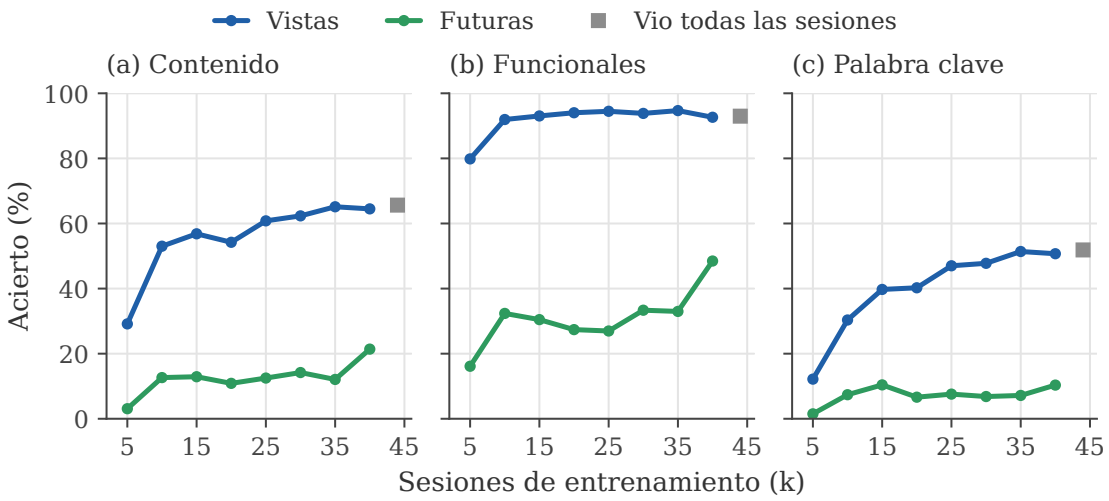


Figura 7.10: Acierto en palabras de contenido (a), en palabras funcionales (b) y en la palabra clave de cada frase (c), en las sesiones vistas y en las futuras, según el número de sesiones de entrenamiento k. El cuadrado gris de la derecha es el modelo de la fase 1 que vio todas las sesiones.

días vistos), así que no se puede separar del todo si las palabras se confunden por cómo suenan o por cómo se escriben.

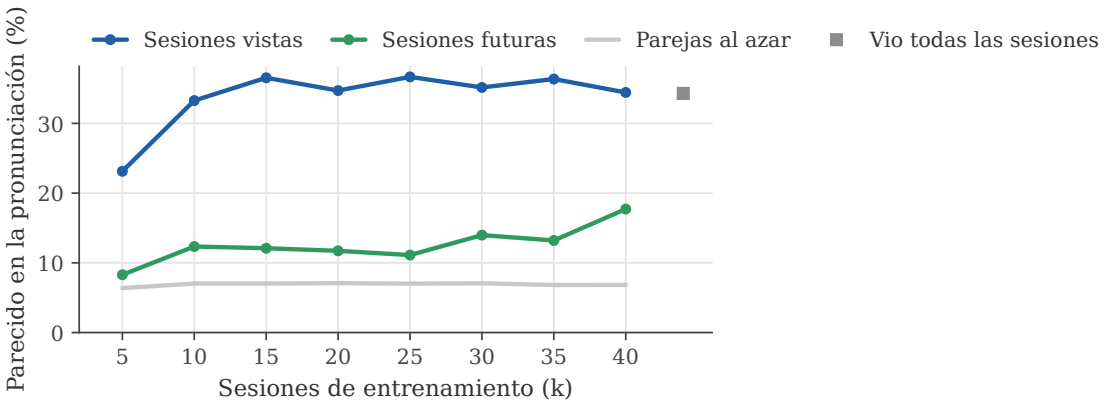


Figura 7.11: Parecido medio en la pronunciación entre la palabra correcta y la que escribe el modelo en las sustituciones (100 % = misma pronunciación), en las sesiones vistas y en las futuras, según el número de sesiones de entrenamiento k. La línea gris clara es el parecido entre parejas de palabras tomadas al azar, y el cuadrado, el modelo de la fase 1 que vio todas las sesiones.

Qué aporta el decodificador. La búsqueda en haz mejora al CTC voraz sobre todo en los días vistos: el modelo que vio todas las sesiones pasa de 0,287 con el CTC voraz a 0,206 con el haz, y en los modelos cronológicos, desde $k = 10$, la mejora en las sesiones vistas es de entre 0,06 y 0,09. Además, el haz casi no inventa palabras: como mucho el 1,5 % de las que escribe no existen, frente a hasta un 8 % con el CTC voraz, que escribe subpalabra a subpalabra. En los días nuevos la ventaja se reduce a entre 0,02 y 0,05, y en las cinco últimas sesiones, con cinco sesiones de entrenamiento, el haz llega a ser algo peor que el CTC voraz (1,028 frente a 0,991): cuando la señal no ayuda, el decodificador escribe frases enteras que no tienen que ver con la referencia.

Qué escribe cuando falla. En los días nuevos el modelo no se queda atascado repitiendo una frase: desde $k = 10$, más del 98 % de sus hipótesis son distintas entre sí. Solo con cinco

sesiones se repiten algunas, un 10 %, casi siempre letras o palabras sueltas como *u*, *s*, *no* o *good*, y aparecen unas pocas vacías (el 0,3 %). Lo que sí cambia es la longitud. En las sesiones vistas escribe frases casi tan largas como la referencia (entre el 98 y el 100 % de su longitud desde $k = 10$), y en las futuras, más cortas, de entre el 83 y el 94 %.

7.3 Comparación con el estado del arte

El mejor sistema de este trabajo tiene un WER de 0,206 en la validación, es decir, un 20,6 %. La referencia justa son los sistemas de extremo a extremo sin modelo de lenguaje externo de la tabla 3.2. En el Brain-to-Text '25, la red recurrente del sistema de referencia llega sin modelo de lenguaje al 16,0 %, BIT al 11,1 % y BrainWhisperer al 10,2 %. Ellos dan la partición de prueba de la competición y aquí se usa la validación, pero en este conjunto las dos dan cifras parecidas [39]. El sistema queda, por tanto, unos cinco puntos por encima de la red recurrente y con el doble de error que los mejores.

Con Khanday et al. sí se puede comparar sobre la misma partición: su Conformer entrenado desde cero, con caracteres y sin modelo de lenguaje, tiene un CER del 23,8 % en validación [46]. El sistema de este trabajo tiene un CER de 0,157 con el profundo $\times 4$, aunque con una diferencia importante: usa el decodificador preentrenado de OWSM, que aporta un conocimiento del idioma que un sistema solo con caracteres no tiene.

Los mejores sistemas tienen ventajas que este no tiene (sección 3.6): aumentos de datos más fuertes, como el enmascarado temporal, supervisión adicional en capas intermedias del codificador, entrenamientos mucho más largos y decodificadores más grandes o ajustados. BrainWhisperer, por ejemplo, ajusta el decodificador de Whisper con LoRA y, en su mejor versión, añade datos de otros participantes [45]. El objetivo de este trabajo, con una sola GPU en Colab y cada configuración entrenada una vez, era ver si reutilizar OWSM es viable. En los días vistos lo es, a una distancia razonable de sistemas mucho más costosos.

7.4 Discusión

7.4.1. Resultados por objetivo

OE1, *frontend*. El profundo $\times 4$ es el mejor compromiso: tiene el WER más bajo (0,206) y entrena un 40 % más rápido que los $\times 2$. Reducir la secuencia a una ventana cada 80 ms no perjudica, y el *patching* del sistema de referencia es el que peor encaja en este modelo. La capa de entrada por sesión no aporta nada apreciable.

OE2, codificador. Los pesos de OWSM no se transfieren a la señal neuronal. Con ellos el modelo no aprende, ni congelados, ni con LoRA, ni ajustados enteros, y la señal falsa confirma que no lee la señal. El codificador entrenado desde cero, en cambio, aprende sin problemas.

OE3, decodificador. El decodificador preentrenado sí se transfiere. Congelado da prácticamente lo mismo que con LoRA, y entrenado desde cero memoriza las frases de entrenamiento y generaliza mucho peor.

OE4, ventana temporal. El sistema funciona bien en los días que ha visto, al nivel del modelo entrenado con todas las sesiones, pero falla en los días nuevos, con un WER de 0,78 a 0,81, y más sesiones antiguas no lo arreglan. En un día nuevo pierde sobre todo las palabras de contenido, y sus errores dejan de parecerse a la palabra correcta.

7.4.2. Una lectura conjunta

Los resultados muestran una transferencia a medias. La parte de OWSM que trabaja con el idioma, el decodificador, sirve tal cual y aporta lo que un decodificador no puede aprender con 8.000 frases. La parte que trabaja con la señal, el codificador, hay que aprenderla desde cero, porque lo que sabe del audio no encaja con la actividad de los electrodos. Lo mismo se ve en el resto de resultados: las diferencias entre configuraciones están en las piezas que aprenden a leer la señal, el *frontend* y el codificador. El decodificador hace que el texto salga bien escrito como frase, pero solo acierta las palabras que el codificador ha aprendido a reconocer en la señal, como muestra el análisis de las palabras vistas y no vistas.

Para el uso real, el problema que más pesa es el paso de los días. Ninguna decisión de la fase 1 cambia tanto el error como pasar de un día visto a uno nuevo, de 0,2 a 0,8. Resolverlo exige datos del día nuevo, ya sea para recalibrar el sistema o para adaptarlo mientras se usa.

Todos estos resultados tienen las limitaciones de un estudio con un solo participante, una sola semilla por configuración y la validación haciendo de conjunto de prueba, que se tratan en el capítulo 8.

CAPÍTULO 8
Limitaciones y trabajo futuro

Este capítulo recoge primero las limitaciones del trabajo, que hay que tener en cuenta al leer sus conclusiones, y después las líneas que quedan abiertas para continuarlo.

8.1 Limitaciones

8.1.1. Un solo participante y un solo modelo

Todas las conclusiones salen de una sola persona, el participante T15, con sus cuatro matrices de electrodos en la corteza motora del habla, el inglés como idioma y una tarea en la que copia frases con habla intentada (sección 4.2). No se sabe si los resultados se mantendrían con otra persona, con los implantes en otra zona, en otro idioma o con habla espontánea.

Con el modelo pasa lo mismo. Solo se ha probado OWSM v3.1 base, y que sus pesos no se transfieran, ni congelados, ni con LoRA, ni ajustados enteros, no quiere decir que no lo hagan los de otros modelos de voz, y de hecho Fiedler et al. encontraron lo contrario con wav2vec 2.0 [51].

8.1.2. Fiabilidad de las cifras

Por coste, cada configuración se entrenó una sola vez y con una sola semilla. Los intervalos de confianza calculados sobre las frases dicen cuánto podría cambiar el WER con otras frases, pero no cuánto cambiaría al repetir el entrenamiento, así que diferencias de una o dos centésimas, como la de LoRA o la de la capa por sesión, no se pueden dar por buenas.

Además, la validación oficial hace de conjunto de prueba, porque el test de la competición no tiene transcripciones públicas. En la fase 1, la mejor época se eligió con esa misma validación, lo que hace las cifras algo optimistas, aunque por igual para todos los modelos. La fase 2 lo evita con la reserva.

8.1.3. Recursos

Todo el trabajo se hizo con una sola GPU NVIDIA L4 en Google Colab. Cada entrenamiento llevaba entre 3 y 15 horas, y las desconexiones obligaron a guardar y reanudar a menudo. Con este presupuesto no se pudieron repetir entrenamientos, buscar hiperparámetros, ajustar la búsqueda en haz ni usar lo que hacen los primeros puestos del

Brain-to-Text '25, como combinar muchos modelos o reordenar las hipótesis con un modelo de lenguaje grande.

8.1.4. Distancia al uso real

El sistema se evalúa en diferido: el codificador ve la frase entera antes de escribir nada, así que tal como está no podría funcionar en tiempo real, y no se ha medido su latencia. Tampoco se ha probado ninguna forma de recalibrarlo en días nuevos, que es justo donde falla (sección 7.2.2). Y, como muestra el análisis de palabras, en la práctica solo escribe bien las palabras que ha visto junto a su señal, lo que limita el vocabulario real del sistema aunque el del decodificador sea abierto.

8.2 Trabajo futuro

Las líneas que siguen van de las más directas, que se podrían probar con los mismos datos y recursos, a las que necesitan más datos o un planteamiento nuevo.

8.2.1. Mejorar el modelo

Hay mejoras directas que este trabajo no llegó a probar. La primera son los aumentos de datos que usan los mejores sistemas: además del ruido, desplazar cada canal, suavizar la señal y, sobre todo, el enmascarado temporal, que tapa trozos de cada frase durante el entrenamiento. Feghhi et al. enmascaran de media más de la mitad de cada frase, y es una de las claves de su mejora [78]. La segunda es entrenar más tiempo: el profundo $\times 2$ pasó de 0,224 a 0,209 al doblar las épocas, y el $\times 4$ solo se entrenó 40. La tercera, que no exige reentrenar, es ajustar la búsqueda en haz: el tamaño del haz, el peso de CTC y la penalización por longitud.

También queda por probar una capa de entrada por día con muchos menos parámetros, como la de BrainWhisperer, que combina una matriz por mes con una corrección de rango bajo por día [45]. Aquí la capa por sesión no aportó, pero una versión más ligera, junto con alguna forma de adaptación a días nuevos (sección 8.2.5), podría hacerlo.

8.2.2. Usar fonemas

Casi todos los sistemas del campo usan fonemas en algún punto, y este trabajo los dejó fuera. El conjunto de datos ya trae la secuencia de fonemas de cada frase (sección 4.2), así que hay dos formas sencillas de aprovecharlos. La primera es añadir una pérdida CTC de fonemas en una capa intermedia del codificador, como hace BrainWhisperer [45]. Así el codificador recibe una señal más directa y más cercana a la articulación, que es lo que se registra en la corteza motora del habla, sin que cambie la salida del modelo. Roth va más allá y entrena primero un decodificador de fonemas antes del entrenamiento de extremo a extremo [79]. La segunda es usar los fonemas como salida, con un diccionario de pronunciación y un modelo de lenguaje para pasar a palabras, como el sistema de referencia del Brain-to-Text '25 [30]. Eso permitiría comparar sobre el mismo codificador el enfoque de extremo a extremo de este trabajo con el habitual en el campo.

8.2.3. Mejorar el decodificador

El decodificador es la parte de OWSM que sí se transfiere, así que mejorarlo es una de las vías más prometedoras. Aquí se usó el de OWSM v3.1 base, la versión más pequeña, con 101 millones de parámetros en total. Hay versiones de 367 y 1.020 millones, y también OWSM v4, del mismo tamaño que la base pero entrenado con unas 320.000 horas en lugar de 180.000 [80]. Dos de los sistemas que llegan a un 10 % de extremo a extremo usan el decodificador de Whisper: Roth lo combina con un codificador recurrente y llega al 10,33 % [79], y BrainWhisperer lo ajusta con LoRA [45]. Con un decodificador más grande, LoRA podría aportar más que aquí, donde la diferencia con el congelado cabía en el margen de error.

Otra posibilidad es adaptar el decodificador al tipo de frases del conjunto, que son sobre todo conversacionales, por ejemplo ajustándolo con texto de ese estilo. Y, si se quiere ir más allá del extremo a extremo, se puede añadir un modelo de lenguaje externo: unos n-gramas dentro de la búsqueda en haz o un modelo de lenguaje grande que reordene las mejores hipótesis, como hacen los primeros puestos del Brain-to-Text '25. Antes de eso, merece la pena medir cuántas veces está la frase correcta entre las hipótesis del haz, porque eso marca lo máximo que puede ganar cualquier reordenación.

8.2.4. Más datos

Con 8.000 frases de una persona, el modelo aprende bien los días que ve pero no generaliza. Más datos podrían ayudar de tres formas. La más directa es añadir otros participantes: los datos del Brain-to-Text '24, del participante T12, también son públicos [23], [29], y BrainWhisperer pasa del 10,2 al 8,7 % al entrenar con datos de varias personas [45]. La segunda es preentrenar el codificador con actividad neuronal en lugar de con audio. BIT usa datos de varios sujetos e incluso de otras especies [43], y Roth preentrena con datos de ratones y de personas, con resultados que dependen de la arquitectura [79]. Si el audio no le sirve al codificador, quizá le sirva la señal neuronal de otras personas. La tercera, a más largo plazo, es tener más sesiones de la misma persona, que permitirían estudiar la fase 2 con más días futuros.

8.2.5. Adaptarse a los días nuevos

Es la línea con más valor práctico, porque es donde el sistema falla: de un WER de 0,2 en los días vistos a uno de 0,8 en los nuevos. Lo más sencillo es recalibrar con unas pocas frases del día nuevo. Este trabajo ya tiene casi todo lo necesario para medir cuántas hacen falta: bastaría con ajustar cada modelo cronológico con 10, 20 o 50 frases de la sesión siguiente y ver cuánto baja el error. También se podría entrenar de forma incremental, partiendo del modelo anterior en lugar de empezar desde cero, que es más barato y se parece más al uso real.

Lo más cómodo para el usuario, en cambio, es adaptarse sin pedirle frases nuevas. Fan et al. recalibran el decodificador de forma continua con pseudoetiquetas, es decir, corrigen la salida con un modelo de lenguaje y entrenan con esas correcciones, y así mantienen la precisión durante más de un año [37]. Feghhi et al. proponen una versión ligera que solo actualiza la entrada del modelo, con un paso de gradiente por frase [78]. Con el decodificador de OWSM, las hipótesis de la búsqueda en haz podrían hacer de pseudoetiquetas, aunque habría que comprobar si son lo bastante buenas cuando el WER de partida ronda 0,8.

8.2.6. Entender qué se transfiere

La pregunta que deja abierta el trabajo es por qué el codificador de OWSM no sirve cuando el de wav2vec 2.0 sí parece servir [51]. Para responderla haría falta comparar varios modelos de voz con el mismo esquema y los mismos datos, cambiando de forma controlada cómo se preentrenaron: sin transcripciones y sobre la forma de onda, como wav2vec 2.0, HuBERT o WavLM, o con transcripciones y sobre el espectrograma, como Whisper u OWSM [36], [48], [81], [82]. También sería útil mirar dentro del codificador aleatorio una vez entrenado, capa a capa, para ver qué aprende de la señal y si se parece en algo a lo que aprende un codificador de voz.

8.2.7. Hacia el uso real

Para usarlo en la vida real, el sistema tendría que funcionar en tiempo real, escribiendo mientras la persona intenta hablar. Eso exige un codificador que no vea el futuro, por ejemplo con atención por bloques, y medir la latencia y el tiempo de cálculo por segundo de señal. El *frontend* $\times 4$ ya va en esa dirección, porque reduce a la cuarta parte lo que procesa el codificador. Por último, para que las conclusiones sean más sólidas, habría que repetir los entrenamientos clave con varias semillas y, si la plataforma de la competición lo permite, evaluar en el test oficial.

CAPÍTULO 9
Conclusiones

Este trabajo se planteó como un estudio de viabilidad: comprobar si se puede reutilizar OWSM, un modelo de reconocimiento de voz abierto y entrenado con 180.000 horas de audio, para pasar a texto la actividad neuronal de una persona que intenta hablar, qué partes del modelo se aprovechan y cómo se comporta el sistema con el paso del tiempo. La respuesta corta es que sí se puede, pero solo en parte. El decodificador de OWSM se aprovecha tal cual y el codificador hay que entrenarlo desde cero. Con una sola GPU, el sistema llega a un WER de 0,206 en los días que conoce, pero en los días nuevos se queda en torno a 0,8.

9.1 Respuesta a los objetivos

OE1, *frontend*. El *frontend* decide cuánto cuesta el modelo y cambia el error en hasta 0,12. El profundo $\times 4$ es el mejor compromiso: da el WER más bajo, 0,206, y entrena un 40 % más rápido que los $\times 2$, porque el codificador procesa la mitad de ventanas. Pasar a un vector cada 80 ms no perjudica, y el *patching* del sistema de referencia del Brain-to-Text '25 es el que peor funciona en este modelo. La capa de entrada por sesión no cambia el resultado de forma apreciable.

OE2, codificador. Los pesos de OWSM no se transfieren a la señal neuronal. Con ellos el sistema no aprende, ni congelados, ni con LoRA, ni ajustados enteros (WER de 0,97 en los tres, frente a 0,209 desde cero), y la señal falsa muestra que no está leyendo la señal. El mismo codificador entrenado desde cero aprende sin problemas.

OE3, decodificador. El decodificador preentrenado sí se transfiere y es necesario. Congelado da prácticamente lo mismo que ajustado con LoRA (0,209 frente a 0,202, dentro del margen de error), y entrenado desde cero aprende de memoria las frases de entrenamiento y generaliza mucho peor (su acc de validación no pasa de 0,770, frente a 0,873).

OE4, ventana temporal. Incluso en los días que conoce, el error cambia mucho de una sesión a otra. En el entrenamiento cronológico, el modelo funciona bien en las sesiones vistas (WER de 0,20 a 0,27 desde diez sesiones) y falla en las futuras (de 0,78 a 0,81), y entrenar con más sesiones antiguas no lo arregla. En un día nuevo pierde sobre todo las palabras de contenido, y sus errores dejan de parecerse a la palabra correcta.

Respecto al objetivo general, reutilizar OWSM es viable para los días que el sistema ha visto. Su WER de 0,206 queda por encima del 10-16 % de los sistemas de extremo a

extremo del estado del arte, que tienen muchos más recursos, pero en un rango razonable para una sola GPU y sin modelo de lenguaje externo. Para días nuevos no es viable todavía sin recalibrar.

9.2 Aportaciones

1. **Una adaptación completa de OWSM a la señal intracortical**, con herramientas abiertas, datos públicos y una sola GPU, que incluye los problemas prácticos que hubo que resolver por el camino.
2. **Una comparación controlada de qué parte del modelo se transfiere.** El decodificador sí y el codificador no, comprobado con la señal falsa.
3. **Una comparación de seis *frontends* que mide a la vez el error y el coste.** En la literatura revisada cada sistema usa uno distinto sin compararlo con los demás.
4. **Una evaluación cronológica** que separa las frases de días vistos y futuros, mide el hueco entre ambos y muestra que más sesiones antiguas no lo cierran.
5. **Un análisis de los errores palabra a palabra**, que muestra qué se pierde en los días nuevos más allá del WER.

9.3 Lecciones aprendidas

Además de los resultados, el trabajo deja algunas lecciones sobre cómo hacerlo.

La primera es comprobar lo que hacen las herramientas por defecto. Varios problemas no dieron ningún error y solo aparecieron al revisar: la parada temprana de ESPnet vigilaba la pérdida y no la métrica de la tarea, un parámetro de la búsqueda en haz desactivaba la detección del final de la frase y hacía la evaluación siete veces más lenta, y ESPnet-EZ no aplicaba el preprocesado de texto de OWSM.

La segunda es hacer pruebas sencillas antes de las grandes. La prueba de memorización separó los errores del código de los problemas de generalización al principio, y la señal falsa explicó en un momento por qué fallaba el codificador preentrenado.

La tercera es medir con la métrica adecuada y saber cuánta precisión tiene. La acc de ESPnet sirve para elegir la época, pero para comparar modelos hay que generar texto, y con 1.426 frases una diferencia de menos de una centésima de WER no dice nada.

La cuarta es evaluar como se usaría el sistema. Con frases de los mismos días que el entrenamiento, el modelo parecía estar cerca de funcionar; solo la evaluación cronológica mostró que el problema real son los días nuevos.

9.4 Relación con los estudios cursados

Este trabajo pone en práctica buena parte de lo aprendido en el Grado en Ciencia de Datos. Las redes neuronales, que son el núcleo del sistema, se estudian en Modelos descriptivos y predictivos II (MDP II), y las redes convolucionales con las que se construyen los *frontends*, en Técnicas escalables en aprendizaje automático (TEAA). Lenguaje natural y recuperación de la información (LNR) está detrás de la parte de texto: los modelos de lenguaje, la atención en la que se basan el codificador y el decodificador, y la tokenización en subpalabras. Evaluación, despliegue y monitorización de modelos está en la forma de

evaluar: elegir la métrica adecuada, separar bien los datos y no dar por buena una cifra sin saber cuánto puede variar. Las asignaturas de estadística, álgebra y programación, como Modelos estadísticos para la toma de decisiones I y II, Álgebra lineal o Programación, son la base de todo lo demás, desde los intervalos de confianza por *bootstrap* hasta el código de los experimentos. Por último, Proyecto I, II y III enseñan a trabajar en un proyecto de datos de principio a fin, que es justo lo que exige un trabajo de este tamaño.

Además, ha exigido competencias que van más allá de una asignatura concreta: leer literatura reciente, decidir qué experimentos hacer con recursos limitados y contar los resultados con honestidad, incluidos los que no salieron como se esperaba.

9.5 Legado

El código del trabajo, con los notebooks de entrenamiento, evaluación y análisis, y esta memoria están en un repositorio público de GitHub[1]. Los datos son públicos [30] y el sistema solo usa herramientas abiertas, ESPnet y OWSM, así que los experimentos se pueden reproducir con una GPU como la usada aquí. Cada entrenamiento guarda su receta junto a sus resultados, lo que facilita repetirlo.

9.6 Valoración final

El trabajo no termina con un sistema listo para que alguien lo use, y tampoco era ese el objetivo. Lo que sí deja es una respuesta concreta a la pregunta de partida: de OWSM sirve el decodificador y no el codificador, y con eso se llega, con muy pocos recursos, a un error razonable en los días que el modelo conoce.

Deja también claro cuál es el siguiente problema. Ninguna decisión sobre el modelo cambia tanto el error como pasar de un día conocido a uno nuevo, y resolverlo es lo que separa a estos sistemas de un uso diario. Es, además, un problema que se puede atacar con lo construido aquí, como se propone en el capítulo 8.

[1] https://github.com/Mrcirilo/tfg-sEEG-brain-to-text

Bibliografía

[1] A. B. Silva, K. T. Littlejohn, J. R. Liu, D. A. Moses y E. F. Chang, «The Speech Neuroprosthesis,» *Nature Reviews Neuroscience*, vol. 25, n.º 7, págs. 473-492, 2024. DOI: 10.1038/s41583-024-00819-9

[2] L.-X. Lin et al., «Stroke-Associated Dysarthria,» *Frontiers in Neurology*, vol. 16, 2025, **artno** 1629640. DOI: 10.3389/fneur.2025.1629640

[3] D. Beukelman, S. Fager y A. Nordness, «Communication Support for People with ALS,» *Neurology Research International*, vol. 2011, 2011, **artno** 714693. DOI: 10.1155/2011/714693

[4] L. Xu et al., «Global Variation in Prevalence and Incidence of Amyotrophic Lateral Sclerosis: A Systematic Review and Meta-Analysis,» *Journal of Neurology*, vol. 267, n.º 4, págs. 944-953, 2020. DOI: 10.1007/s00415-019-09652-y

[5] Servimedia. «Los casos de ELA aumentarán un 40 % en Europa en los próximos 25 años.» Datos de la Sociedad Española de Neurología (SEN), Servicio de Información sobre Discapacidad (SID-INICO), Universidad de Salamanca, visitado 7 de nov. de 2026. dirección: https://sid-inico.usal.es/noticias/los-casos-de-ela-aumentaran-un-40-en-europa-en-los-proximos-25-anos/

[6] GBD 2021 Stroke Risk Factor Collaborators, «Global, Regional, and National Burden of Stroke and Its Risk Factors, 1990–2021: A Systematic Analysis for the Global Burden of Disease Study 2021,» *The Lancet Neurology*, vol. 23, n.º 10, págs. 973-1003, 2024. DOI: 10.1016/S1474-4422(24)00369-7

[7] T. Sánchez. «Cerca de 90.000 nuevos casos de ictus en España cada año: "Ha disminuido la mortalidad pero tenemos más personas con secuelas".» Datos de la Sociedad Española de Neurología (SEN), Vozpópuli, visitado 7 de nov. de 2026. dirección: https://www.vozpopuli.com/sanitatem/cerca-de-90000-nuevos-casos-de-ictus-en-espana-cada-ano-ha-disminuido-la-mortalidad-pero-tenemos-mas-personas-con-secuelas.html

[8] M. Summaka et al., «Neuroanatomical Regions Associated with Non-Progressive Dysarthria Post-Stroke: A Systematic Review,» *BMC Neurology*, vol. 22, pág. 353, 2022. DOI: 10.1186/s12883-022-02877-x

[9] E. Smith y M. Delargy, «Locked-In Syndrome,» *BMJ*, vol. 330, n.º 7488, págs. 406-409, 2005. DOI: 10.1136/bmj.330.7488.406

[10] M.-A. Bruno, J. L. Bernheim, D. Ledoux, F. Pellas, A. Demertzi y S. Laureys, «A Survey on Self-Assessed Well-Being in a Cohort of Chronic Locked-In Syndrome Patients: Happy Majority, Miserable Minority,» *BMJ Open*, vol. 1, n.º 1, e000039, 2011. DOI: 10.1136/bmjopen-2010-000039

[11] G. C. Vanderheiden, «A Journey through Early Augmentative Communication and Computer Access,» *Journal of Rehabilitation Research and Development*, vol. 39, n.º 6, suplemento, págs. 39-53, 2002.

[12] R. Spataro, M. Ciriacono, C. Manno y V. La Bella, «The Eye-Tracking Computer Device for Communication in Amyotrophic Lateral Sclerosis,» *Acta Neurologica Scandinavica*, vol. 130, n.º 1, págs. 40-45, 2014. DOI: 10.1111/ane.12214

[13] M. J. Vansteensel et al., «Fully Implanted Brain–Computer Interface in a Locked-In Patient with ALS,» *New England Journal of Medicine*, vol. 375, n.º 21, págs. 2060-2066, 2016. DOI: 10.1056/NEJMoa1608085

[14] T. Schultz, M. Wand, T. Hueber, D. J. Krusienski, C. Herff y J. S. Brumberg, «Biosignal-Based Spoken Communication: A Survey,» *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 25, n.º 12, págs. 2257-2271, 2017. DOI: 10.1109/TASLP.2017.2752365

[15] L. A. Farwell y E. Donchin, «Talking off the Top of Your Head: Toward a Mental Prosthesis Utilizing Event-Related Brain Potentials,» *Electroencephalography and Clinical Neurophysiology*, vol. 70, n.º 6, págs. 510-523, 1988. DOI: 10.1016/0013-4694(88)90149-6

[16] N. Birbaumer et al., «A Spelling Device for the Paralysed,» *Nature*, vol. 398, n.º 6725, págs. 297-298, 1999. DOI: 10.1038/18581

[17] D. A. Moses et al., «Neuroprosthesis for Decoding Speech in a Paralyzed Person with Anarthria,» *New England Journal of Medicine*, vol. 385, n.º 3, págs. 217-227, 2021. DOI: 10.1056/NEJMoa2027540

[18] S. L. Metzger et al., «A High-Performance Neuroprosthesis for Speech Decoding and Avatar Control,» *Nature*, vol. 620, n.º 7976, págs. 1037-1046, 2023. DOI: 10.1038/s41586-023-06443-4

[19] M. Angrick et al., «Real-Time Synthesis of Imagined Speech Processes from Minimally Invasive Recordings of Neural Activity,» *Communications Biology*, vol. 4, pág. 1055, 2021. DOI: 10.1038/s42003-021-02578-0

[20] L. R. Hochberg et al., «Neuronal Ensemble Control of Prosthetic Devices by a Human with Tetraplegia,» *Nature*, vol. 442, n.º 7099, págs. 164-171, 2006. DOI: 10.1038/nature04970

[21] C. Pandarinath et al., «High Performance Communication by People with Paralysis Using an Intracortical Brain-Computer Interface,» *eLife*, vol. 6, e18554, 2017. DOI: 10.7554/eLife.18554

[22] F. R. Willett, D. T. Avansino, L. R. Hochberg, J. M. Henderson y K. V. Shenoy, «High-Performance Brain-to-Text Communication via Handwriting,» *Nature*, vol. 593, n.º 7858, págs. 249-254, 2021. DOI: 10.1038/s41586-021-03506-2

[23] F. R. Willett et al., «A High-Performance Speech Neuroprosthesis,» *Nature*, vol. 620, n.º 7976, págs. 1031-1036, 2023. DOI: 10.1038/s41586-023-06377-x

[24] N. S. Card et al., «An Accurate and Rapidly Calibrating Speech Neuroprosthesis,» *New England Journal of Medicine*, vol. 391, n.º 7, págs. 609-618, 2024. DOI: 10.1056/NEJMoa2314132

[25] M. Wairagkar et al., «An Instantaneous Voice-Synthesis Neuroprosthesis,» *Nature*, vol. 644, n.º 8075, págs. 145-152, 2025. DOI: 10.1038/s41586-025-09127-3

[26] Instituto Nacional de Estadística. «Encuesta de Discapacidad, Autonomía Personal y Situaciones de Dependencia (EDAD). Año 2020.» Nota de prensa, visitado 7 de nov. de 2026. dirección: https://www.ine.es/prensa/edad_2020_p.pdf

[27] A. Londral, A. Pinto, S. Pinto, L. Azevedo y M. De Carvalho, «Quality of Life in Amyotrophic Lateral Sclerosis Patients and Caregivers: Impact of Assistive Communication from Early Stages,» *Muscle & Nerve*, vol. 52, n.º 6, págs. 933-941, 2015.

[28] Jefatura del Estado. «Ley 3/2024, de 30 de octubre, para mejorar la calidad de vida de personas con Esclerosis Lateral Amiotrófica y otras enfermedades o procesos de alta complejidad y curso irreversible.» BOE núm. 263, BOE-A-2024-22438, Boletín Oficial del Estado, visitado 7 de nov. de 2026. dirección: https://www.boe.es/eli/es/l/2024/10/30/3

[29] F. R. Willett et al., *Brain-to-Text Benchmark '24: Lessons Learned*, 2024. DOI: 10.48550/arXiv.2412.17227 arXiv: 2412.17227 [cs.CL].

[30] N. S. Card, F. R. Willett, S. D. Stavisky y D. M. Brandman, *Brain-to-Text Benchmark '25*, Competición y conjunto de datos públicos, Datos depositados en Dryad, https://doi.org/10.5061/dryad.dncjsxm85, 2025. visitado 4 de sep. de 2026. dirección: https://www.kaggle.com/competitions/brain-to-text-25

[31] T. J. Oxley et al., «Motor Neuroprosthesis Implanted with Neurointerventional Surgery Improves Capacity for Activities of Daily Living Tasks in Severe Paralysis: First In-Human Experience,» *Journal of NeuroInterventional Surgery*, vol. 13, n.º 2, págs. 102-108, 2021. DOI: 10.1136/neurintsurg-2020-016862

[32] Y. Peng et al., «Reproducing Whisper-Style Training Using an Open-Source Toolkit and Publicly Available Data,» en *IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023. DOI: 10.1109/ASRU57964.2023.10389676

[33] Y. Peng et al., «OWSM v3.1: Better and Faster Open Whisper-Style Speech Models Based on E-Branchformer,» en *Proc. Interspeech 2024*, 2024, págs. 352-356. DOI: 10.21437/Interspeech.2024-1194

[34] S. Watanabe et al., «ESPnet: End-to-End Speech Processing Toolkit,» en *Proc. Interspeech 2018*, 2018, págs. 2207-2211. DOI: 10.21437/Interspeech.2018-1456

[35] M. Someki et al., «ESPnet-EZ: Python-Only ESPnet for Easy Fine-Tuning and Integration,» en *IEEE Spoken Language Technology Workshop (SLT)*, 2024. DOI: 10.1109/SLT61566.2024.10832299

[36] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey e I. Sutskever, «Robust Speech Recognition via Large-Scale Weak Supervision,» en *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ép. Proceedings of Machine Learning Research, vol. 202, PMLR, 2023, págs. 28 492-28 518. visitado 4 de sep. de 2026. dirección: https://proceedings.mlr.press/v202/radford23a.html

[37] C. Fan et al., «Plug-and-Play Stability for Intracortical Brain-Computer Interfaces: A One-Year Demonstration of Seamless Brain-to-Text Communication,» en *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023. visitado 7 de nov. de 2026. dirección: https://proceedings.neurips.cc/paper_files/paper/2023/hash/83a14a36de4502bac5b580db36e81858-Abstract.html

[38] E. J. Hu et al., «LoRA: Low-Rank Adaptation of Large Language Models,» en *International Conference on Learning Representations (ICLR)*, 2022. visitado 4 de sep. de 2026. dirección: https://openreview.net/forum?id=nZeVKeeFYf9

[39] E. Feghhi, J. Hu, N. Hadidi y J. C. Kao, *LightBeam: An Accurate and Memory-Efficient CTC Decoder for Speech Neuroprostheses*, 2026. DOI: 10.48550/arXiv.2603.14002 arXiv: 2603.14002 [eess.AS].

[40] H. Jo, Y. Yang, J. Han, Y. Duan, H. Xiong y W. H. Lee, *Are EEG-to-Text Models Working?* 2024. DOI: 10.48550/arXiv.2405.06459 arXiv: 2405.06459.

[41] M. Verwoert et al., «Dataset of Speech Production in Intracranial Electroencephalography,» *Scientific Data*, vol. 9, 2022, **artno** 434. DOI: 10.1038/s41597-022-01542-9

[42] J. Kohler et al., «Synthesizing Speech from Intracranial Depth Electrodes Using an Encoder-Decoder Framework,» *Neurons, Behavior, Data Analysis, and Theory*, vol. 6, 2022. DOI: 10.51628/001c.57524

[43] Y. Zhang et al., «A Cross-Species Neural Foundation Model for End-to-End Speech Decoding,» en *International Conference on Learning Representations (ICLR)*, 2026. arXiv: 2511.21740.

[44] D. Sussillo, S. D. Stavisky, J. C. Kao, S. I. Ryu y K. V. Shenoy, «Making Brain–Machine Interfaces Robust to Future Neural Variability,» *Nature Communications*, vol. 7, 2016, **artno** 13749. DOI: 10.1038/ncomms13749

[45] T. Boccato, M. Olak y M. Ferrante, *BrainWhisperer: Leveraging Large-Scale ASR Models for Neural Speech Decoding*, 2026. DOI: 10.48550/arXiv.2603.13321 arXiv: 2603.13321 [eess.AS].

[46] O. M. Khanday, J. A. Gonzalez-Lopez, M. Ouellet, A. Galdón y G. Olivares Granados, «End-to-End Intracortical Speech Decoding from Neural Activity,» en *Proc. Odyssey 2026: The Speaker and Language Recognition Workshop*, 2026. DOI: 10.21437/Odyssey.2026-24

[47] D. Jurafsky y J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*, 3.ª ed. 2026, Borrador en línea, versión del 19 de agosto de 2026. visitado 8 de nov. de 2026. dirección: https://web.stanford.edu/~jurafsky/slp3/

[48] A. Baevski, H. Zhou, A. Mohamed y M. Auli, «wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,» en *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, págs. 12 449-12 460.

[49] M. Kim, Z. Piao, J. Lee y H.-G. Kang, *BrainTalker: Low-Resource Brain-to-Speech Synthesis with Transfer Learning Using Wav2Vec 2.0*, 2023. DOI: 10.48550/arXiv.2312.13600 arXiv: 2312.13600 [eess.AS].

[50] B. A. Yuan y J. G. Makin, *Improving Speech Decoding from ECoG with Self-Supervised Pretraining*, 2024. DOI: 10.48550/arXiv.2405.18639 arXiv: 2405.18639 [eess.AS].

[51] T. Fiedler et al., *Teaching Wav2Vec2 the Language of the Brain*, 2025. DOI: 10.48550/arXiv.2501.09459 arXiv: 2501.09459 [cs.LG].

[52] I. Vales Cortina et al., «UGR-MINDVOICE: A Multimodal EEG-Audio Dataset for Overt and Covert Iberian Spanish Speech Production,» *Computer Speech & Language*, vol. 100, 2026, **artno** 101964. DOI: 10.1016/j.csl.2026.101964

[53] J. J. Godfrey, E. C. Holliman y J. McDaniel, «SWITCHBOARD: Telephone Speech Corpus for Research and Development,» en *Proc. IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 1, 1992, págs. 517-520. DOI: 10.1109/ICASSP.1992.225858

[54] IEEE, «IEEE Recommended Practice for Speech Quality Measurements,» *IEEE Transactions on Audio and Electroacoustics*, vol. 17, n.º 3, págs. 225-246, 1969. DOI: 10.1109/TAU.1969.1162058

[55] D. Hendrycks y K. Gimpel, *Gaussian Error Linear Units (GELUs)*, 2016. arXiv: 1606.08415.

[56] J. L. Ba, J. R. Kiros y G. E. Hinton, *Layer Normalization*, 2016. arXiv: 1607.06450.

[57] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov, «Dropout: A Simple Way to Prevent Neural Networks from Overfitting,» *Journal of Machine Learning Research*, vol. 15, n.º 56, págs. 1929-1958, 2014.

[58] K. He, X. Zhang, S. Ren y J. Sun, «Deep Residual Learning for Image Recognition,» en *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, págs. 770-778. DOI: 10.1109/CVPR.2016.90

[59] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017, págs. 5998-6008.

[60] T. Kudo y J. Richardson, «SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,» en *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2018, págs. 66-71. DOI: 10.18653/v1/D18-2012

[61] A. Gulati et al., «Conformer: Convolution-Augmented Transformer for Speech Recognition,» en *Proc. Interspeech 2020*, 2020, págs. 5036-5040. DOI: 10.21437/Interspeech.2020-3015

[62] Y. Peng, S. Dalmia, I. Lane y S. Watanabe, «Branchformer: Parallel MLP-Attention Architectures to Capture Local and Global Context for Speech Recognition and Understanding,» en *Proc. 39th International Conference on Machine Learning (ICML)*, ép. PMLR, vol. 162, 2022, págs. 17 627-17 643.

[63] K. Kim et al., «E-Branchformer: Branchformer with Enhanced Merging for Speech Recognition,» en *IEEE Spoken Language Technology Workshop (SLT)*, 2023, págs. 84-91. DOI: 10.1109/SLT54892.2023.10022656

[64] A. Graves, S. Fernández, F. Gomez y J. Schmidhuber, «Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,» en *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, 2006, págs. 369-376. DOI: 10.1145/1143844.1143891

[65] A. Hannun, «Sequence Modeling with CTC,» *Distill*, 2017. DOI: 10.23915/distill.00008 dirección: https://distill.pub/2017/ctc

[66] S. Kim, T. Hori y S. Watanabe, «Joint CTC-Attention Based End-to-End Speech Recognition Using Multi-Task Learning,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, págs. 4835-4839. DOI: 10.1109/ICASSP.2017.7953075

[67] S. Watanabe, T. Hori, S. Kim, J. R. Hershey y T. Hayashi, «Hybrid CTC/Attention Architecture for End-to-End Speech Recognition,» *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, n.º 8, págs. 1240-1253, 2017. DOI: 10.1109/JSTSP.2017.2763455

[68] J. Lee y S. Watanabe, «Intermediate Loss Regularization for CTC-Based Speech Recognition,» en *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2021. arXiv: 2102.03216.

[69] J. Yosinski, J. Clune, Y. Bengio y H. Lipson, «How Transferable Are Features in Deep Neural Networks?» En *Advances in Neural Information Processing Systems 27 (NeurIPS 2014)*, 2014, págs. 3320-3328.

[70] X. Glorot e Y. Bengio, «Understanding the Difficulty of Training Deep Feedforward Neural Networks,» en *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ép. Proceedings of Machine Learning Research, vol. 9, 2010, págs. 249-256.

[71] D. P. Kingma y J. Ba, «Adam: A Method for Stochastic Optimization,» en *International Conference on Learning Representations (ICLR)*, 2015. arXiv: 1412.6980.

[72] I. Loshchilov y F. Hutter, «Decoupled Weight Decay Regularization,» en *International Conference on Learning Representations (ICLR)*, 2019. arXiv: 1711.05101.

[73] R. Pascanu, T. Mikolov e Y. Bengio, «On the Difficulty of Training Recurrent Neural Networks,» en *Proceedings of the 30th International Conference on Machine Learning (ICML)*, ép. Proceedings of Machine Learning Research, vol. 28, 2013, págs. 1310-1318.

[74] L. Prechelt, «Early Stopping — But When?» En *Neural Networks: Tricks of the Trade*, ép. Lecture Notes in Computer Science, G. B. Orr y K.-R. Müller, eds., vol. 1524, Springer, 1998, págs. 55-69. DOI: 10.1007/3-540-49430-8_3

[75] V. I. Levenshtein, «Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,» *Soviet Physics Doklady*, vol. 10, n.º 8, págs. 707-710, 1966.

[76] M. Bisani y H. Ney, «Bootstrap Estimates for Confidence Intervals in ASR Performance Evaluation,» en *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 1, Montreal, Canada, 2004, págs. 409-412. DOI: 10.1109/ICASSP.2004.1326009

[77] M. Honnibal, I. Montani, S. Van Landeghem y A. Boyd, *spaCy: Industrial-strength Natural Language Processing in Python*, 2020. DOI: 10.5281/zenodo.1212303

[78] E. Feghhi, S. Kaasyap, N. Hadidi y J. C. Kao, «Time-Masked Transformers with Lightweight Test-Time Adaptation for Neural Speech Decoding,» en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025. arXiv: 2507.02800.

[79] S. Roth, «Efficient End-to-End Brain-to-Text Decoding via Neural Pretraining,» Diplomarbeit, Technische Universität Wien, Viena, 2026. DOI: 10.34726/hss.2026.141781

[80] Y. Peng et al., «OWSM v4: Improving Open Whisper-Style Speech Models via Data Scaling and Cleaning,» en *Proc. Interspeech 2025*, 2025. arXiv: 2506.00338.

[81] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov y A. Mohamed, «HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units,» *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, págs. 3451-3460, 2021. DOI: 10.1109/TASLP.2021.3122291

[82] S. Chen et al., «WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing,» *IEEE Journal of Selected Topics in Signal Processing*, vol. 16, n.º 6, págs. 1505-1518, 2022. DOI: 10.1109/JSTSP.2022.3188113

APÉNDICE A

Código del pipeline de entrenamiento

APÉNDICE B

Análisis exploratorio adicional

APÉNDICE C

Relación del trabajo con los Objetivos de Desarrollo Sostenible de la Agenda 2030

Objetivos de Desarrollo Sostenible	Alto	Medio	Bajo	No procede
ODS 1. Fin de la pobreza.				
ODS 2. Hambre cero.				
ODS 3. Salud y bienestar.				
ODS 4. Educación de calidad.				
ODS 5. Igualdad de género.				
ODS 6. Agua limpia y saneamiento.				
ODS 7. Energía asequible y no contaminante.				
ODS 8. Trabajo decente y crecimiento económico.				
ODS 9. Industria, innovación e infraestructuras.				
ODS 10. Reducción de las desigualdades.				
ODS 11. Ciudades y comunidades sostenibles.				
ODS 12. Producción y consumo responsables.				
ODS 13. Acción por el clima.				
ODS 14. Vida submarina.				
ODS 15. Vida de ecosistemas terrestres.				
ODS 16. Paz, justicia e instituciones sólidas.				
ODS 17. Alianzas para lograr objetivos.				

Tabla C.1: Grado de relación del trabajo con los Objetivos de Desarrollo Sostenible.

Descripción de la alineación del TFG con los ODS con un grado de relación más alto

[Pendiente de redactar.]